

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG

HUỲNH LÊ ĐẠI THẮNG – 23521422
NGUYỄN TRƯỜNG DUY – 23520380

BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

PHÁT TRIỂN HỆ THỐNG QUẢN LÝ TÀI LIỆU BẢO
MẬT DỰA TRÊN KIẾN TRÚC MICROSERVICES VÀ
QUY TRÌNH DEVSECOPS

Development of a Secure Document Management
System based on Microservices Architecture and
DevSecOps Pipeline

Giảng viên hướng dẫn

ThS. Lê Anh Tuấn

TP. Hồ Chí Minh, tháng 06 năm 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH & TRUYỀN THÔNG



HUỲNH LÊ ĐẠI THẮNG – 23521422
NGUYỄN TRƯỜNG DUY – 23520380

BÁO CÁO ĐỒ ÁN CHUYÊN NGÀNH

PHÁT TRIỂN HỆ THỐNG QUẢN LÝ TÀI LIỆU BẢO
MẬT DỰA TRÊN KIẾN TRÚC MICROSERVICES VÀ
QUY TRÌNH DEVSECOPS

**Development of a Secure Document Management
System based on Microservices Architecture and
DevSecOps Pipeline**

Giảng viên hướng dẫn

ThS. Lê Anh Tuấn

TP. Hồ Chí Minh, tháng 06 năm 2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án chuyên ngành với đề tài “Phát triển hệ thống quản lý tài liệu bảo mật dựa trên kiến trúc Microservices và quy trình DevSecOps”, nhóm đã có dịp đưa kiến thức về phát triển phần mềm, kiến trúc microservices, bảo mật ứng dụng và tự động hóa triển khai vào một hệ thống tương đối hoàn chỉnh. Quá trình này có không ít khó khăn, từ giới hạn tài nguyên và chi phí cho đến các vấn đề phát sinh khi triển khai và vận hành hệ thống. Vượt qua những trở ngại đó là một phần đáng giá của trải nghiệm, giúp nhóm trưởng thành hơn trong cách xây dựng và vận hành một hệ thống thực tế.

Nhóm xin gửi lời cảm ơn đến thầy ThS. Lê Anh Tuấn đã định hướng, góp ý và hỗ trợ nhóm trong quá trình xác định phạm vi, thiết kế kiến trúc và hoàn thiện sản phẩm. Những góp ý trong quá trình thực hiện giúp nhóm nhìn nhận rõ hơn mối liên hệ giữa yêu cầu nghiệp vụ, yêu cầu bảo mật và khả năng vận hành của hệ thống.

Nhóm cũng xin cảm ơn nhà trường và khoa đã tạo điều kiện để nhóm được thực hiện đồ án theo hướng thực hành, qua đó rèn luyện kỹ năng xây dựng hệ thống, kiểm thử, triển khai và trình bày kết quả kỹ thuật một cách có hệ thống. Mặc dù đã cố gắng hoàn thiện, báo cáo và sản phẩm vẫn có thể còn thiếu sót. Nhóm mong nhận được góp ý để tiếp tục cải thiện hệ thống trong các giai đoạn sau.

TÓM TẮT

DocVault là hệ thống quản lý tài liệu bảo mật được xây dựng theo kiến trúc microservices và đi kèm quy trình DevSecOps. Đề tài vừa giải quyết bài toán nghiệp vụ tài liệu có vòng đời, quyền truy cập và bằng chứng xử lý, vừa kiểm soát cách hệ thống được phát hành từ mã nguồn đến môi trường triển khai.

Ở phần ứng dụng, hệ thống tập trung vào việc quản lý tài liệu từ lúc tạo mới, upload phiên bản, gửi duyệt, phê duyệt, xuất bản, truy cập, tải xuống cho đến retention hoặc archive. Mỗi tài liệu không chỉ được lưu dưới dạng file, mà còn có metadata, version file, trạng thái workflow, quyền truy cập, lịch sử phê duyệt và audit log. Nhờ đó, khi cần kiểm tra lại, hệ thống có thể cho biết tài liệu đã đi qua những bước nào, ai đã thao tác và kết quả xử lý ra sao.

Đối với các tài liệu có tính nhạy cảm, việc chỉ lưu file và phân quyền cơ bản là chưa đủ. Vì vậy, DocVault tích hợp các cơ chế như RBAC, ACL, classification, kiểm soát preview/download ở backend, Compliance Officer metadata-only, grant token ngắn hạn, malware scanning, DLP và audit hash-chain. Các cơ chế này giúp hệ thống kiểm soát quyền truy cập, bảo vệ nội dung file và ghi nhận lại các hoạt động quan trọng liên quan đến tài liệu.

Bên cạnh truy vết vòng đời tài liệu, hệ thống còn ghi nhận một số hoạt động bảo mật và vận hành để hỗ trợ quản trị viên kiểm tra hành vi bất thường và xử lý khuyến nghị bảo mật.

Ở phần DevSecOps, nhóm xây dựng Jenkins pipeline để kiểm soát chuỗi phát hành từ code, container artifact, GitOps revision đến workload trên EKS. Các bằng chứng như kết quả scan, image digest, SBOM, trạng thái ký, Argo CD health, smoke test và ZAP report được sử dụng để đối chiếu lại từng lần triển khai.

Kết quả của đề tài cho thấy DocVault đã triển khai được các luồng chính của một hệ thống quản lý tài liệu bảo mật, đồng thời xây dựng được quy trình DevSecOps có khả năng kiểm tra và truy vết quá trình triển khai. Tuy nhiên, hệ thống vẫn còn một số điểm cần hoàn thiện thêm nếu hướng đến môi trường production, chẳng hạn như NetworkPolicy chi tiết, admission policy, provenance chuẩn hóa, kiểm thử trình duyệt đầy đủ và diễn tập khôi phục dữ liệu.

Từ khóa: DocVault, quản lý tài liệu, bảo mật ứng dụng, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

ABSTRACT

DocVault is a secure document management system built with a microservices architecture and a DevSecOps delivery process. The project covers both the controlled lifecycle of sensitive documents and the traceable delivery path from source code to the deployed environment.

In the application part, DocVault manages documents from creation, version upload, review submission, approval, publication, access, download, retention, and archiving. A document is not treated only as a stored file, but as an object with metadata, file versions, workflow status, access permissions, approval history, and audit logs. These records help the system review how a document has been handled over time.

For sensitive documents, simple file storage and basic authorization are not enough. Therefore, DocVault integrates mechanisms such as RBAC, ACL, classification, backend-based preview/download control, a metadata-only Compliance Officer boundary, short-lived grant tokens, malware scanning, DLP, and audit hash-chain. These mechanisms help control document access, protect file content, and record important activities related to documents.

Beyond document lifecycle traceability, the system also records selected security and operational activities to help administrators review unusual behavior and handle security recommendations.

In the DevSecOps part, the project uses a Jenkins pipeline to control the delivery path from code, container artifacts, and GitOps revisions to workloads running on EKS. Evidence such as scan results, image digests, SBOM, signing status, Argo CD health, smoke tests, and ZAP reports is used to review each deployment.

The result shows that DocVault has implemented the main workflows of a secure document management system and a DevSecOps process that supports controlled delivery and deployment traceability. Some production-grade improvements are still needed, such as fine-grained NetworkPolicy, admission policy, standardized provenance, full browser-based E2E testing, and data recovery drills.

Keywords: DocVault, document management, application security, microservices, DevSecOps, RBAC, ACL, audit, DLP, Jenkins, GitOps.

MỤC LỤC

Lời cảm ơn	i
Tóm tắt	ii
Abstract	iii
Danh mục từ viết tắt	xi
1 Giới thiệu đề tài	1
1.1 Lý do chọn đề tài	1
1.2 Vấn đề cần giải quyết	2
1.3 Mục tiêu đề tài	3
1.4 Đối tượng và phạm vi thực hiện	4
1.5 Phương pháp thực hiện	6
1.6 Đóng góp của đề tài	6
1.7 Phân công công việc	6
1.8 Cấu trúc báo cáo	7
2 Cơ sở lý thuyết	9
2.1 Hệ thống quản lý tài liệu bảo mật	9
2.2 Khảo sát các hệ thống tương tự	9
2.3 Kiến trúc microservices	11
2.4 Xác thực và phân quyền	12
2.5 Phân loại tài liệu và kiểm soát nội dung	14
2.6 Audit, hash-chain và compliance evidence	14
2.7 DevSecOps	15
2.8 Các nhóm kiểm thử bảo mật trong pipeline	16
2.9 GitOps và Argo CD	16
2.10 Tổng hợp công nghệ và công cụ	17
3 Phân tích yêu cầu và thiết kế hệ thống	20
3.1 Tổng quan bài toán	20
3.2 Tác nhân hệ thống	21
3.3 Yêu cầu chức năng	22
3.4 Yêu cầu phi chức năng	24
3.5 Mô hình threat cơ bản	24
3.6 Kiến trúc tổng thể	25
3.7 Luồng nghiệp vụ chính	28

3.8	Nguyên tắc thiết kế	29
3.9	Tổng quan thiết kế dữ liệu	30
3.10	Các thực thể chính	32
3.11	Document lifecycle	33
3.12	Ma trận phân quyền	35
3.13	Chính sách Compliance Officer	37
3.14	Thiết kế preview/download authorization	37
3.15	Luồng phê duyệt tài liệu	39
3.16	Malware scanning và DLP	39
3.17	Audit ingest boundary	40
3.18	Ghi nhận hoạt động hệ thống và khuyến nghị bảo mật	40
3.19	Luồng compliance evidence	41
3.20	API contract tổng quan	42
4	Hiện thực và triển khai hệ thống	44
4.1	Hiện thực ứng dụng DocVault	44
4.2	Hạ tầng triển khai trên AWS và EKS	52
4.3	Quy trình DevSecOps với GitOps và CI/CD	67
4.4	Kiểm soát bảo mật trong pipeline	73
4.5	Vận hành, giám sát và khôi phục	81
4.6	Đánh giá trạng thái triển khai	84
5	Kiểm thử và đánh giá hệ thống	87
5.1	Chiến lược kiểm thử	87
5.2	Kiểm thử runtime	87
5.3	Kiểm thử giao diện và bằng chứng UI	89
5.4	Bằng chứng kiểm thử bảo mật	95
5.5	Bằng chứng quy trình pipeline	97
5.6	Đánh giá theo yêu cầu	98
5.7	Hạn chế hiện tại	98
5.8	Nhận xét chung	99
6	Kết luận và hướng phát triển	100
6.1	Kết quả đạt được	100
6.2	Mức độ đáp ứng mục tiêu	101
6.3	Hạn chế	101
6.4	Hướng phát triển	102
6.5	Kết luận chung	102
A	Hướng dẫn chạy hệ thống	104
A.1	Yêu cầu môi trường	104
A.2	Cài đặt phụ thuộc	104
A.3	Khởi động hạ tầng local	104

A.4	Chạy migration	104
A.5	Khởi động backend	105
A.6	Khởi động frontend	105
A.7	Tài khoản demo	105
A.8	Kiểm thử nhanh	106
B	Tóm tắt API contract	107
B.1	Nguyên tắc chung	107
B.2	Nhóm endpoint chính	107
B.3	Compliance Officer rule	108
B.4	Grant token	108

DANH SÁCH HÌNH

2.1	Ví dụ về cách Keycloak hoạt động với Spring Boot Gateway	13
2.2	Mô hình kết hợp phát triển, bảo mật và vận hành trong DevSecOps	15
2.3	Luồng triển khai GitOps với Argo CD	17
3.1	Sơ đồ use case tổng thể của DocVault theo tác nhân	22
3.2	Kiến trúc tổng thể ứng dụng DocVault	27
3.3	Luồng nghiệp vụ demo chính theo vai trò trong DocVault	29
3.4	ERD và ranh giới sở hữu dữ liệu trong DocVault	31
3.5	Vòng đời trạng thái tài liệu trong DocVault	34
3.6	Sequence cấp quyền download và stream file	38
3.7	Sequence luồng phê duyệt tài liệu (submit và approve)	39
3.8	Luồng tổng hợp compliance evidence trong DocVault	41
4.1	Màn hình dashboard của DocVault	45
4.2	Màn hình danh sách tài liệu	46
4.3	Màn hình rà soát quyền truy cập	49
4.4	Màn hình phê duyệt tài liệu	50
4.5	Màn hình audit log	50
4.6	Màn hình security posture	51
4.7	Luồng tổng quan CI, CD theo GitOps và kiểm tra sau triển khai	53
4.8	Luồng triển khai hạ tầng AWS EKS bằng IaC	54
4.9	EKS cluster <code>docvault-eks</code> đang hoạt động trên khu vực Singapore	55
4.10	Managed node group và các worker node ở trạng thái sẵn sàng	56
4.11	Resource map của VPC gồm public/private subnet trên hai Availability Zone	56
4.12	Kiến trúc triển khai DocVault trên AWS EKS	57
4.13	S3 bucket lưu tài liệu đã bật versioning trong môi trường testing	59
4.14	S3 bucket tài liệu được Terraform quản lý và mã hóa mặc định bằng SSE-KMS	59
4.15	Trạng thái backup CloudNativePG metadata với base backup hằng ngày và WAL đang được archive lên S3	60
4.16	Trạng thái Percona MongoDB replica set audit với một PRIMARY và hai SECONDARY	60
4.17	Trạng thái backup và PITR của PBM	61
4.18	Các customer-managed KMS key dùng cho EKS, tài liệu và backup	61
4.19	S3 bucket lưu backup cơ sở dữ liệu	61
4.20	Cấu hình bản ghi DNS của DocVault trở tên miền dịch vụ về Load Balancer hoặc Tunnel trên Cloudflare	63

4.21	Trạng thái hoạt động của Cloudflare Tunnel trên trang quản trị Cloudflare Zero Trust	64
4.22	Application route của Cloudflare Tunnel cho Argo CD, Grafana, Harbor trong cụm EKS	64
4.23	Danh sách workload trên cụm EKS và trạng thái vận hành tại thời điểm thu thập bằng chứng	65
4.24	Các IAM role cho document-service, External Secrets và tiến trình backup qua IRSA	66
4.25	Các secret của DocVault được quản lý tập trung trong AWS Secrets Manager	66
4.26	Trạng thái đồng bộ Secret của External Secrets Operator (ESO)	66
4.27	Application gốc <code>docvault-root</code> đạt trạng thái Healthy và Synced	67
4.28	Hình chụp dashboard ArgoCD hiển thị các ứng dụng (Ảnh 1)	68
4.29	Hình chụp dashboard ArgoCD hiển thị các ứng dụng (Ảnh 2)	68
4.30	Project riêng <code>docvault-dev</code> và các repository dịch vụ trên Harbor	70
4.31	Artifact Harbor được định danh bằng digest, có trạng thái ký và thông tin SBOM	70
4.32	Chữ ký Cosign và SBOM của artifact trên Harbor	70
4.33	Jenkins build và Stage View	72
4.34	SonarQube Quality Gate của DocVault	73
4.35	Log và artifact cho ba bước IaC	74
4.36	Kết quả Terraform Validate	74
4.37	Kết quả Checkov	75
4.38	Kết quả Kyverno	75
4.39	Repo Harbor có tên <code>docvault-cache</code> , dùng để lưu cache của các image . . .	76
4.40	Các image cache được đánh dấu bằng chữ C ở cột artifact, có dung lượng lớn do chứa cả các lớp build trung gian.	76
4.41	Kết quả Argo CD health check	78
4.42	Log post deploy smoke test	79
4.43	Log OWASP ZAP	80
4.44	Kết quả OWASP ZAP	80
4.45	Grafana dashboard giám sát workload DocVault trong namespace <code>docvault</code>	82
4.46	Grafana dashboard giám sát EKS cluster	82
4.47	Loki tập trung log của namespace DocVault	83
5.1	Bằng chứng giao diện dashboard command center	90
5.2	Bằng chứng giao diện hàng đợi phê duyệt và trạng thái SLA	90
5.3	Bằng chứng trang đăng nhập ứng dụng DocVault	91
5.4	Bằng chứng trang xác thực Keycloak	91
5.5	Bằng chứng quản trị user trong realm Keycloak	92
5.6	Bằng chứng danh sách tài liệu của người dùng	92
5.7	Bằng chứng tạo tài liệu mới và upload file	93
5.8	Bằng chứng chi tiết tài liệu và metadata	93

5.9	Bảng chứng preview tài liệu qua luồng kiểm soát	94
5.10	Bảng chứng giao diện Evidence Center	94
5.11	Bảng chứng giao diện Retention và trạng thái lưu giữ	95

DANH SÁCH BẢNG

1.1	Phạm vi và trạng thái thực hiện của đề tài	5
1.2	Phân công công việc giữa các thành viên	7
2.1	So sánh DocVault với một số hệ thống quản lý tài liệu tiêu biểu	11
2.2	Các nhóm kiểm thử bảo mật trong DevSecOps pipeline	16
2.3	Công nghệ xây dựng ứng dụng DocVault	18
2.4	Công cụ DevSecOps và hạ tầng triển khai	19
3.1	Các tác nhân chính trong DocVault	21
3.2	Yêu cầu phi chức năng	24
3.3	Mô hình threat của DocVault ánh xạ theo STRIDE	25
3.4	Quyền xem và tải nội dung file theo vai trò và classification	36
3.5	Nhóm API chính của DocVault	42
4.1	Mapping kiểm soát DevSecOps với công cụ và bằng chứng	84
5.1	Tóm tắt bằng chứng kiểm thử runtime	88
5.2	Kiểm chứng audit event cho hoạt động bảo mật và vận hành	96
5.3	Mapping yêu cầu với hiện thực và bằng chứng	98
B.1	Tóm tắt endpoint chính	107

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Diễn giải trong báo cáo
DMS	Document Management System	Hệ thống quản lý tài liệu.
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò.
ACL	Access Control List	Danh sách quyền truy cập theo người dùng, vai trò, nhóm hoặc tất cả.
OIDC	OpenID Connect	Lớp định danh dùng trên OAuth 2.0, được Keycloak sử dụng cho đăng nhập.
JWT	JSON Web Token	Token mang thông tin định danh và vai trò của người dùng.
CI/CD	Continuous Integration / Continuous Delivery	Tích hợp, kiểm thử, đóng gói và triển khai tự động.
DevSecOps	Development, Security, Operations	Tích hợp bảo mật vào quy trình phát triển và vận hành.
SAST	Static Application Security Testing	Kiểm thử bảo mật bằng phân tích mã nguồn tĩnh.
SCA	Software Composition Analysis	Phân tích thành phần phụ thuộc và lỗ hổng thư viện.
DAST	Dynamic Application Security Testing	Kiểm thử bảo mật động trên ứng dụng đang chạy.
IaC	Infrastructure as Code	Mô tả hạ tầng bằng mã, ví dụ Terraform/Kubernetes manifest.
SBOM	Software Bill of Materials	Danh mục thành phần phần mềm và phụ thuộc.
DLP	Data Loss Prevention	Cơ chế phát hiện và giảm rủi ro rò rỉ dữ liệu nhạy cảm.
EKS	Elastic Kubernetes Service	Dịch vụ Kubernetes được quản lý trên AWS.
GitOps	Git Operations	Mô hình dùng Git làm nguồn sự thật cho trạng thái triển khai.
API	Application Programming Interface	Giao diện lập trình ứng dụng giữa frontend, gateway và các service.

AWS	Amazon Web Services	Nền tảng điện toán đám mây dùng để triển khai hạ tầng đề tài.
ERD	Entity Relationship Diagram	Sơ đồ quan hệ thực thể mô tả dữ liệu và ranh giới sở hữu.
VPC	Virtual Private Cloud	Mạng riêng ảo trên AWS chứa các subnet của cụm EKS.
NAT	Network Address Translation	Cổng cho phép node trong private subnet truy cập Internet ra ngoài.
CIDR	Classless Inter-Domain Routing	Cách biểu diễn dải địa chỉ IP, dùng giới hạn truy cập Kubernetes API.
TLS	Transport Layer Security	Giao thức mã hóa kênh truyền cho lưu lượng HTTPS.
S3	Simple Storage Service	Dịch vụ lưu trữ object của AWS dùng cho file tài liệu và backup.
KMS	Key Management Service	Dịch vụ quản lý khóa mã hóa của AWS.
SSE-KMS	Server-Side Encryption with KMS	Mã hóa phía máy chủ cho object S3 bằng khóa KMS.
IRSA	IAM Roles for Service Accounts	Cấp quyền AWS cho từng Kubernetes ServiceAccount qua OIDC.
IAM	Identity and Access Management	Dịch vụ quản lý định danh và quyền truy cập của AWS.
IMDSv2	Instance Metadata Service version 2	Phiên bản metadata service yêu cầu token, giảm rủi ro SSRF trên EC2.
CVE	Common Vulnerabilities and Exposures	Định danh chuẩn cho lỗ hổng bảo mật đã được công bố.
CVSS	Common Vulnerability Scoring System	Hệ thống chấm điểm mức độ nghiêm trọng của lỗ hổng.
SLSA	Supply-chain Levels for Software Artifacts	Khung chuẩn hóa provenance của artifact trong chuỗi cung ứng.
RPO	Recovery Point Objective	Mức dữ liệu tối đa chấp nhận mất khi khôi phục sau sự cố.
RTO	Recovery Time Objective	Thời gian tối đa chấp nhận để khôi phục dịch vụ sau sự cố.
MVP	Minimum Viable Product	Sản phẩm khả dụng tối thiểu đủ để minh họa nghiệp vụ cốt lõi.

CHƯƠNG 1

GIỚI THIỆU ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong quá trình làm việc của nhiều tổ chức, tài liệu nội bộ thường không chỉ được tạo ra rồi lưu trữ cố định. Một tài liệu có thể được cập nhật nhiều lần, gửi cho người khác duyệt, được phê duyệt, xuất bản, tải xuống, lưu trữ hoặc được kiểm tra lại sau một khoảng thời gian. Với các loại tài liệu như hợp đồng, biểu mẫu phê duyệt, tài liệu kỹ thuật, hồ sơ nhân sự hoặc tài liệu kiểm toán, việc chỉ lưu file trong một thư mục chung và phân quyền cơ bản thường chưa đủ để theo dõi toàn bộ quá trình xử lý.

Vấn đề này rõ hơn khi tài liệu có tính nhạy cảm. Khi cần kiểm tra lại một tài liệu, người quản trị không chỉ cần biết file đang được lưu ở đâu, mà còn cần biết ai đã tạo, ai đã cập nhật, tài liệu đã qua những phiên bản nào, ai đã phê duyệt, ai từng truy cập và hiện tại tài liệu đang ở trạng thái nào. Nếu hệ thống không lưu lại các thông tin này một cách có cấu trúc, việc truy vết sau này sẽ phụ thuộc nhiều vào thao tác thủ công hoặc dữ liệu rời rạc, khó đối chiếu.

Từ đó, nhóm chọn xây dựng DocVault theo hướng xem tài liệu như một đối tượng nghiệp vụ có vòng đời riêng, thay vì chỉ xem tài liệu như một file lưu trữ. Mỗi tài liệu trong hệ thống cần đi kèm metadata, version, trạng thái workflow, quyền truy cập, lịch sử phê duyệt và audit log. Khi thử các luồng như upload phiên bản mới, gửi duyệt, phê duyệt hoặc tải file, nhóm nhận thấy các quyết định quan trọng không nên chỉ nằm ở giao diện, vì người dùng vẫn có thể gọi trực tiếp API nếu backend không kiểm soát chặt.

Bên cạnh phần ứng dụng, nhóm cũng muốn quá trình triển khai hệ thống có thể kiểm tra lại được. Một hệ thống microservices thường gồm nhiều service, nhiều database, object storage, Kubernetes manifest, secret runtime và các thành phần hạ tầng liên quan. Nếu việc build image, scan, cập nhật manifest và triển khai lên Kubernetes được làm thủ công, quá trình phát hành dễ bị lệch cấu hình và khó xác định nguyên nhân khi có lỗi.

Vì vậy, đề tài “Phát triển hệ thống quản lý tài liệu bảo mật dựa trên kiến trúc Microservices và quy trình DevSecOps” được thực hiện theo hai hướng song song. Hướng thứ nhất là xây dựng ứng dụng DocVault để quản lý tài liệu có kiểm soát. Hướng thứ hai là xây dựng quy trình DevSecOps để việc phát hành hệ thống từ mã nguồn đến workload trên Kubernetes có log, artifact và bằng chứng đối chiếu.

1.2 Vấn đề cần giải quyết

Đề tài đặt ra hai câu hỏi chính. Thứ nhất, làm sao xây dựng một hệ thống quản lý tài liệu nhạy cảm mà vẫn kiểm tra lại được quá trình xử lý sau này. Thứ hai, làm sao phát hành hệ thống đó theo một quy trình đủ rõ để biết mã nguồn nào tạo ra image nào, image nào được triển khai và workload sau triển khai có đạt trạng thái mong muốn hay không.

Đối với phần ứng dụng, vấn đề chính là làm sao quản lý được toàn bộ vòng đời của một tài liệu. Một tài liệu có thể được tạo mới, upload phiên bản, cập nhật metadata, gửi duyệt, phê duyệt, từ chối, xuất bản, truy cập, tải xuống, lưu trữ hoặc đưa vào retention. Nếu chỉ lưu file và trạng thái hiện tại, hệ thống sẽ khó trả lời các câu hỏi như tài liệu này đã được xử lý qua những bước nào, ai đã thao tác, thao tác đó có hợp lệ không và có bằng chứng nào để kiểm tra lại.

Vì vậy, hệ thống cần quản lý đồng thời nhiều thông tin liên quan đến tài liệu, bao gồm metadata, version file, checksum, trạng thái workflow, quyền truy cập, mức phân loại bảo mật, lịch sử phê duyệt và audit log. Các thông tin này cần được liên kết với nhau để tạo thành một bức tranh đầy đủ về quá trình xử lý tài liệu. Khi cần kiểm tra lại, hệ thống phải có khả năng tổng hợp các dữ liệu này thành evidence để phục vụ việc đối chiếu.

Ngoài các thao tác gắn trực tiếp với vòng đời tài liệu, hệ thống còn cần lưu lại một số hoạt động khác trong quá trình vận hành. Ví dụ, khi người dùng xem audit log, bị từ chối quyền xem hoặc tải tài liệu, chạy retention job, thực hiện thao tác nhạy cảm, thay đổi thành viên trong tổ chức, sử dụng share link hoặc cập nhật trạng thái xử lý một khuyến nghị bảo mật, các sự kiện này cũng cần được ghi nhận lại.

Việc ghi nhận các hoạt động này giúp audit log không chỉ phản ánh lịch sử của từng tài liệu, mà còn cho thấy cách người dùng và các thành phần trong hệ thống tương tác với nhau. Khi có một hành vi cần kiểm tra lại, quản trị viên có thể dựa vào các sự kiện đã ghi nhận để xem hành động đó diễn ra khi nào, do ai thực hiện, kết quả xử lý ra sao và có cần tiếp tục điều tra hay không.

Đối với phần DevSecOps, vấn đề đặt ra là làm sao đưa một hệ thống nhiều service vào một quy trình triển khai có thể lặp lại và kiểm soát được. Nếu build image, cập nhật manifest, triển khai Kubernetes và kiểm tra trạng thái đều làm thủ công, quá trình phát hành sẽ dễ sai sót và khó truy vết. Do đó, hệ thống cần pipeline kiểm thử, đóng gói, phát hành artifact, cập nhật GitOps và kiểm tra lại môi trường sau triển khai.

Cụ thể, đề tài tập trung vào các yêu cầu sau:

- Quản lý vòng đời tài liệu với các trạng thái như DRAFT, PENDING, PUBLISHED và ARCHIVED.
- Quản lý metadata, version file, checksum, workflow history và trạng thái hiện tại của tài liệu.
- Kiểm soát quyền truy cập dựa trên RBAC, ACL, classification, trạng thái tài liệu

và quyền sở hữu.

- Ghi nhận audit log cho các thao tác quan trọng như tạo tài liệu, upload version, submit, approve, reject, preview, download, retention và compliance.
- Cung cấp evidence packet để tổng hợp metadata, version checksum, workflow history, retention evidence và audit event liên quan đến tài liệu.
- Kiểm soát preview/download thông qua backend policy và grant token ngắn hạn.
- Đảm bảo Compliance Officer chỉ xem metadata, audit và evidence phục vụ kiểm tra, không truy cập trực tiếp nội dung file.
- Xây dựng pipeline DevSecOps để kiểm soát chuỗi phát hành từ mã nguồn, container artifact, GitOps revision đến workload đang chạy, đồng thời thu thập bằng chứng vận hành.

1.3 Mục tiêu đề tài

Mục tiêu tổng quát của đề tài là xây dựng và triển khai DocVault như một hệ thống hoàn chỉnh trong phạm vi đồ án. Hệ thống cần có phần ứng dụng quản lý tài liệu bảo mật, có hạ tầng triển khai và có quy trình phát hành đủ rõ để kiểm tra lại khi cần.

Ở lớp ứng dụng, nhóm tập trung vào việc quản lý tài liệu theo vòng đời, kiểm soát truy cập ở backend và tạo audit/evidence cho các thao tác quan trọng. Ở lớp triển khai, nhóm xây dựng pipeline và hạ tầng cloud-native để mỗi lần phát hành có thể lần ngược từ commit, image digest, GitOps revision đến trạng thái workload trên Kubernetes.

Các mục tiêu cụ thể gồm:

1. Thiết kế kiến trúc tổng thể cho hệ thống DocVault, gồm frontend, API Gateway, các backend service, các thành phần lưu trữ dữ liệu và hạ tầng triển khai.
2. Xây dựng các chức năng quản lý vòng đời tài liệu như tạo tài liệu, cập nhật metadata, upload version file, gửi duyệt, phê duyệt, từ chối, xuất bản, lưu trữ và retention.
3. Thiết kế mô hình dữ liệu cho documents, document versions, ACL, workflow history, audit events và các thông tin evidence liên quan.
4. Xây dựng cơ chế kiểm soát truy cập dựa trên RBAC, ACL, trạng thái tài liệu, classification, quyền sở hữu và quy tắc deny-overrides.
5. Ghi nhận audit log cho các thao tác quan trọng trong vòng đời tài liệu và áp dụng hash-chain để hỗ trợ kiểm tra tính toàn vẹn của audit event.
6. Bổ sung cơ chế ghi nhận audit ở mức hệ thống cho các hoạt động bảo mật và vận hành ngoài các thao tác trực tiếp trên tài liệu, chẳng hạn như truy cập thành công hoặc bị từ chối, xem audit log, chạy retention job, thực hiện thao tác nhạy cảm, quản trị thành viên, thao tác comment/share link và xử lý khuyến nghị bảo mật. Các dữ liệu này được sử dụng để phục vụ truy vết, kiểm tra hành vi người dùng và

hỗ trợ quản trị viên đánh giá các hoạt động có dấu hiệu bất thường.

7. Xây dựng evidence packet để tổng hợp metadata, checksum phiên bản, workflow history, retention evidence và audit event, giúp kiểm tra lại quá trình quản lý tài liệu.
8. Tích hợp các cơ chế bảo mật như grant token ngắn hạn cho preview/download, Compliance Officer metadata-only boundary, malware scanning và DLP.
9. Xây dựng hạ tầng triển khai cloud-native trên AWS EKS với Terraform, Helm/Kustomize, S3/KMS, IRSA, External Secrets, Harbor và các thành phần quan sát vận hành.
10. Xây dựng pipeline DevSecOps với Jenkins, các cổng kiểm tra chất lượng/bảo mật và luồng GitOps triển khai lên Kubernetes.
11. Thu thập bằng chứng cho các luồng ứng dụng và chuỗi phát hành, bao gồm kiểm thử E2E, audit/evidence, retention, scan report, image digest, SBOM, signing status, GitOps revision, Argo CD health, smoke test và ZAP report.

1.4 Đối tượng và phạm vi thực hiện

1.4.1 Đối tượng thực hiện

Đối tượng thực hiện của đề tài là hệ thống DocVault, một hệ thống quản lý tài liệu bảo mật được xây dựng theo kiến trúc microservices và triển khai kèm quy trình DevSecOps. Hệ thống hướng đến nhóm tài liệu nội bộ cần quản lý vòng đời, phân quyền truy cập, phê duyệt, audit và bằng chứng xử lý.

Ở phần ứng dụng, các đối tượng chính gồm tài liệu, phiên bản tài liệu, metadata, workflow history, ACL, audit event, retention evidence và evidence packet. Hệ thống có các vai trò chính như Editor, Approver, Viewer, Compliance Officer và Admin. Mỗi vai trò tham gia vào một phần khác nhau của luồng xử lý, từ tạo tài liệu, duyệt tài liệu, truy cập nội dung đến kiểm tra audit/evidence.

Ở phần triển khai, đối tượng thực hiện gồm frontend, API Gateway, các backend service, database, object storage, Kubernetes workload, registry, GitOps repository, pipeline CI/CD và các công cụ kiểm soát bảo mật trong chuỗi phát hành.

1.4.2 Phạm vi thực hiện

Phạm vi đề tài gồm hai phần chính: phần ứng dụng DocVault và phần DevSecOps triển khai hệ thống. Phần ứng dụng bao gồm giao diện web, các service backend, luồng nghiệp vụ tài liệu, kiểm soát truy cập, audit, DLP, quét mã độc, retention và evidence center. Phần DevSecOps bao gồm hạ tầng AWS EKS được quản lý bằng Terraform, pipeline

Jenkins, Harbor registry, GitOps với Argo CD, các bước scan bảo mật và cơ chế quan sát sau triển khai.

Trong phạm vi đề án, nhóm tập trung chứng minh các luồng chính như tạo tài liệu, upload version, gửi duyệt, phê duyệt, xuất bản, preview/download có kiểm soát, ghi audit log, kiểm tra ranh giới quyền của Compliance Officer, xử lý DLP/malware scanning, retention và xuất evidence packet. Ở phần DevSecOps, nhóm tập trung vào các bước kiểm thử, scan, build image, push artifact, cập nhật GitOps, kiểm tra trạng thái Argo CD, smoke test, DAST và thu thập bằng chứng triển khai.

Bảng 1.1: Phạm vi và trạng thái thực hiện của đề tài

Nhóm nội dung	Mô tả	Trạng thái
Ứng dụng web	Frontend Next.js, các màn hình dashboard, documents, approvals, audit, security, evidence, retention.	Đã triển khai
Backend microservices	Gateway, metadata-service, document-service, workflow-service, audit-service, notification-service.	Đã triển khai
Bảo mật runtime	RBAC, ACL, grant token, compliance boundary, DLP, malware scanning, audit hash-chain.	Đã triển khai
Local infrastructure	Postgres, MongoDB, MinIO, Keycloak qua Docker Compose.	Đã triển khai
DevSecOps pipeline	Jenkins pipeline với các stage kiểm thử, quét bảo mật, build, Harbor, GitOps, Argo CD, smoke test và DAST.	Đã triển khai
Hạ tầng cloud-native	Terraform, EKS, S3/KMS, IRSA, External Secrets, backup, Harbor và observability.	Đã triển khai
Gia cố vận hành production	Remote Terraform state, private node, admission enforcement, provenance chuẩn hóa, NetworkPolicy giới hạn egress, log persistence và diễn tập khôi phục.	Đang hoàn thiện

1.5 Phương pháp thực hiện

Đề tài được thực hiện theo hướng kết hợp giữa khảo sát, thiết kế, hiện thực và kiểm chứng bằng bằng chứng triển khai. Trước hết, nhóm khảo sát một số hệ thống quản lý tài liệu và các công cụ DevSecOps liên quan để xác định phạm vi phù hợp cho DocVault. Từ đó, nhóm phân tích yêu cầu, xác định các luồng nghiệp vụ chính và thiết kế hệ thống theo kiến trúc microservices.

Sau giai đoạn thiết kế, nhóm hiện thực các thành phần chính của DocVault, bao gồm frontend, API Gateway, các service backend, database, object storage và các cơ chế bảo mật như RBAC, ACL, grant token, DLP, malware scanning và audit hash-chain. Song song với phần ứng dụng, nhóm xây dựng hạ tầng triển khai trên AWS EKS và pipeline DevSecOps để kiểm soát quá trình phát hành từ mã nguồn đến workload đang chạy.

Cuối cùng, nhóm kiểm thử hệ thống thông qua các luồng runtime, API, boundary bảo mật và pipeline. Các bằng chứng như log kiểm thử, scan report, image digest, SBOM, trạng thái ký, GitOps revision, Argo CD health, smoke test, ZAP report và ảnh chụp giao diện được dùng để đối chiếu với mục tiêu ban đầu và đánh giá mức độ hoàn thành của đề tài.

1.6 Đóng góp của đề tài

Đề tài có bốn đóng góp. Thứ nhất là một hệ thống quản lý tài liệu chạy được, bao phủ luồng từ khởi tạo, tải lên, phê duyệt đến lưu trữ. Thứ hai là mô hình kiểm soát truy cập nhiều lớp, kết hợp vai trò, ACL, mức phân loại, trạng thái và quyền sở hữu thay vì chỉ dựa vào một trong số đó. Thứ ba là cách đưa yêu cầu compliance vào ngay trong thiết kế: audit hash-chain để phát hiện sửa đổi, retention evidence, evidence packet metadata-only và ghi nhận các hoạt động bảo mật/vận hành phục vụ kiểm tra hành vi người dùng. Thứ tư là quy trình DevSecOps được nhóm đầu tư triển khai khá đầy đủ, có khả năng truy vết từ mã nguồn qua container artifact và GitOps revision đến workload trên EKS, với các cổng kiểm soát chất lượng, bảo mật chuỗi cung ứng, quản lý bí mật và giám sát vận hành gắn vào từng chặng.

1.7 Phân công công việc

Đề tài do hai thành viên cùng thực hiện theo hình thức đồng phụ trách: cả hai cùng tham gia khảo sát, thiết kế và kiểm thử toàn hệ thống, đồng thời mỗi người nhận vai trò chính ở một số hạng mục để bảo đảm tiến độ. Bảng 1.2 tóm tắt phân công theo vai trò chính và vai trò phối hợp.

Bảng 1.2: Phân công công việc giữa các thành viên

Hạng mục	Phụ trách chính	Phối hợp
Thiết kế kiến trúc tổng thể	Cả hai	—
Frontend Next.js và permission-aware UI	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Backend microservices và bảo mật runtime	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Mô hình phân quyền (RBAC, ACL, classification)	Huỳnh Lê Đại Thắng	Nguyễn Trường Duy
Audit hash-chain, DLP và evidence	Huỳnh Lê Đại Thắng	—
Pipeline DevSecOps (Jenkins, các security gate)	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Hạ tầng cloud-native (Terraform, EKS, GitOps)	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Quản lý secret, registry và observability	Nguyễn Trường Duy	Huỳnh Lê Đại Thắng
Kiểm thử, thu thập bằng chứng và viết báo cáo	Cả hai	—

1.8 Cấu trúc báo cáo

Báo cáo được tổ chức thành sáu chương chính:

Chương 1. Giới thiệu đề tài: Trình bày lý do chọn đề tài, vấn đề cần giải quyết, mục tiêu, đối tượng và phạm vi thực hiện, phương pháp thực hiện, đóng góp của đề tài và phân công công việc.

Chương 2. Cơ sở lý thuyết: Trình bày các khái niệm, mô hình bảo mật, hệ thống tương tự và các công nghệ được sử dụng trong DocVault.

Chương 3. Phân tích yêu cầu và thiết kế hệ thống: Phân tích tác nhân, yêu cầu chức năng, yêu cầu phi chức năng, mô hình threat, kiến trúc tổng thể, thiết kế dữ liệu, API và các chính sách bảo mật.

Chương 4. Hiện thực và triển khai hệ thống: Trình bày quá trình hiện thực ứng dụng DocVault, các thành phần microservices, hạ tầng triển khai trên AWS EKS, GitOps, CI/CD và các kiểm soát bảo mật trong pipeline.

Chương 5. Kiểm thử và đánh giá hệ thống: Trình bày chiến lược kiểm thử, bằng chứng giao diện, kiểm thử runtime, kiểm thử bảo mật, bằng chứng pipeline và đánh

giá mức độ đáp ứng yêu cầu.

Chương 6. Kết luận và hướng phát triển: Tổng kết kết quả đạt được, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 Hệ thống quản lý tài liệu bảo mật

Trong phạm vi đề tài này, một hệ thống quản lý tài liệu bảo mật không chỉ được hiểu là nơi lưu trữ file có phân quyền. Hệ thống cần quản lý tài liệu như một đối tượng nghiệp vụ có vòng đời, trạng thái, phiên bản, quyền truy cập và lịch sử thao tác. Một tài liệu được tạo ra, thay đổi, gửi duyệt, phê duyệt, xuất bản hoặc lưu trữ đều cần để lại thông tin đủ rõ để kiểm tra lại.

Điểm quan trọng của hệ thống là khả năng kiểm chứng. Với tài liệu nhạy cảm, câu hỏi không chỉ là file còn tồn tại hay không, mà còn là file này đã được ai tạo, đã qua những phiên bản nào, ai đã phê duyệt, ai đã truy cập và có bằng chứng nào cho quá trình xử lý đó. Vì vậy, audit log, workflow history, version checksum, retention evidence và evidence packet là các thành phần quan trọng trong thiết kế của DocVault.

Các cơ chế phân quyền và kiểm soát truy cập vẫn rất cần thiết, nhưng chúng phục vụ mục tiêu rộng hơn là bảo đảm tài liệu được xử lý đúng theo chính sách. Frontend có thể hiển thị hoặc ẩn thao tác để hỗ trợ trải nghiệm người dùng, nhưng quyết định cuối cùng về quyền truy cập cần được thực thi ở backend để tránh việc chính sách chỉ tồn tại ở giao diện.

Trong DocVault, nhóm chọn tập trung vào các điểm có thể kiểm chứng được trong phạm vi đồ án: vòng đời tài liệu, policy truy cập nội dung file, audit log, retention và evidence packet. Các chức năng rộng hơn của một DMS thương mại như cộng tác thời gian thực, chỉnh sửa văn bản trực tuyến hoặc quản trị tri thức doanh nghiệp không phải trọng tâm chính của bản triển khai này.

2.2 Khảo sát các hệ thống tương tự

Trên thị trường đã có nhiều hệ thống quản lý tài liệu, từ giải pháp thương mại đến mã nguồn mở. Việc đối chiếu với chúng giúp nhóm xác định các tính năng ứng dụng cần học hỏi, các điểm khác biệt cần làm rõ và phạm vi mà DocVault chọn tập trung.

- **Microsoft SharePoint:** nền tảng quản lý tài liệu và cộng tác thương mại, mạnh về tích hợp hệ sinh thái Microsoft 365, phân quyền và compliance [1]. DocVault học hỏi ở cách xem tài liệu như một đối tượng quản lý có quyền, trạng thái và lịch sử, nhưng thu hẹp phạm vi vào vòng đời tài liệu bảo mật và bằng chứng kiểm chứng

thay vì bộ cộng tác văn phòng đầy đủ.

- **Alfresco:** hệ thống ECM mã nguồn mở (có bản thương mại), hỗ trợ vòng đời tài liệu, phân quyền và audit [2], [3]. Đây là tham chiếu gần với hướng quản lý nội dung doanh nghiệp, nhưng DocVault đơn giản hóa mô hình ứng dụng và nhấn mạnh policy truy cập theo classification, audit hash-chain và evidence packet.
- **Nextcloud:** giải pháp self-hosted phổ biến cho lưu trữ và chia sẻ file, dễ triển khai; mạnh về chia sẻ và đồng bộ [4]; nhưng kiểm soát theo classification, DLP và audit tamper-evident không phải trọng tâm mặc định.
- **Dropbox Business:** nền tảng lưu trữ, đồng bộ và cộng tác tài liệu trên cloud, có team folders, nhóm/vai trò quản trị, kiểm soát chia sẻ, SSO, compliance tracking và key management ở các gói doanh nghiệp [5], [6]. DocVault tham khảo cách tổ chức chia sẻ và quản trị nhóm, nhưng khác ở việc không tập trung vào đồng bộ/cộng tác file mà tập trung vào kiểm soát truy cập nội dung, audit và evidence cho tài liệu nhạy cảm.
- **Box và Box Shield:** nền tảng quản lý nội dung cloud có lớp bảo vệ dữ liệu như gán nhãn classification, chính sách phân loại tự động và các quy tắc phát hiện bất thường hoặc nội dung độc hại [7], [8], [9], [10]. Đây là tham chiếu quan trọng cho ý tưởng classification-based protection; DocVault khác ở phạm vi nhỏ hơn, chỉ triển khai các cơ chế cốt lõi có thể minh họa rõ trong đồ án như classification policy, DLP nâng classification và evidence packet.

Phần khảo sát này nhằm định vị hướng tiếp cận của DocVault ở tầng ứng dụng so với một số hệ thống quản lý tài liệu phổ biến. Nhóm chỉ dựa trên tài liệu công khai và đặc điểm tổng quát của các sản phẩm, chưa thực hiện benchmark hoặc triển khai thử trực tiếp từng hệ thống. Vì vậy, phần so sánh không nhằm kết luận hệ thống nào tốt hơn, mà chỉ làm rõ phạm vi mà DocVault tập trung trong đồ án: quản lý vòng đời tài liệu bảo mật, kiểm soát truy cập theo policy, classification, audit/evidence và compliance boundary.

Từ các hệ thống trên, có thể thấy DocVault không được xây dựng để thay thế các hệ thống quản lý tài liệu hoàn chỉnh về độ rộng tính năng. Điểm nhấn của ứng dụng nằm ở một phạm vi hẹp nhưng có thể kiểm chứng: tài liệu có vòng đời rõ ràng, truy cập nội dung file được quyết định bởi policy, classification có ảnh hưởng trực tiếp đến preview/download, audit log có hash-chain và evidence packet không chứa dữ liệu nhạy cảm. Bảng 2.1 tóm tắt các điểm học hỏi và điểm khác biệt chính.

Bảng 2.1: So sánh DocVault với một số hệ thống quản lý tài liệu tiêu biểu

Hệ thống	Mô hình ứng dụng	Điểm học hỏi liên quan	Điểm khác biệt so với DocVault
SharePoint	Quản lý tài liệu và cộng tác trong Microsoft 365	Thư viện tài liệu, quyền truy cập, cộng tác và compliance cho tổ chức.	Không làm bộ cộng tác văn phòng đầy đủ; ưu tiên policy và audit/evidence.
Alfresco	ECM/DMS doanh nghiệp	Metadata, vòng đời tài liệu, phân quyền, audit và quản trị nội dung.	Phạm vi ECM hẹp hơn; nhấn mạnh classification, deny-overrides và evidence packet.
Nextcloud	Lưu trữ, đồng bộ và chia sẻ file self-hosted	Trải nghiệm file server nội bộ, chia sẻ file và đồng bộ nhiều thiết bị.	Không tập trung đồng bộ file; ưu tiên classification, DLP và audit tamper-evident.
Dropbox Business	Lưu trữ, đồng bộ và cộng tác file trên SaaS	Team folders, nhóm/vai trò quản trị, chia sẻ file và kiểm soát bảo mật/compliance ở gói doanh nghiệp.	Không cạnh tranh đồng bộ/cộng tác file; ưu tiên policy preview/download và evidence.
Box + Box Shield	Quản lý nội dung doanh nghiệp trên SaaS	Classification, chính sách phân loại tự động, phát hiện bất thường và nội dung độc hại.	Học hướng classification-based protection; triển khai bản thu gọn với lifecycle, ACL, DLP và evidence.
DocVault	Ứng dụng quản lý tài liệu bảo mật trong phạm vi đồ án	RBAC, ACL deny-overrides, classification policy, DLP nâng classification, audit hash-chain, compliance boundary và evidence packet.	Không phủ rộng như DMS thương mại; tập trung các cơ chế bảo mật có thể kiểm chứng.

2.3 Kiến trúc microservices

Microservices chia hệ thống thành nhiều service nhỏ, mỗi service sở hữu một năng lực nghiệp vụ và phát triển, kiểm thử, triển khai tương đối độc lập. DocVault dùng cách tách này để phân định các bounded context:

- Gateway chịu trách nhiệm xác thực, routing và truyền ngữ cảnh.
- Metadata-service sở hữu metadata, ACL, trạng thái và policy.
- Document-service sở hữu upload/download file và tích hợp object storage.
- Workflow-service sở hữu state machine phê duyệt.
- Audit-service sở hữu audit trail và hash-chain.

- Notification-service nhận thông báo nghiệp vụ trong môi trường demo/dev.

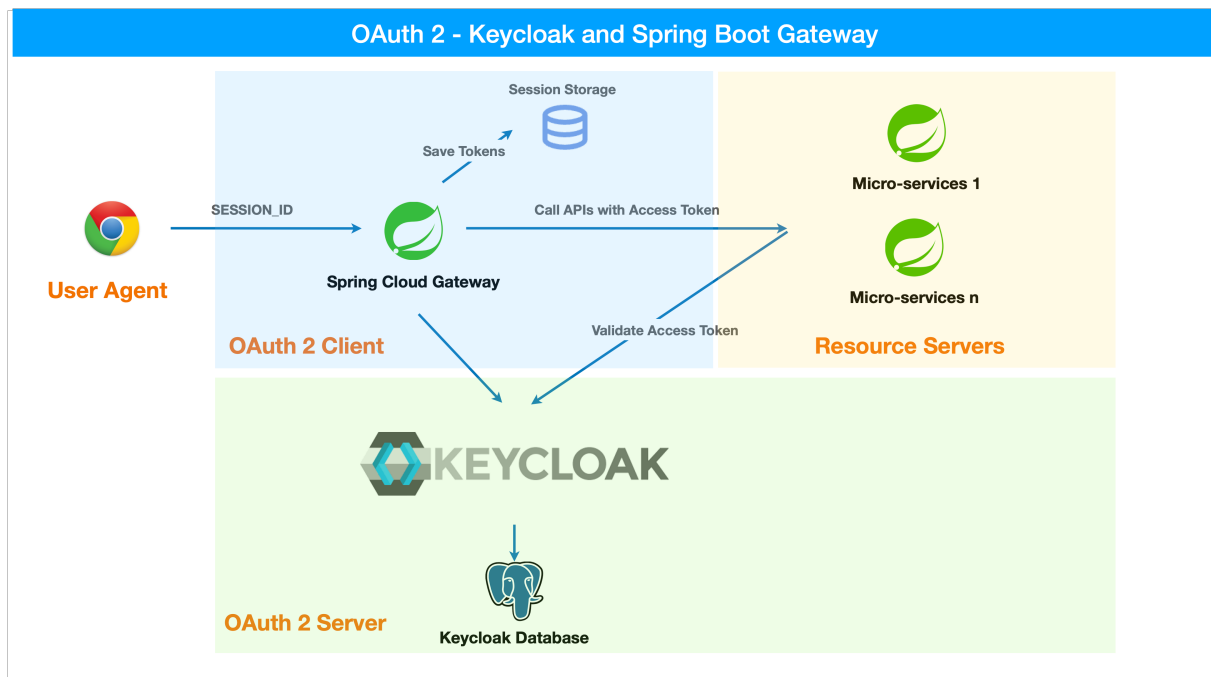
Lý do tách quan trọng nhất với một hệ thống bảo mật là kéo lưu trữ file ra khỏi nơi quyết định quyền. File thật nằm ở MinIO/S3, còn quyền truy cập nằm ở metadata-service. Khi người dùng muốn tải file, document-service không tự ý quyết định mà phải dựa vào policy hoặc grant token do metadata-service cấp và xác minh lại, nên không service nào một mình nắm trọn cả file lẫn quyền. Ban đầu nhóm tách trách nhiệm khá nhiều giữa các service, nhưng khi kiểm thử luồng download thì ranh giới này trở thành điểm cần giữ chặt nhất: object storage không được biến thành đường vòng để bỏ qua policy.

2.4 Xác thực và phân quyền

2.4.1 Keycloak, OIDC và JWT

Keycloak là hệ thống quản lý định danh và truy cập mã nguồn mở, hỗ trợ các chuẩn phổ biến như OAuth 2.0 và OpenID Connect [11]. Trong DocVault, Keycloak được sử dụng để quản lý người dùng, vai trò, nhóm và phát hành token. Sau khi đăng nhập, người dùng nhận JWT chứa thông tin định danh và các claim cần thiết như user id, username, roles và groups.

Gateway và các service phía sau dùng JWT để xác định actor của request. Gateway là entry point chính, xác thực token và truyền các header ngữ cảnh như **x-user-id**, **x-roles**, **x-groups** xuống các service nội bộ. Nhờ đó service nghiệp vụ không phụ thuộc trực tiếp vào phiên đăng nhập frontend mà vẫn đủ thông tin để thực thi chính sách. Các nhóm yêu cầu về xác thực, quản lý phiên, kiểm soát truy cập và validation trong OWASP ASVS được dùng như tham chiếu nền khi thiết kế và kiểm thử các ranh giới này [12].



Hình 2.1: Ví dụ về cách Keycloak hoạt động với Spring Boot Gateway

2.4.2 RBAC

RBAC gán quyền cho vai trò thay vì gán trực tiếp cho từng người dùng, rồi người dùng nhận một hoặc nhiều vai trò. Trong DocVault có năm vai trò:

- **viewer**: đọc tài liệu đã xuất bản nếu policy cho phép.
- **editor**: tạo tài liệu, chỉnh sửa metadata, upload file và gửi duyệt.
- **approver**: xem và phê duyệt hoặc từ chối tài liệu đang chờ duyệt.
- **compliance_officer**: xem audit, metadata và evidence trong phạm vi compliance, nhưng không được truy cập nội dung file.
- **admin**: quản trị hệ thống và có quyền rộng nhất.

RBAC đủ để kiểm soát quyền ở mức route hay chức năng, nhưng nó dừng lại ở mức vai trò: nó không nói được rằng riêng tài liệu này thì người dùng A được tải còn vai trò editor nói chung thì không. Đó là lý do DocVault bổ sung ACL ở mức từng tài liệu.

2.4.3 ACL và deny-overrides

ACL gán rule truy cập cho từng tài liệu. Một rule áp cho user, role, group hoặc tất cả, mang permission như **READ**, **DOWNLOAD**, **WRITE**, **APPROVE** và effect **ALLOW** hoặc **DENY**.

Khi nhiều rule cùng khớp, DocVault chọn chiến lược deny-overrides: chỉ cần có một rule **DENY** phù hợp thì kết quả là từ chối, kể cả khi tồn tại rule **ALLOW** khác. Nhóm chọn hướng này thay vì allow-overrides vì với tài liệu **SECRET**, một lệnh cấm bị ghi đè nguy hiểm

hơn nhiều so với một quyền cho phép bị bỏ sót; khi không chắc, hệ thống nên nghiêng về từ chối.

2.5 Phân loại tài liệu và kiểm soát nội dung

DocVault sử dụng bốn mức phân loại tài liệu:

- **PUBLIC**: tài liệu công khai trong phạm vi người dùng đã xác thực.
- **INTERNAL**: tài liệu nội bộ.
- **CONFIDENTIAL**: tài liệu mật, cần kiểm soát preview/download nghiêm ngặt hơn.
- **SECRET**: tài liệu rất nhạy cảm, chỉ một số vai trò/đối tượng được phép truy cập.

Mức phân loại không dừng ở metadata hiển thị mà còn ảnh hưởng đến chính sách truy cập. Chẳng hạn, tài liệu **CONFIDENTIAL** hoặc **SECRET** có thể không trả về presigned URL trực tiếp mà phải đi qua endpoint stream, để document-service áp thêm kiểm soát như watermark posture và tái xác thực grant.

2.6 Audit, hash-chain và compliance evidence

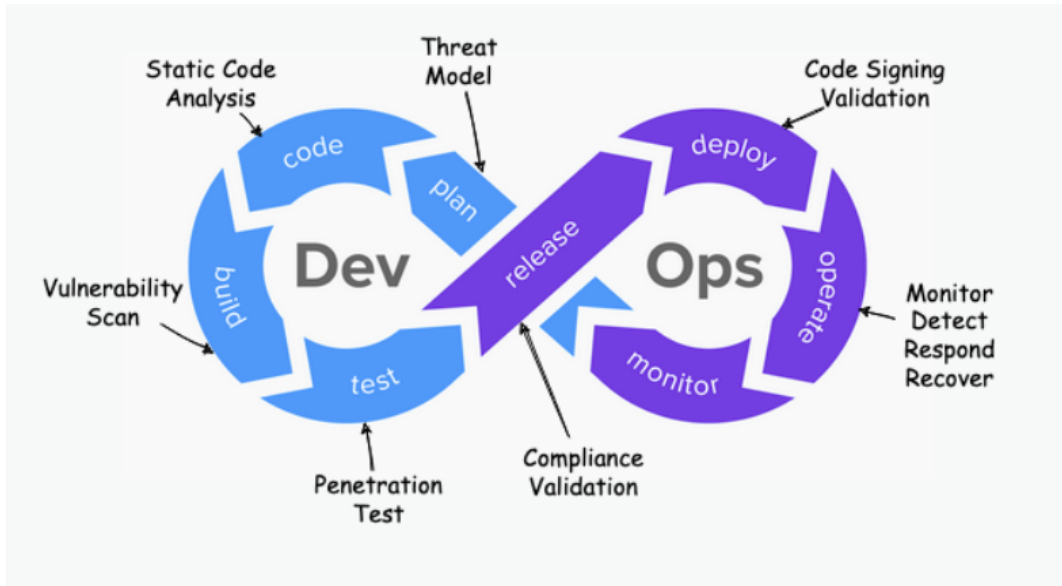
Audit log ghi lại các sự kiện quan trọng trong hệ thống: tạo tài liệu, upload file, submit, approve, reject, download authorized/denied, DLP detected, malware blocked, retention archive và các thao tác compliance. Một audit log hữu ích thì ngoài việc có dữ liệu còn phải cho phép kiểm tra tính toàn vẹn. Cách tiếp cận này phù hợp với hướng dẫn quản lý log của NIST và nhóm kiểm soát Audit and Accountability trong NIST SP 800-53, trong đó log được xem là cơ sở để truy vết, điều tra và đánh giá lại hoạt động hệ thống [13], [14].

DocVault dùng hash-chain cho audit events. Mỗi event mang một hash phụ thuộc vào nội dung của chính nó và hash của event trước. Nếu ai đó sửa thẳng một event cũ trong storage, các hash phía sau sẽ lệch và verify-chain phát hiện được. Cần nói rõ giới hạn: cơ chế này không biến audit log thành lưu trữ bất biến tuyệt đối, nó chỉ làm cho việc sửa đổi để lại dấu vết, tức tamper-evident, và mức đó là đủ cho phạm vi đồ án.

Ngoài audit query, hệ thống còn xuất evidence packet gom metadata, version checksum, workflow history, retention evidence, trạng thái verify-chain và các audit event liên quan đến một tài liệu. Một chi tiết được giữ rất kỹ: evidence packet không bao giờ chứa nội dung file, presigned URL, grant token hay object key nhạy cảm.

2.7 DevSecOps

Trong phạm vi đề tài, DevSecOps không chỉ là phần triển khai phụ trợ, mà là cách nhóm kiểm soát đường đi của một thay đổi từ mã nguồn đến môi trường EKS. Cách tiếp cận này phù hợp với tinh thần shift-left của OWASP DevSecOps Guideline và NIST SSDF: đưa bảo mật vào sớm trong quá trình phát triển, chuẩn hóa các bước kiểm tra, giảm rủi ro lỗ hổng trong phần mềm phát hành và để lại bằng chứng cho từng giai đoạn [15], [16].



Hình 2.2: Mô hình kết hợp phát triển, bảo mật và vận hành trong DevSecOps

Với một hệ thống gồm nhiều microservice, nhiều cơ chế bảo mật và nhiều thành phần hạ tầng, việc triển khai thủ công rất khó lặp lại và khó chứng minh. DevSecOps giúp đưa các bước kiểm tra vào pipeline, từ mã nguồn đến manifest triển khai và ứng dụng sau khi chạy. Kết quả của pipeline không chỉ là một bản release mới, mà còn là tập bằng chứng cho thấy release đó đã đi qua các cổng kiểm soát tương ứng.

Ở DocVault, Jenkins pipeline được chia thành các gate cho chất lượng mã nguồn, dependency, secret, IaC, policy Kubernetes, container artifact và kiểm tra sau triển khai. Các kết quả như scan report, image digest, SBOM, trạng thái ký, GitOps revision và Argo CD health là một phần quan trọng trong bằng chứng của đề tài. Đối với chuỗi cung ứng phần mềm, SLSA được dùng như tham chiếu để nhìn nhận artifact, provenance và xác minh nguồn gốc build, dù phạm vi đồ án mới dừng ở digest, SBOM và ký artifact thay vì triển khai đầy đủ các mức SLSA [17].

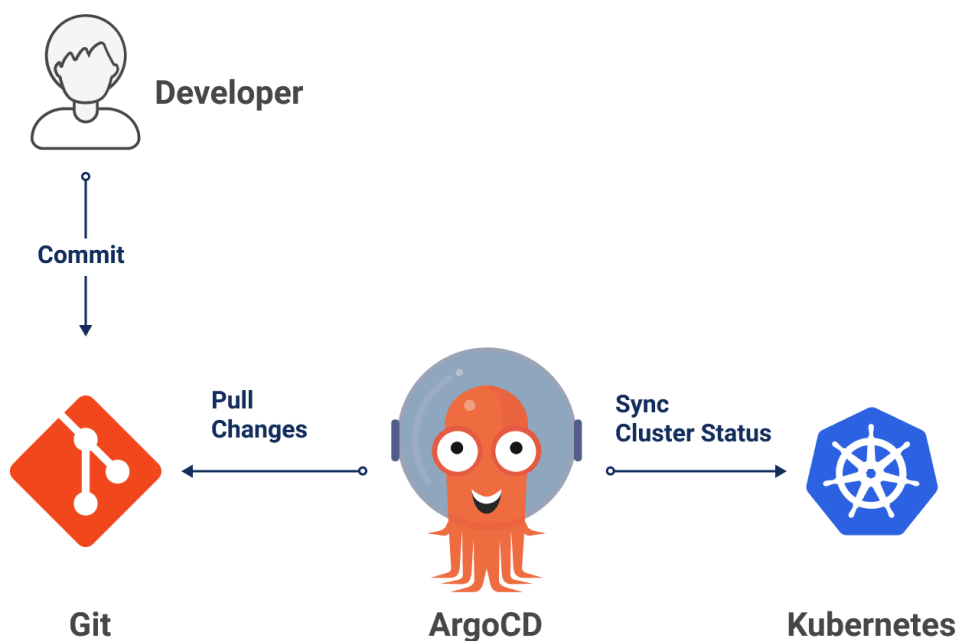
2.8 Các nhóm kiểm thử bảo mật trong pipeline

Bảng 2.2: Các nhóm kiểm thử bảo mật trong DevSecOps pipeline

Nhóm	Mục tiêu	Ví dụ áp dụng trong DocVault	Công cụ
SAST	Phân tích mã nguồn tĩnh để phát hiện lỗi chất lượng và bảo mật.	Kiểm tra backend/frontend TypeScript, quality gate và code smell.	SonarQube
SCA	Phân tích thư viện phụ thuộc và lỗ hổng đã biết.	Quét dependency của monorepo Node.js.	OWASP Dependency Check
FS scan	Quét filesystem repository.	Phát hiện lỗ hổng trong file hệ thống, lockfile, cấu hình.	Trivy
Image scan	Quét Docker image sau build.	Phát hiện CVE trong image backend/frontend.	Trivy
IaC scan	Quét cấu hình hạ tầng.	Kiểm tra Terraform/Kubernetes manifest.	Checkov
DAST	Kiểm thử ứng dụng đang chạy.	Baseline scan web app sau deploy.	OWASP ZAP

2.9 GitOps và Argo CD

GitOps lấy Git làm nguồn sự thật cho trạng thái triển khai. Khi pipeline build xong image mới, thay vì chạy lệnh sửa cluster thủ công, nó cập nhật Helm values hoặc manifest trong một branch GitOps; Argo CD theo dõi branch đó và đồng bộ trạng thái mong muốn xuống Kubernetes [18], [19]. Khác biệt thực tế là không ai `kubectl apply` trực tiếp lên cụm, mọi thay đổi triển khai đều đi qua một commit. Cơ chế reconcile liên tục cũng giúp phát hiện configuration drift khi trạng thái thật trong cluster lệch khỏi manifest trong Git [19].



Hình 2.3: Luồng triển khai GitOps với Argo CD

DocVault tận dụng đúng điểm này. Pipeline cập nhật các file values theo từng service, rồi Argo CD sync các application gateway, metadata-service, document-service, workflow-service, audit-service, notification-service và web. Việc tách build pipeline khỏi deployment controller giúp mỗi bên giữ một trách nhiệm, đồng thời để lại lịch sử thay đổi triển khai ngay trong Git.

2.10 Tổng hợp công nghệ và công cụ

Để có cái nhìn tổng thể, phần này tổng hợp các công nghệ và công cụ chính được sử dụng trong đề tài, chia thành hai nhóm: nhóm xây dựng ứng dụng (Bảng 2.3) và nhóm quy trình DevSecOps cùng hạ tầng triển khai (Bảng 2.4). Các phiên bản được ghi nhận theo cấu hình thực tế trong repository tại thời điểm hoàn thành báo cáo. Khi mô tả vai trò của từng công cụ, nhóm ưu tiên đối chiếu với tài liệu chính thức của dự án hoặc nhà cung cấp, chẳng hạn Keycloak, Argo CD, Harbor, SonarQube, Trivy, OWASP ZAP và các dịch vụ AWS được dùng trong triển khai [11], [18], [20], [21], [22], [23], [24], [25].

Bảng 2.3: Công nghệ xây dựng ứng dụng DocVault

Thành phần	Công nghệ	Phiên bản	Vai trò
Frontend	Next.js, React	16.2, 19.2	Giao diện web theo App Router.
UI & form	Tailwind CSS, Radix UI, React Hook Form, Zod	4.x, 7.7, 4.3	Thiết kế giao diện, form và validation phía client.
Server state	TanStack Query	5.90	Quản lý dữ liệu server, cache và đồng bộ.
Backend	NestJS	10.0	Framework cho gateway và các microservice.
Ngôn ngữ	TypeScript	5.x	Ngôn ngữ chính cho frontend và backend.
Xác thực	Keycloak, passport-jwt, jwks-rsa	—	OIDC/JWT, xác minh token bằng JWKS RS256.
ORM	Prisma + adapter-pg	7.5	Truy cập PostgreSQL bằng driver adapter.
Metadata DB	PostgreSQL	—	Lưu metadata, ACL, workflow history.
Audit DB	MongoDB, Mongoose	—	Lưu audit event và hash-chain.
Object storage	MinIO / Amazon S3	—	Lưu blob file tài liệu.
Rate limit	@nestjs/throttler	6.5	Giới hạn tần suất request theo cấu hình chung.
Monorepo	pnpm, Turbo	9.15, 2.1	Quản lý workspace và điều phối build.

Bảng 2.4: Công cụ DevSecOps và hạ tầng triển khai

Nhóm	Công cụ	Vai trò trong đề tài
CI/CD	Jenkins, Shared Library	Điều phối pipeline, tách CI validation và CD release.
Secret scan	TruffleHog	Phát hiện credential bị commit trong repository.
SAST	SonarQube	Phân tích mã nguồn tĩnh và Quality Gate.
SCA	OWASP Dependency-Check	Quét lỗ hổng thư viện phụ thuộc theo CVSS.
FS/Image scan	Trivy	Quét filesystem và Docker image.
IaC scan	Checkov	Quét cấu hình Terraform, Kubernetes, Helm.
Policy-as-code	Kyverno CLI	Kiểm tra policy trên manifest đã render.
IaC	Terraform	Mô tả hạ tầng AWS dưới dạng mã.
Container	Docker BuildKit	Build và cache image theo batch.
Registry	Harbor	Private registry, lưu SBOM và trạng thái ký.
Ký artifact	Cosign	Ký và xác minh image digest.
GitOps	Git, Argo CD	Đồng bộ desired state theo mô hình app-of-apps.
Orchestration	Amazon EKS, Helm, Kustomize	Chạy workload trên Kubernetes.
Secret & key	AWS Secrets Manager, External Secrets, KMS	Quản lý bí mật và khóa mã hóa (customer-managed key, xoay vòng hằng năm).
DAST	OWASP ZAP	Quét động ứng dụng sau triển khai.
Observability	Prometheus, Grafana, Loki	Thu thập metric, log và trực quan hóa.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích yêu cầu và thiết kế hệ thống DocVault. Nội dung chương bao gồm xác định tác nhân, yêu cầu chức năng, yêu cầu phi chức năng, mô hình threat cơ bản, kiến trúc tổng thể, thiết kế dữ liệu, chính sách phân quyền, các luồng nghiệp vụ quan trọng và API contract tổng quan. Đây là cơ sở để hiện thực hệ thống và triển khai quy trình DevSecOps ở chương tiếp theo.

3.1 Tổng quan bài toán

DocVault hướng đến bài toán quản lý tài liệu trong môi trường có nhiều vai trò và nhiều mức độ nhạy cảm dữ liệu. Một tài liệu đi qua nhiều trạng thái: bản nháp, chờ duyệt, đã xuất bản và đã lưu trữ. Trong quá trình đó, mỗi vai trò chỉ được thực hiện những thao tác phù hợp. Editor có thể tạo và gửi duyệt tài liệu; Approver có thể phê duyệt hoặc từ chối; Viewer chỉ được truy cập tài liệu đã xuất bản nếu policy cho phép; Compliance Officer có thể xem audit và evidence nhưng không được truy cập nội dung file; Admin có quyền quản trị rộng hơn.

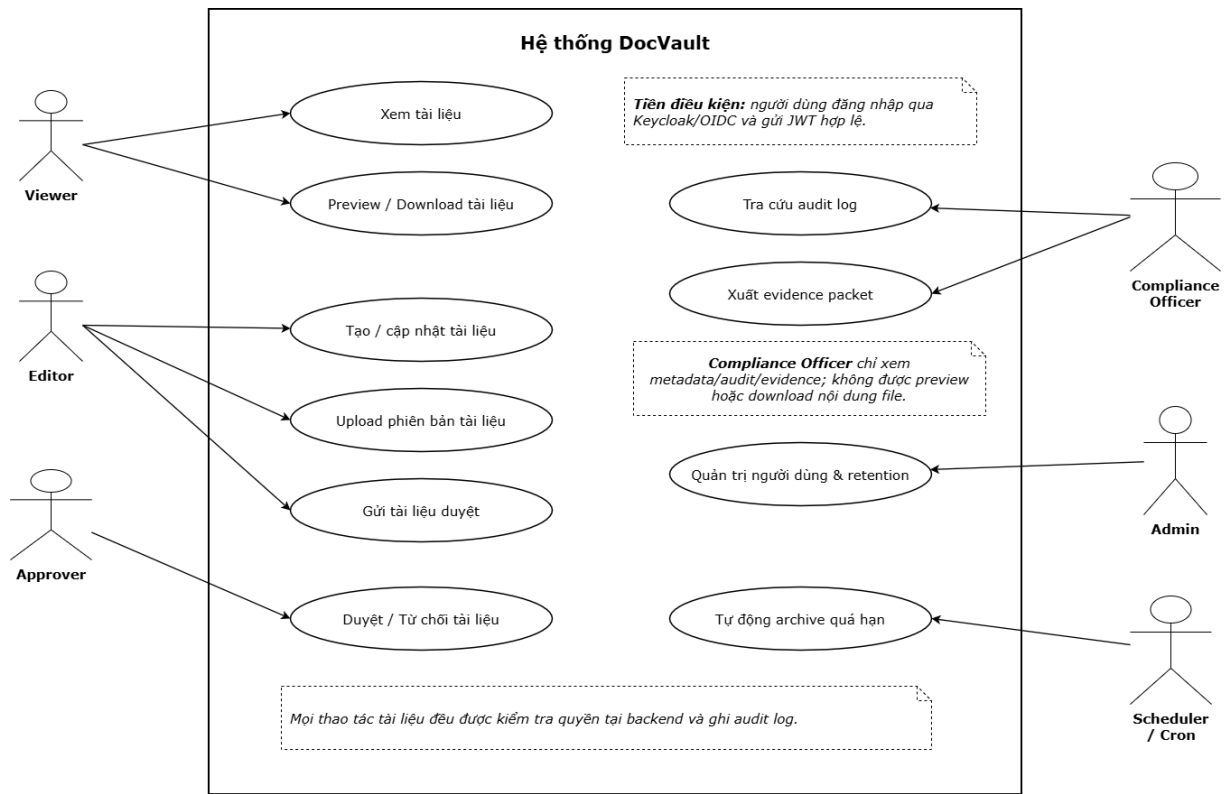
Bài toán ở đây không dừng ở việc lưu file, mà là kiểm soát file theo ngữ cảnh. Cùng một file có thể được phép preview với người này nhưng bị từ chối với người khác; có thể được cấp presigned URL khi là tài liệu public nhưng phải stream qua backend khi là tài liệu confidential; có thể cho xem metadata nhưng không cho tải nội dung. Vì vậy, hệ thống cần một mô hình policy đủ rõ và được thực thi ở backend.

3.2 Tác nhân hệ thống

Bảng 3.1: Các tác nhân chính trong DocVault

Tác nhân	Vai trò trong hệ thống
Viewer	Người dùng có nhu cầu đọc và theo dõi các tài liệu được phép truy cập trong hệ thống. Viewer chủ yếu thao tác với danh sách tài liệu, metadata và các tài liệu đã được xuất bản trong phạm vi policy cho phép.
Editor	Người tạo và quản lý tài liệu trong phạm vi được cấp quyền. Editor có thể tạo draft, chỉnh metadata, upload phiên bản file, gửi duyệt và archive tài liệu khi thỏa điều kiện policy.
Approver	Người phụ trách kiểm tra các tài liệu đang chờ duyệt. Approver có thể xem hàng đợi duyệt, kiểm tra nội dung/metadata cần thiết và đưa ra quyết định approve hoặc reject theo workflow.
Compliance Officer	Người phụ trách kiểm tra tuân thủ. Vai trò này tập trung vào audit, metadata, evidence, retention và các thông tin phục vụ đối soát, không đại diện cho quyền truy cập nội dung file.
Admin	Người quản trị hệ thống. Admin có quyền rộng hơn trong việc quản lý tài liệu, thành viên, evidence, retention và các thao tác nhạy cảm.
Scheduler / Cron	Tác nhân hệ thống không phải người dùng. Tác nhân này chạy nền theo lịch để tự động archive tài liệu đến hạn retention và dọn tài liệu trong thùng rác quá hạn lưu giữ.

Hình 3.1 tổng hợp các use case chính của từng tác nhân trong một sơ đồ thống nhất. Ngoài các vai trò người dùng, sơ đồ bổ sung tác nhân Scheduler/Cron đại diện cho các tác vụ nền theo lịch. Sơ đồ làm rõ hai điểm thiết kế cốt lõi của DocVault: mọi tác nhân phải đăng nhập qua Keycloak/OIDC trước khi thao tác, và mọi thao tác quan trọng trên tài liệu đều được kiểm tra quyền tại backend cũng như ghi audit log.



Hình 3.1: Sơ đồ use case tổng thể của DocVault theo tác nhân

3.3 Yêu cầu chức năng

Các yêu cầu chức năng được nhóm theo miền nghiệp vụ như sau:

3.3.1 Xác thực và phân quyền

Hệ thống cần hỗ trợ đăng nhập thông qua Keycloak, đọc thông tin người dùng hiện tại và phân quyền theo vai trò. Navigation, command palette và các thao tác trên UI cần thay đổi theo role, nhưng backend vẫn phải là nơi quyết định cuối cùng. Các request không có token, token hết hạn hoặc token không hợp lệ phải bị từ chối.

3.3.2 Quản lý tài liệu

Người dùng có quyền có thể tạo tài liệu mới với tiêu đề, mô tả, classification và tags. Sau khi tạo, tài liệu ở trạng thái **DRAFT**. Editor hoặc Admin có thể upload file ban đầu hoặc upload phiên bản mới. Hệ thống cần lưu checksum, kích thước, MIME type, tên file và người upload cho từng version.

3.3.3 Workflow phê duyệt

Tài liệu đi qua vòng đời chính DRAFT -> PENDING -> PUBLISHED -> ARCHIVED. Editor gửi tài liệu từ Draft sang Pending. Approver phê duyệt tài liệu Pending sang Published hoặc từ chối để quay về Draft. Tài liệu Published có thể được archive theo quyền. Mỗi transition cần ghi workflow history và audit event.

3.3.4 Preview và download

Người dùng có quyền có thể preview hoặc download tài liệu tùy trạng thái, classification, role, ownership và ACL. Với tài liệu nhạy cảm, hệ thống có thể không trả về URL tải trực tiếp mà trả về endpoint stream có kiểm soát. Compliance Officer luôn bị chặn truy cập nội dung file.

3.3.5 Audit, security và evidence

Hệ thống cần ghi audit event cho các hành động quan trọng, hỗ trợ truy vấn audit theo bộ lọc, kiểm tra hash-chain và hiển thị security summary. Compliance/Admin có thể xuất evidence packet liên quan đến tài liệu, recommendation hoặc compliance flow.

3.3.6 Retention và access review

Tài liệu đã xuất bản cần có retention evidence dựa trên classification. Admin có thể chạy retention job demo để auto-archive các tài liệu đến hạn. Compliance/Admin có thể xem access review cho tài liệu nhạy cảm, broad ACL grant hoặc quyền đã cũ.

3.4 Yêu cầu phi chức năng

Bảng 3.2: Yêu cầu phi chức năng

Nhóm yêu cầu	Mô tả
Bảo mật	Backend phải thực thi policy; token và grant phải được xác minh; nội dung file không được lộ qua evidence hoặc audit.
Truy vết	Các thao tác quan trọng cần tạo audit event; audit chain cần có khả năng verify.
Tách biệt trách nhiệm	Mỗi service sở hữu một bounded context rõ ràng; document-service không tự quyết định policy download.
Khả năng kiểm thử	Có unit test, E2E runtime, frontend test/lint/build và pipeline scan.
Khả năng triển khai	Hỗ trợ local Docker Compose, Docker image, Kubernetes/Helm và GitOps.
Khả năng mở rộng	Kiến trúc cho phép bổ sung message queue, kênh thông báo, môi trường triển khai và replica độc lập; nền tảng hiện tại đã tích hợp SBOM, ký image, KMS, External Secrets và policy-as-code trong CI.

3.5 Mô hình threat cơ bản

Mô hình threat trong đề tài được sử dụng như một công cụ hỗ trợ thiết kế bảo mật cho phần ứng dụng, không nhằm mô phỏng tấn công chuyên sâu. Mục tiêu của phần này là xác định các điểm trong hệ thống cần kiểm soát, chẳng hạn truy cập metadata, tải nội dung file, thay đổi trạng thái tài liệu, ghi audit event và xuất evidence. Các rủi ro được ánh xạ theo STRIDE để nhóm có cơ sở liên kết từng rủi ro với biện pháp giảm thiểu tương ứng trong DocVault.

Bảng 3.3: Mô hình threat của DocVault ánh xạ theo STRIDE

Rủi ro	Loại STRIDE	Biện pháp giảm thiểu trong DocVault
Truy cập trái phép metadata	Information Disclosure	Người dùng đoán document id và gọi API chi tiết, workflow history, comments hoặc ACL; backend policy-filter theo role, owner, ACL, classification và status.
Tải nội dung file trái phép	Elevation of Privilege	Gọi trực tiếp presign, stream hoặc download endpoint mà không có quyền; chặn bằng grant token ngắn hạn và tái xác minh ở document-service.
Lạm dụng Compliance Officer	Elevation of Privilege	Vai trò compliance xem được nhiều metadata nhưng luôn bị deny nội dung file ở mọi classification.
Upload file độc hại	Tampering	File EICAR/malware bị chặn trước khi ghi MinIO và không tạo version metadata.
Rò rỉ dữ liệu nhạy cảm	Information Disclosure	DLP phát hiện email, số điện thoại, từ khóa nhạy cảm trong tài liệu public/internal và nâng classification.
Downgrade classification không hợp lệ	Tampering	Tài liệu đã bị DLP phát hiện không được hạ xuống public/internal nếu thiếu quyền và lý do override hợp lệ.
Giả mạo audit event	Spoofing	Người dùng thường không append audit trực tiếp; audit ingest yêu cầu service token.
Sửa audit log cũ	Tampering	Hash-chain phát hiện khi audit event bị thay đổi trực tiếp trong storage.

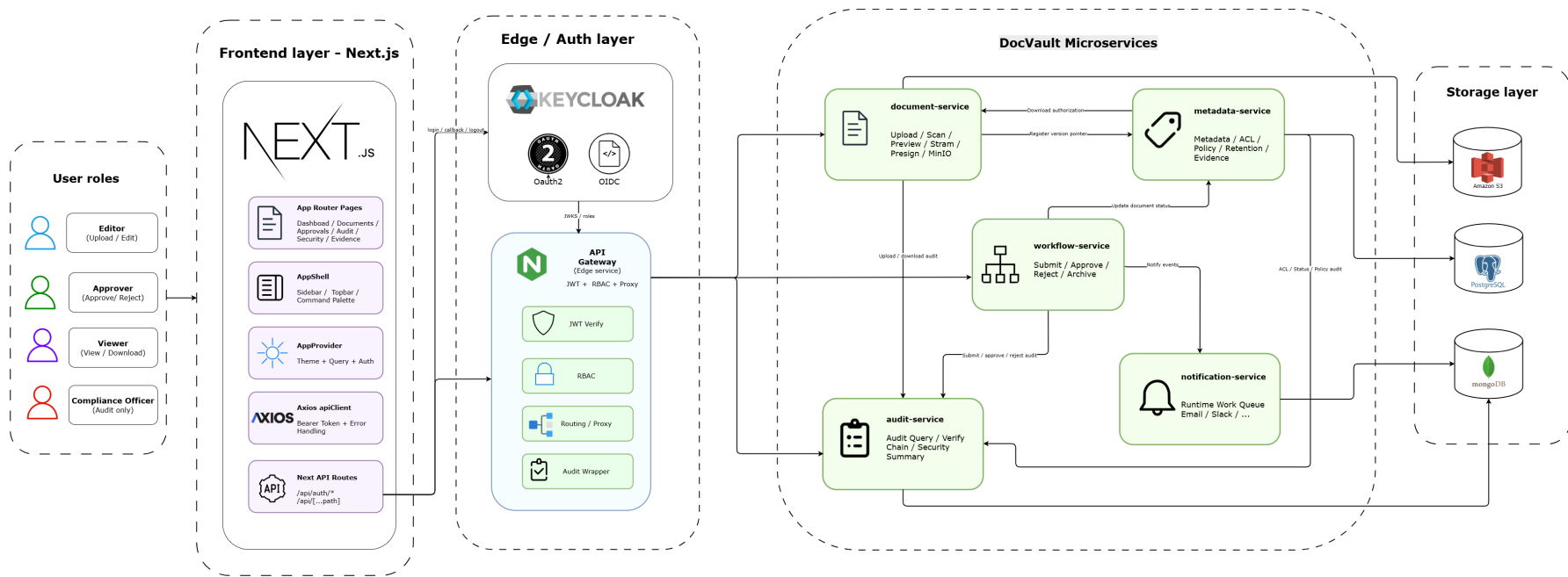
Nhìn chung, các rủi ro trong mô hình threat đều quay về mục tiêu của phần ứng dụng: bảo đảm tài liệu được truy cập, xử lý và ghi nhận đúng theo chính sách. Information Disclosure và Elevation of Privilege liên quan đến việc lộ hoặc vượt quyền truy cập tài liệu. Tampering và Spoofing liên quan đến tính đúng đắn của file, metadata và audit log. Trong phạm vi đồ án, mô hình threat đóng vai trò định hướng cho các kiểm soát như backend policy, grant token, audit ingest boundary và hash-chain.

3.6 Kiến trúc tổng thể

Kiến trúc DocVault gồm bốn lớp: frontend, gateway, microservices và hạ tầng dữ liệu. Frontend Next.js là giao diện người dùng. API Gateway là điểm vào duy nhất cho frontend, chịu trách nhiệm xác thực, role guard, proxy request và truyền ngữ cảnh. Các service phía sau xử lý nghiệp vụ riêng. Hạ tầng dữ liệu gồm Keycloak, PostgreSQL, MongoDB

và MinIO/S3.

Hình 3.2 mô tả cách các lớp này liên kết với nhau. Trong kiến trúc đó, frontend chỉ giao tiếp qua gateway, còn các microservice phía sau giữ ranh giới nghiệp vụ riêng: metadata-service sở hữu policy và metadata, document-service sở hữu blob file, workflow-service điều phối trạng thái, audit-service ghi nhận sự kiện và notification-service xử lý thông báo.



Hình 3.2: Kiến trúc tổng thể ứng dụng DocVault

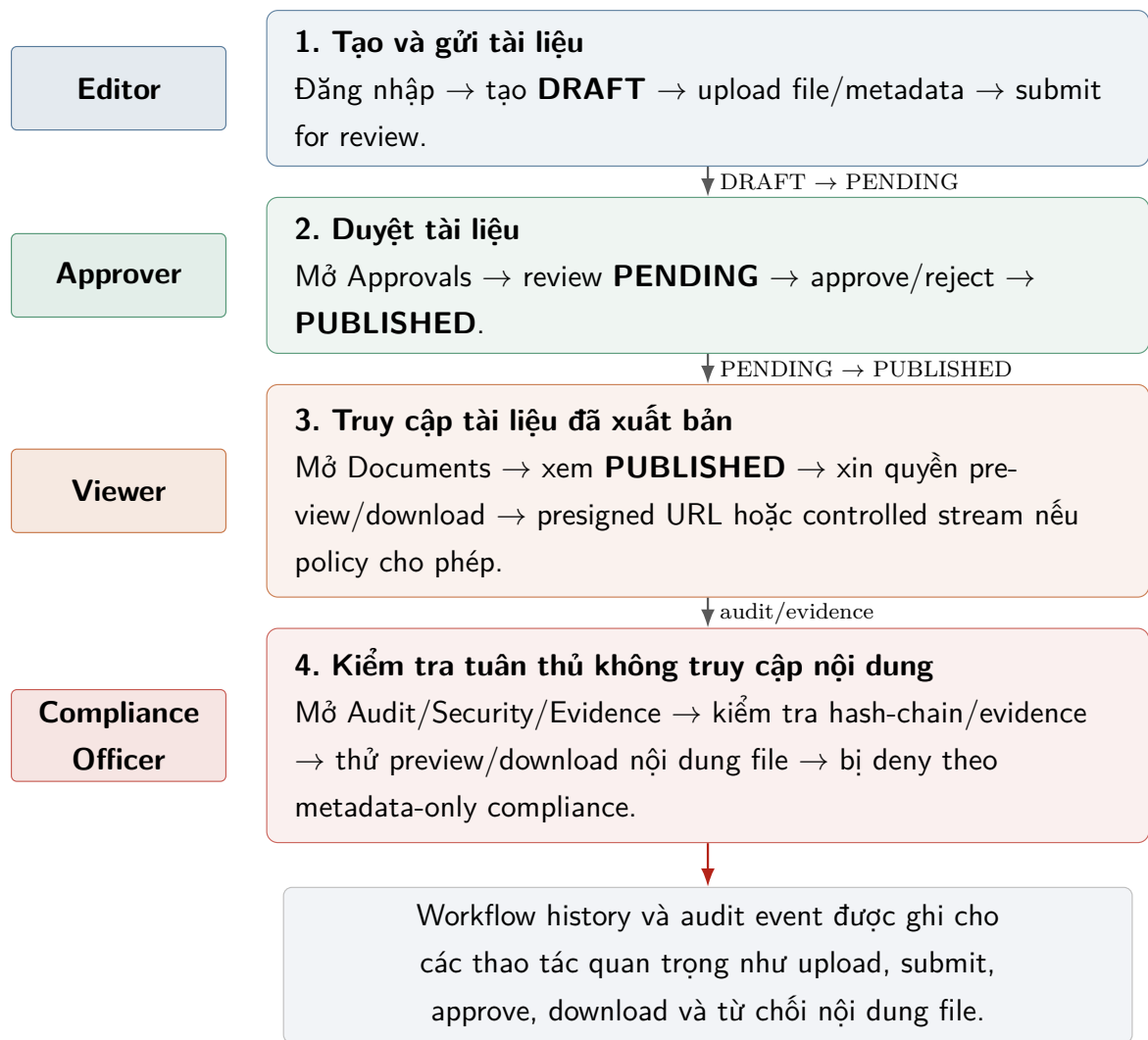
DocVault được tổ chức thành bốn lớp xếp chồng từ client đến hạ tầng dữ liệu, như minh họa ở Hình 3.2:

- **Lớp client và frontend:** trình duyệt người dùng tải ứng dụng Next.js. Frontend chỉ giữ một API base URL duy nhất trở đến gateway và không gọi trực tiếp bất kỳ microservice nào. Người dùng đăng nhập qua Keycloak theo luồng OIDC, sau đó frontend đính kèm JWT vào mỗi request gửi lên gateway.
- **Lớp gateway:** API Gateway là điểm vào duy nhất. Tại đây JWT được xác minh với Keycloak qua JWKS, role được kiểm tra ở mức route, và request hợp lệ được proxy xuống service phù hợp kèm các header ngữ cảnh (`x-user-id`, `x-roles`, `x-groups`, `x-request-id`). Gateway không chứa nghiệp vụ tài liệu.
- **Lớp microservice:** mỗi service sở hữu một bounded context riêng. Metadata-service giữ metadata, ACL, policy và cấp grant token; document-service xử lý blob file và đối tượng lưu trữ; workflow-service điều phối state machine; audit-service ghi và xác minh hash-chain; notification-service nhận thông báo nghiệp vụ. Các service giao tiếp với nhau bằng HTTP nội bộ (Axios) và được miễn rate limit qua header gọi nội bộ.
- **Lớp hạ tầng dữ liệu:** Keycloak quản lý định danh; PostgreSQL lưu metadata; MongoDB lưu audit event; MinIO/S3 lưu file. Mỗi service chỉ truy cập kho dữ liệu thuộc bounded context của nó, tránh việc một service đọc trực tiếp database của service khác.

Cách phân lớp này được thiết kế để không tồn tại đường tắt bỏ qua kiểm soát chính sách. Với một thao tác nhạy cảm như tải file, request luôn xuất phát từ frontend, đi qua gateway để xác thực, rồi mới đến metadata-service để xin phép; document-service chỉ phát file sau khi nhận được grant token hợp lệ. Lớp backend vì thế luôn giữ quyền quyết định cuối cùng về truy cập.

3.7 Luồng nghiệp vụ chính

Luồng demo chính của hệ thống gồm bốn vai trò. Editor đăng nhập, tạo tài liệu, upload file và submit. Approver đăng nhập, kiểm tra tài liệu pending và approve. Viewer đăng nhập, xem tài liệu đã published và download nếu policy cho phép. Compliance Officer đăng nhập, xem audit/evidence nhưng bị chặn preview/download nội dung file. Bản thân chuỗi trạng thái mà tài liệu đi qua trong các luồng này được mô tả chi tiết ở Mục 3.11.



Hình 3.3: Luồng nghiệp vụ demo chính theo vai trò trong DocVault

3.8 Nguyên tắc thiết kế

Thiết kế DocVault tuân theo các nguyên tắc sau:

1. **Backend authoritative:** frontend chỉ hỗ trợ UX; backend luôn là nơi thực thi policy.
2. **Tách metadata và blob:** metadata nằm ở PostgreSQL, file nằm ở MinIO/S3, version metadata chỉ giữ object key và checksum.
3. **Deny wins:** ACL deny có độ ưu tiên cao hơn allow.
4. **Metadata-only compliance:** compliance có thể kiểm tra hệ thống mà không cần truy cập nội dung file.
5. **Evidence without leakage:** evidence không chứa nội dung file, presigned URL, grant token hoặc object key nhạy cảm.
6. **Security as pipeline gates:** các bước scan/test cần chạy tự động trong pipeline

thay vì phụ thuộc thao tác thủ công.

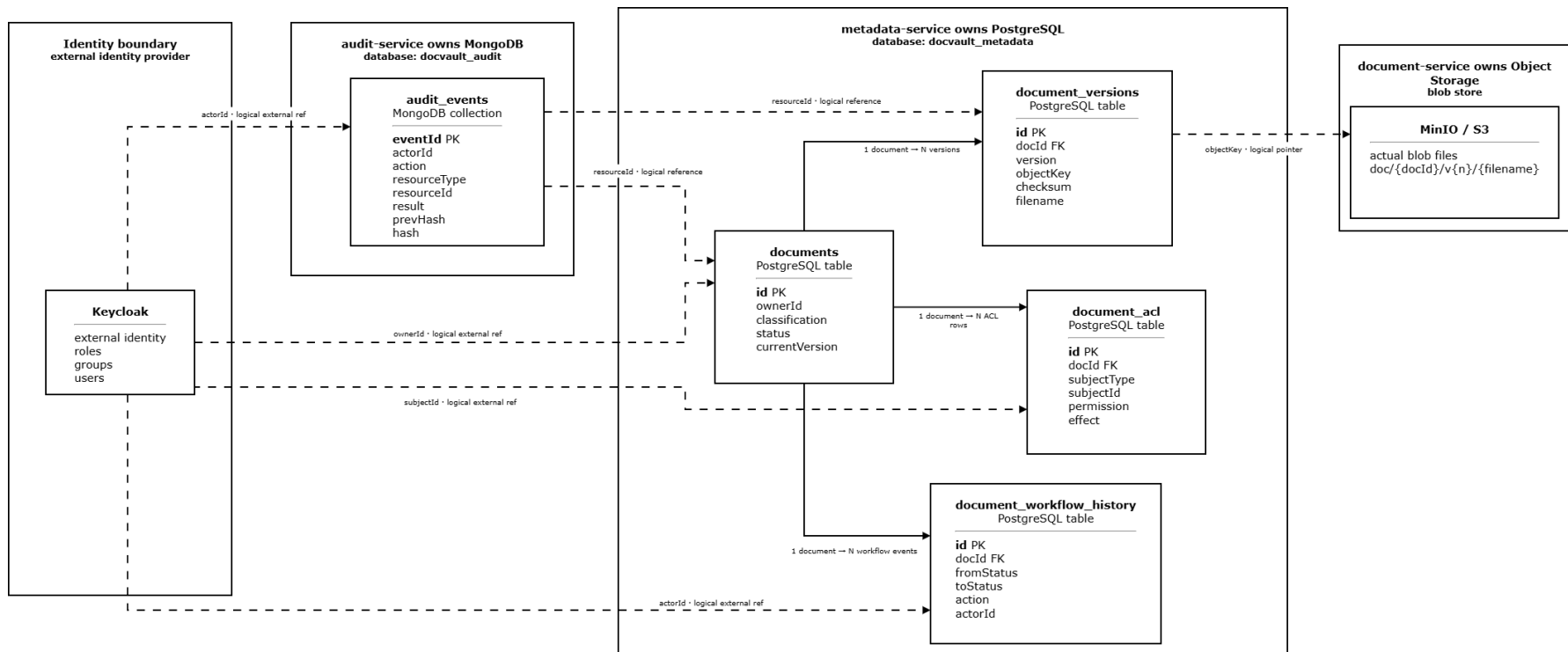
Thiết kế hệ thống của DocVault được xây dựng xoay quanh tài liệu và vòng đời của tài liệu. Thay vì chỉ lưu thông tin file, hệ thống cần lưu thêm metadata, version, trạng thái workflow, quyền truy cập, lịch sử thao tác và các audit event liên quan. Cách thiết kế này giúp hệ thống có thể truy vết lại quá trình xử lý tài liệu, từ lúc tạo mới đến khi phê duyệt, truy cập hoặc lưu trữ.

Các bảng và collection trong hệ thống được chia theo trách nhiệm của từng service. Metadata-service quản lý thông tin chính của tài liệu, document-service quản lý file và checksum, workflow-service quản lý quá trình chuyển trạng thái, audit-service lưu lại các hoạt động quan trọng, còn các thành phần evidence và retention giúp tổng hợp thông tin phục vụ kiểm tra lại tài liệu. Cách tách này giúp hệ thống rõ ràng hơn và phù hợp với kiến trúc microservices.

3.9 Tổng quan thiết kế dữ liệu

DocVault tách dữ liệu thành ba nhóm chính. Metadata tài liệu thuộc metadata-service và lưu trong PostgreSQL. Nội dung file thuộc document-service và lưu trong MinIO/S3. Audit event thuộc audit-service và lưu trong MongoDB. Việc tách này giúp mỗi service sở hữu dữ liệu của mình, tránh việc service khác truy vấn trực tiếp vào database không thuộc bounded context của nó.

Metadata database lưu documents, document versions, ACL và workflow history; audit database lưu audit events; object storage lưu blob file. Nhờ tách như vậy, hệ thống truy vấn metadata nhanh, lưu file dung lượng lớn trong object storage, đồng thời giữ audit log độc lập với nghiệp vụ tài liệu.



Hình 3.4: ERD và ranh giới sở hữu dữ liệu trong DocVault

DocVault tổ chức dữ liệu xoay quanh thực thể trung tâm `documents`, với các thực thể vệ tinh `document_versions`, `document_acl`, `document_workflow_history`, `document_comments` và `document_share_links` cùng tham chiếu đến `documents` qua `docId` theo quan hệ một-nhiều (Hình 3.4). Mỗi tài liệu có thể có nhiều phiên bản, nhiều rule ACL, nhiều bản ghi lịch sử workflow và nhiều comment.

Bên cạnh quan hệ thực thể, mô hình dữ liệu được phân chia theo ranh giới sở hữu tương ứng ba kho lưu trữ. Vùng PostgreSQL (thuộc `metadata-service`) gồm `documents` và các bảng vệ tinh, là nơi lưu metadata và quyền truy cập. Vùng object storage (thuộc `document-service`) lưu blob file; lưu ý `document_versions` chỉ giữ `objectKey` và `checksum` trỏ tới file thật chứ không lưu nội dung. Vùng MongoDB (thuộc `audit-service`) lưu `audit_events` và cố ý không dùng khóa ngoại cứng đến `documents`, vì audit là bounded context độc lập và phải tồn tại được kể cả khi tài liệu bị xóa. Đây cũng là lý do một thao tác như tải file phải đi qua nhiều service: metadata nằm một nơi, file nằm một nơi, và dấu vết kiểm toán nằm ở nơi thứ ba.

3.10 Các thực thể chính

3.10.1 documents

Bảng `documents` là nguồn sự thật cho tài liệu logic. Mỗi record đại diện cho một tài liệu, gồm tiêu đề, mô tả, owner, classification, tags, status, current version và các mốc thời gian như `createdAt`, `updatedAt`, `publishedAt`, `archivedAt`.

Trạng thái tài liệu gồm DRAFT, PENDING, PUBLISHED và ARCHIVED. Classification gồm PUBLIC, INTERNAL, CONFIDENTIAL và SECRET. Hai trường này là nền tảng cho phần lớn quyết định policy.

3.10.2 document_versions

Bảng `document_versions` lưu thông tin từng phiên bản file. Mỗi version có `docId`, version number, object key, checksum, size, filename, contentType và `createdBy`. File thật không nằm trong PostgreSQL; database chỉ giữ pointer đến object storage.

Checksum SHA-256 hỗ trợ kiểm tra tính nhất quán và làm evidence. Khi export evidence packet, hệ thống có thể chứng minh tài liệu có version cụ thể và checksum cụ thể mà không cần đưa nội dung file vào packet.

3.10.3 document_acl

Bảng `document_acl` lưu các rule truy cập theo tài liệu. Rule có `subjectType`, `subjectId`, permission và effect. `SubjectType` hỗ trợ USER, ROLE, GROUP và ALL. Permission hỗ trợ READ, DOWNLOAD, WRITE và APPROVE. Effect là ALLOW hoặc DENY.

ACL không thay thế RBAC mà bổ sung cho RBAC. RBAC xác định năng lực chung theo vai trò, trong khi ACL xác định ngoại lệ hoặc quyền cụ thể theo tài liệu. Khi có rule deny phù hợp, hệ thống ưu tiên từ chối.

3.10.4 document_workflow_history

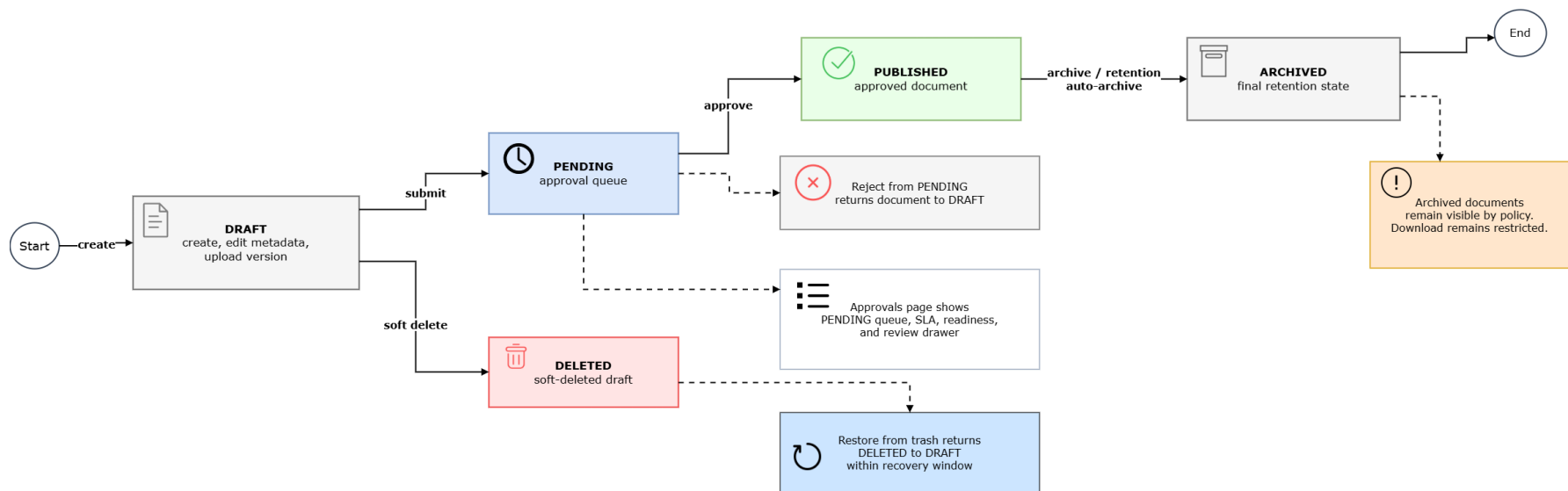
Bảng `document_workflow_history` lưu lịch sử chuyển trạng thái nghiệp vụ. Mỗi record có `docId`, `fromStatus`, `toStatus`, `action`, `actorId`, `reason` và `createdAt`. Bảng này phục vụ truy vấn nhanh lịch sử phê duyệt trong màn hình chi tiết tài liệu, tách khỏi `audit-service` vốn ghi nhiều loại sự kiện hệ thống hơn.

3.10.5 audit_events

Audit event ghi lại hành động trong hệ thống. Mỗi event chứa `actor`, `action`, `resource type`, `resource id`, `result`, `reason`, `metadata` và `hash`. Audit-service áp dụng hash-chain để phát hiện sửa đổi ngoài luồng. Vì audit là bounded context riêng, audit event không dùng foreign key cứng đến documents.

3.11 Document lifecycle

Luồng trạng thái tài liệu trong DocVault được giữ đủ gọn để phù hợp với phạm vi MVP nhưng vẫn bao phủ demo doanh nghiệp. Các transition chính gồm tạo draft, gửi duyệt, phê duyệt, từ chối và lưu trữ; ngoài ra hệ thống còn mô phỏng soft delete và restore cho tài liệu nháp. Phần này đi sâu vào state machine; cách bốn vai trò tương tác với các trạng thái đó đã được trình bày ở Mục 3.7.



Hình 3.5: Vòng đời trạng thái tài liệu trong DocVault

DocVault định nghĩa vòng đời tài liệu bằng một state machine với các trạng thái DRAFT, PENDING, PUBLISHED, ARCHIVED, DELETED và các transition có ràng buộc vai trò (Hình 3.5). Tài liệu mới luôn bắt đầu ở DRAFT. Editor thực hiện SUBMIT để chuyển sang PENDING; Approver thực hiện APPROVE để sang PUBLISHED hoặc REJECT để quay về DRAFT; tài liệu PUBLISHED có thể được ARCHIVE. Ngoài vòng đời chính, hệ thống còn hỗ trợ nhánh soft delete và restore áp dụng cho bản nháp.

Hai nguyên tắc chi phối vòng đời này. Thứ nhất, các transition là một chiều và ràng buộc theo vai trò, nên giữa hai trạng thái bất kỳ không có đường đi tùy ý; workflow-service kiểm tra transition có hợp lệ không trước khi yêu cầu metadata-service cập nhật. Thứ hai, mỗi transition sinh một bản ghi `document_workflow_history` kèm một audit event, đủ để trang chi tiết tài liệu dựng lại timeline nghiệp vụ và để compliance đối chiếu chéo với audit log. Khi tài liệu đi qua APPROVE, hệ thống đặt `publishedAt`; khi đi qua ARCHIVE, hệ thống đặt `archivedAt`.

3.12 Ma trận phân quyền

Các vai trò chính của hệ thống đã được giới thiệu ở Mục 3.2. Trong mục này, báo cáo tập trung vào cách các vai trò đó được ánh xạ thành quyền truy cập nội dung file theo classification.

Policy của DocVault không chỉ dựa trên role. Kết quả truy cập được xác định bởi nhiều yếu tố như role, owner, ACL, classification, trạng thái tài liệu, version tồn tại và trạng thái deleted/archived. Vì vậy, ma trận dưới đây chỉ mô tả quyền xem trước và tải nội dung file trong trường hợp tổng quan. Khi thực thi thực tế, backend vẫn áp dụng thêm các điều kiện như ACL DENY, ownership, trạng thái workflow và grant token.

3.12.1 Quyền xem và tải theo classification

Ma trận dưới đây mô tả quyền truy cập nội dung file trong luồng người dùng thông thường. Ký hiệu **X** là quyền xem trước nội dung, còn **T** là quyền tải file. Quyền tải chỉ được xét khi tài liệu ở trạng thái PUBLISHED và đã có version; quyền xem trước có thể áp dụng ở các trạng thái workflow khác nếu tài liệu đã có version. Trong mọi trường hợp, ACL DENY phù hợp được ưu tiên hơn quyền role. Các ô ghi “Owner/ACL” yêu cầu người dùng là chủ sở hữu hoặc có ACL ALLOW đúng loại quyền.

Bảng 3.4: Quyền xem và tải nội dung file theo vai trò và classification

Vai trò	PUBLIC	INTERNAL	CONFIDENTIAL	SECRET
Viewer	X: Có T: Có	X: Không T: Không	X: Owner/ACL READ T: Owner/ACL DOWNLOAD	X: Không T: Không
Editor	X: Có T: Có	X: Có T: Có	X: Owner/ACL READ T: Owner/ACL DOWNLOAD	X: Chỉ owner T: Không
Approver	X: Có T: Có	X: Có T: Có	X: Có T: Owner/ACL DOWNLOAD	X: Có T: Owner/ACL DOWNLOAD
Compliance Officer	X: Không T: Không	X: Không T: Không	X: Không T: Không	X: Không T: Không
Admin	X: Có T: Có	X: Có T: Có	X: Có T: Có	X: Có T: Có

Compliance Officer vẫn có thể đọc metadata của tài liệu PUBLISHED/ARCHIVED để phục vụ audit và evidence, nhưng không có quyền xem trước, stream, presign hoặc tải nội dung file ở bất kỳ classification nào. Share link có grant hợp lệ là một cơ chế cấp quyền riêng và không được biểu diễn trong ma trận role cơ sở này.

3.13 Chính sách Compliance Officer

Một quyết định thiết kế quan trọng của DocVault là tách rõ quyền compliance khỏi quyền truy cập nội dung file. Compliance Officer cần xem metadata và audit để đánh giá tuân thủ, nhưng có quyền audit không đồng nghĩa với được tải file. Tách hai quyền này lại giảm rủi ro ai đó lạm dụng vai trò compliance để chạm tới tài liệu nhạy cảm.

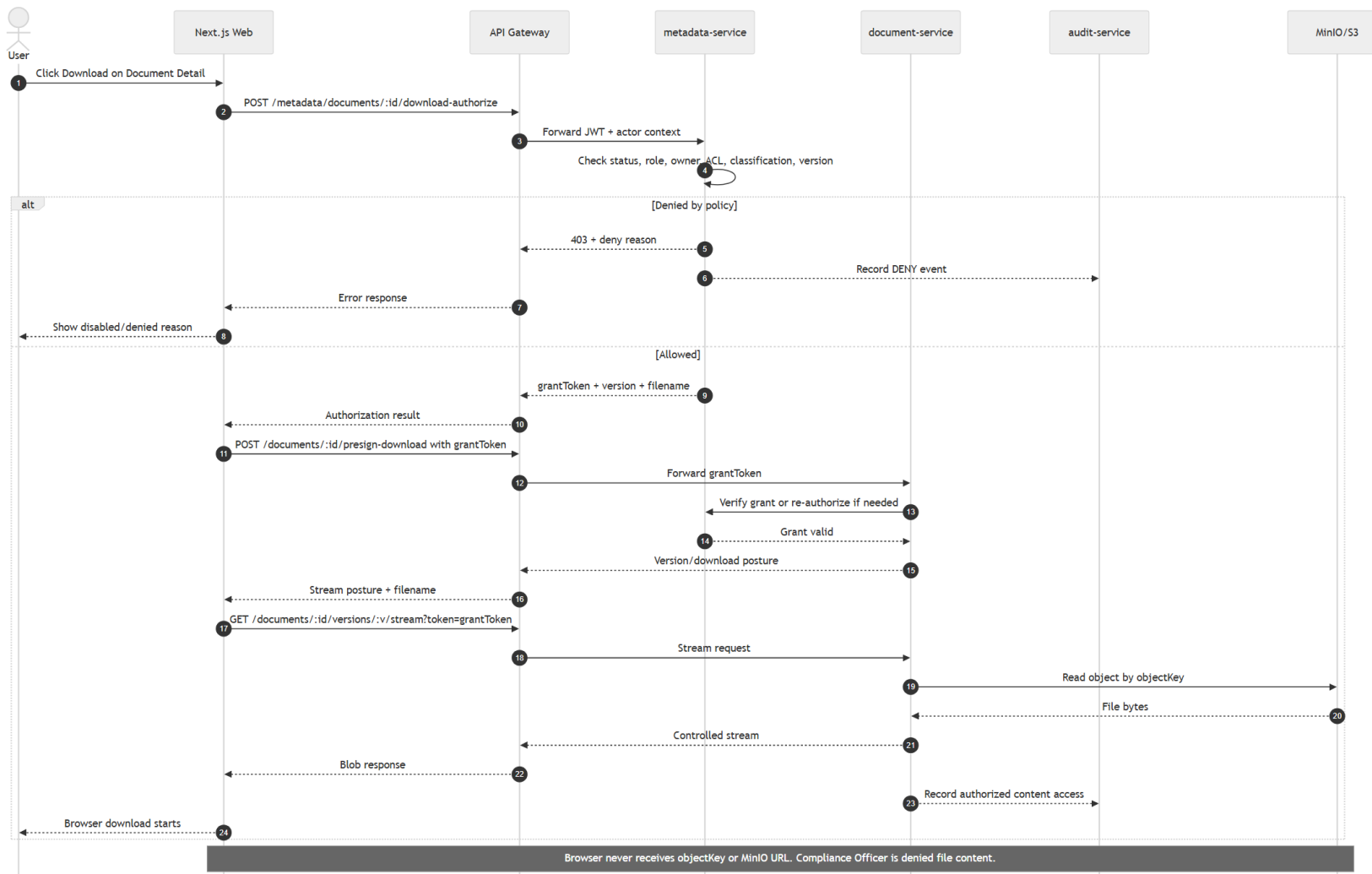
Trong DocVault, Compliance Officer:

- được truy vấn audit events;
- được verify audit hash-chain;
- được xem security summary;
- được xem retention evidence;
- được export evidence packet;
- không được preview, stream, presign hoặc download nội dung file.

Quy tắc này được thực thi ở nhiều tầng: frontend permission helper để hiển thị lý do từ chối, gateway proxy để chặn đường gọi phù hợp và service policy để đảm bảo backend là nguồn quyết định cuối cùng.

3.14 Thiết kế preview/download authorization

Download được thiết kế thành luồng hai bước. Ở bước đầu, frontend gọi metadata-service thông qua gateway để xin authorize. Metadata-service kiểm tra policy và nếu hợp lệ sẽ cấp grant token ngắn hạn. Ở bước sau, frontend dùng grant token để gọi document-service. Document-service xác minh grant token hoặc tái authorize trước khi cấp presigned URL hoặc stream endpoint.

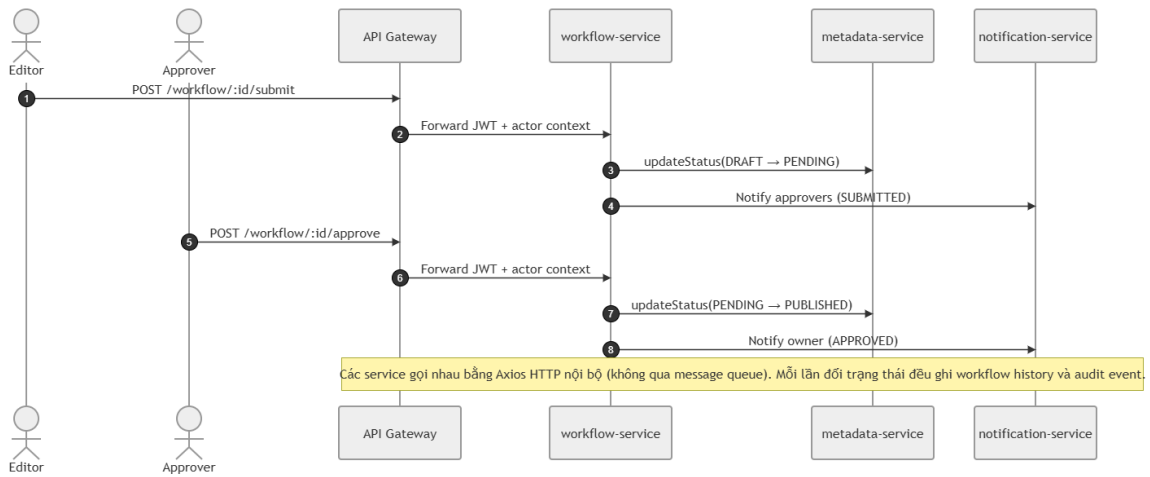


Hình 3.6: Sequence cấp quyền download và stream file

Với tài liệu **PUBLIC** hoặc **INTERNAL**, nếu policy cho phép, hệ thống có thể trả về presigned URL. Với tài liệu **CONFIDENTIAL** hoặc **SECRET**, hệ thống trả về `url: null`, `watermarkRequired: true` và `streamingEndpoint`, để document-service nắm được đường tải nội dung nhạy cảm và kiểm soát chặt hơn.

3.15 Luồng phê duyệt tài liệu

Workflow phê duyệt là luồng đa-service tiêu biểu của DocVault. Editor gửi tài liệu duyệt và Approver phê duyệt; trong cả hai bước, workflow-service đóng vai trò điều phối: gọi metadata-service để đổi trạng thái, rồi gọi notification-service để thông báo cho đúng người nhận. Hình 3.7 mô tả luồng chính từ **DRAFT** đến **PUBLISHED**.



Hình 3.7: Sequence luồng phê duyệt tài liệu (submit và approve)

Điểm thiết kế đáng chú ý là các service giao tiếp với nhau bằng Axios HTTP nội bộ, không qua message queue, và mỗi lần đổi trạng thái đều ghi workflow history kèm audit event. Luồng từ chối (reject) đưa tài liệu **PENDING** quay lại **DRAFT** kèm lý do, theo cùng mô hình điều phối này.

3.16 Malware scanning và DLP

Document-service thực hiện scanning trước khi ghi object vào MinIO. Với payload EICAR trong môi trường demo, upload bị chặn và không tạo object cũng như không đăng ký metadata version. Đây là một điểm quan trọng: nếu scan chỉ phát hiện sau khi file đã lưu, hệ thống sẽ phải xử lý cleanup phức tạp hơn.

DLP scan phát hiện một số pattern nhạy cảm như email, số điện thoại/số định danh và từ khóa như **secret**, **confidential**, **internal only**. Khi phát hiện dữ liệu nhạy cảm trong tài liệu public/internal, hệ thống nâng classification lên **CONFIDENTIAL**. Người dùng

không có quyền không thể hạ classification xuống public/internal nếu không có lý do override hợp lệ.

3.17 Audit ingest boundary

Audit-service không cho người dùng thường append audit event trực tiếp. Service tin cậy phải gửi kèm header service token `x-docvault-service-token`, cấu hình qua `AUDIT_INGEST_TOKEN`. Rào này chặn việc một người có JWT hợp lệ tạo audit event giả để che giấu hoặc làm sai lệch lịch sử.

3.18 Ghi nhận hoạt động hệ thống và khuyến nghị bảo mật

Trong DocVault, audit log không chỉ dùng để lưu lại các thao tác trên tài liệu như tạo mới, cập nhật metadata, upload version, gửi duyệt, phê duyệt, xem trước hoặc tải xuống. Một phần khác cũng quan trọng là ghi nhận các hoạt động diễn ra trong quá trình vận hành hệ thống, đặc biệt là các hoạt động liên quan đến quyền truy cập, bảo mật và quản trị.

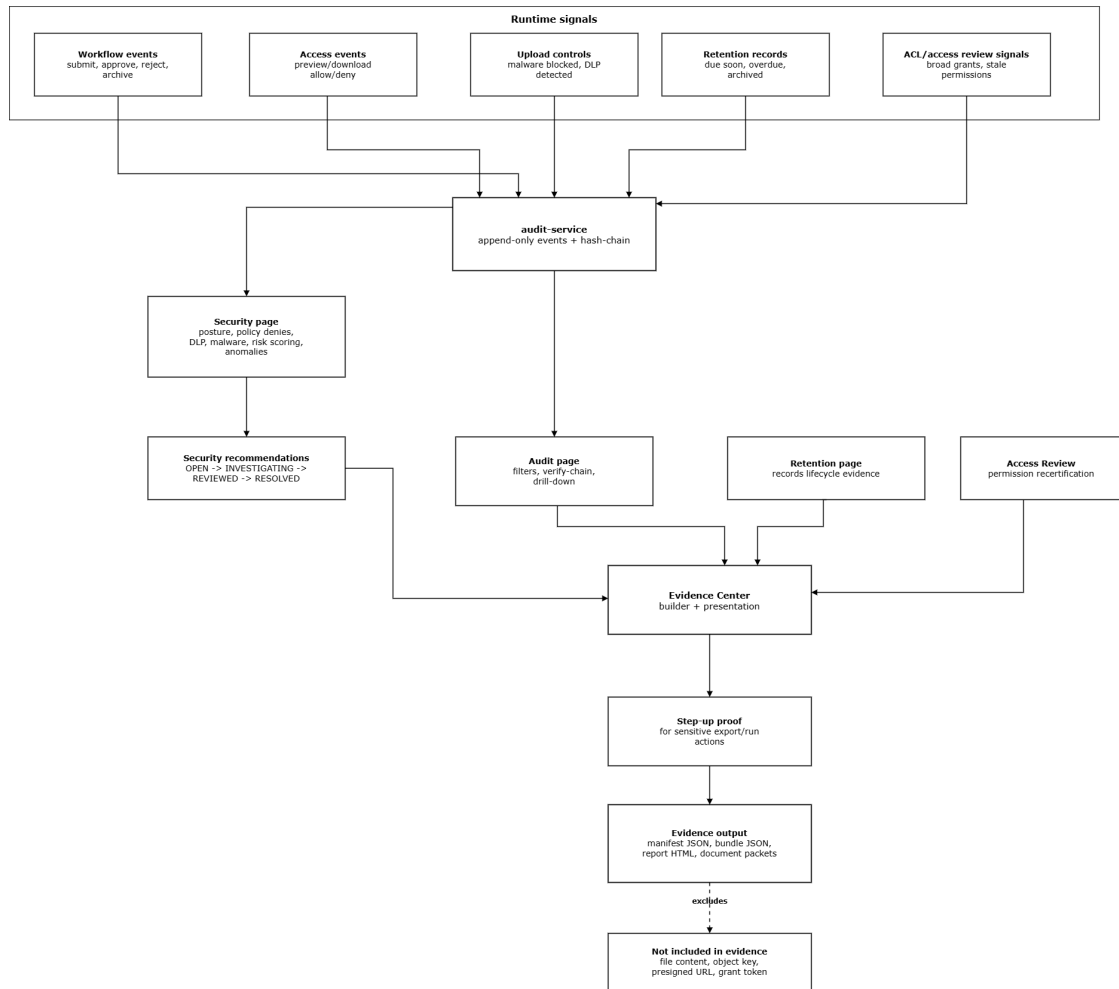
Các sự kiện được ghi nhận có thể chia thành một số nhóm chính. Nhóm thứ nhất là các quyết định truy cập, bao gồm trường hợp người dùng được phép xem/tải tài liệu và trường hợp bị từ chối quyền. Nhóm thứ hai là các sự kiện liên quan đến kiểm soát nội dung, chẳng hạn như phát hiện DLP, malware scanning hoặc các guardrail bảo mật. Nhóm thứ ba là các thao tác vận hành có ảnh hưởng đến dữ liệu hoặc trạng thái hệ thống, ví dụ chạy retention job, thực hiện thao tác nhạy cảm hoặc cập nhật trạng thái xử lý khuyến nghị bảo mật. Ngoài ra, hệ thống cũng ghi nhận các thay đổi về thành viên, vai trò, comment và share link.

Cách tổ chức này giúp audit log có vai trò rộng hơn so với một lịch sử thao tác đơn thuần. Khi một hành vi cần được kiểm tra lại, quản trị viên có thể xem các sự kiện liên quan để biết bối cảnh của hành động đó. Chẳng hạn, một lần truy cập bị từ chối không chỉ cho biết người dùng không có quyền, mà còn cho biết thời điểm truy cập, đối tượng bị truy cập và lý do xử lý của hệ thống.

Từ các audit event đã ghi nhận, hệ thống có thể tạo hoặc cập nhật các khuyến nghị bảo mật. Các khuyến nghị này không nhằm tự động kết luận người dùng vi phạm, mà đóng vai trò như một gợi ý để quản trị viên xem xét. Trong quá trình xử lý, quản trị viên có thể chuyển trạng thái khuyến nghị qua các bước như đang mở, đang điều tra, đã xem xét hoặc đã xử lý. Nhờ đó, hệ thống vừa lưu được bằng chứng kỹ thuật, vừa hỗ trợ quá trình kiểm tra và phản hồi sau khi phát hiện hành vi đáng chú ý.

3.19 Luồng compliance evidence

Compliance evidence trong DocVault được xây dựng từ nhiều tín hiệu runtime: workflow event, access event, upload control, retention record và access review signal. Các tín hiệu này được audit-service ghi nhận theo dạng append-only và hash-chain, sau đó được các màn hình Security, Audit, Retention, Access Review và Evidence Center tổng hợp thành bằng chứng có thể xuất ra ngoài.



Hình 3.8: Luồng tổng hợp compliance evidence trong DocVault

DocVault tổng hợp compliance evidence theo một luồng ba giai đoạn từ tín hiệu runtime đến bằng chứng có thể xuất ra ngoài (Hình 3.8). Giai đoạn nguồn là các tín hiệu phát sinh trong vận hành: workflow event, access event, upload control (malware/DLP), retention record và access review signal. Giai đoạn lưu trữ là audit-service ghi nhận các tín hiệu này theo dạng append-only kèm hash-chain, tạo thành một dòng sự kiện có thể kiểm tra tính toàn vẹn. Giai đoạn tổng hợp là các màn hình Security, Audit, Retention, Access Review và Evidence Center đọc dòng sự kiện đó và kết tinh thành bằng chứng.

Luồng được thiết kế một chiều từ nguồn đến evidence, nên bằng chứng luôn dẫn xuất từ tín hiệu đã ghi chứ không phải dữ liệu nhập tay, khó bị làm giả sau sự việc. Ngay trước

khi xuất, một lần ranh lọc bảo đảm evidence packet không bao giờ chứa file content, object key, presigned URL hay grant token. Compliance Officer nhờ đó kiểm tra được tính tuân thủ và truy vết hoạt động, còn quyền của họ không bị nói rộng thành quyền đọc nội dung tài liệu.

3.20 API contract tổng quan

API của hệ thống được expose qua gateway với base path `/api`. Các nhóm endpoint chính gồm:

Bảng 3.5: Nhóm API chính của DocVault

Nhóm API	Ví dụ endpoint	Mục đích
Auth	<code>/auth/login,</code> <code>/auth/logout</code> <code>/auth/me,</code>	Đăng nhập, callback, logout và lấy người dùng hiện tại.
Metadata	<code>/metadata/documents,</code> <code>/metadata/documents/:id</code>	Tạo, đọc, cập nhật metadata tài liệu.
ACL	<code>/metadata/documents/:id/acl</code>	Thêm/xem rule ACL theo tài liệu.
Version	<code>/metadata/documents/:id/versions</code>	Đăng ký version metadata sau upload.
Download authorization	<code>/metadata/documents/:id/download-authorize</code>	Xin quyền download và grant token.
Preview authorization	<code>/metadata/documents/:id/preview-authorize</code>	Xin quyền preview nội dung file.
Document blob	<code>/documents/:id/upload</code> <code>/documents/:id/presign-download</code> <code>/documents/:id/versions/:v/stream</code>	Upload, presign hoặc stream file.
Workflow	<code>/workflow/:id/submit,</code> <code>/workflow/:id/approve</code>	Chuyển trạng thái tài liệu.
Audit	<code>/audit/query,</code> <code>/audit/verify-chain,</code> <code>/audit/security-summary</code>	Truy vấn audit và kiểm tra hash-chain.
Evidence	<code>/metadata/documents/:id/evidence-packet</code>	Xuất evidence packet metadata-only.

Nhóm API	Ví dụ endpoint	Mục đích
Retention	/metadata/retention/	Xem và chạy retention demo.
	documents	
	/metadata/retention/run	

Các thiết kế về dữ liệu, policy, workflow, audit và API trong chương này tạo thành nền tảng cho phần hiện thực. Chương tiếp theo trình bày cách các thiết kế đó được triển khai trong mã nguồn, service runtime, hạ tầng Kubernetes và quy trình DevSecOps.

CHƯƠNG 4

HIỆN THỰC VÀ TRIỂN KHAI HỆ THỐNG

Chương này trình bày quá trình hiện thực và triển khai hệ thống DocVault. Nội dung chương được tổ chức thành các nhóm chính gồm hiện thực ứng dụng, hạ tầng triển khai trên AWS EKS, quy trình GitOps và CI/CD, các kiểm soát bảo mật trong pipeline, vận hành hệ thống và đánh giá trạng thái triển khai.

4.1 Hiện thực ứng dụng DocVault

4.1.1 Cấu trúc monorepo

DocVault được tổ chức dưới dạng monorepo sử dụng pnpm và Turbo, gom frontend, backend services, shared libraries, infrastructure và tài liệu vào cùng một repository. Các thư mục chính gồm:

- `apps/web`: ứng dụng frontend Next.js.
- `services/gateway`: API Gateway.
- `services/metadata-service`: service quản lý metadata, ACL, policy và retention.
- `services/document-service`: service xử lý upload/download file và MinIO/S3.
- `services/workflow-service`: service xử lý state machine tài liệu.
- `services/audit-service`: service ghi/truy vấn audit log và verify hash-chain.
- `services/notification-service`: notification sink cho môi trường demo/dev.
- `libs/auth`, `libs/contracts`, `libs/throttler`: các thư viện dùng chung.
- `infra`: Docker Compose, Kubernetes, Helm, Argo CD, Terraform và các cấu hình hạ tầng.
- `docs`: tài liệu thiết kế, hướng dẫn chạy, evidence và screenshot.

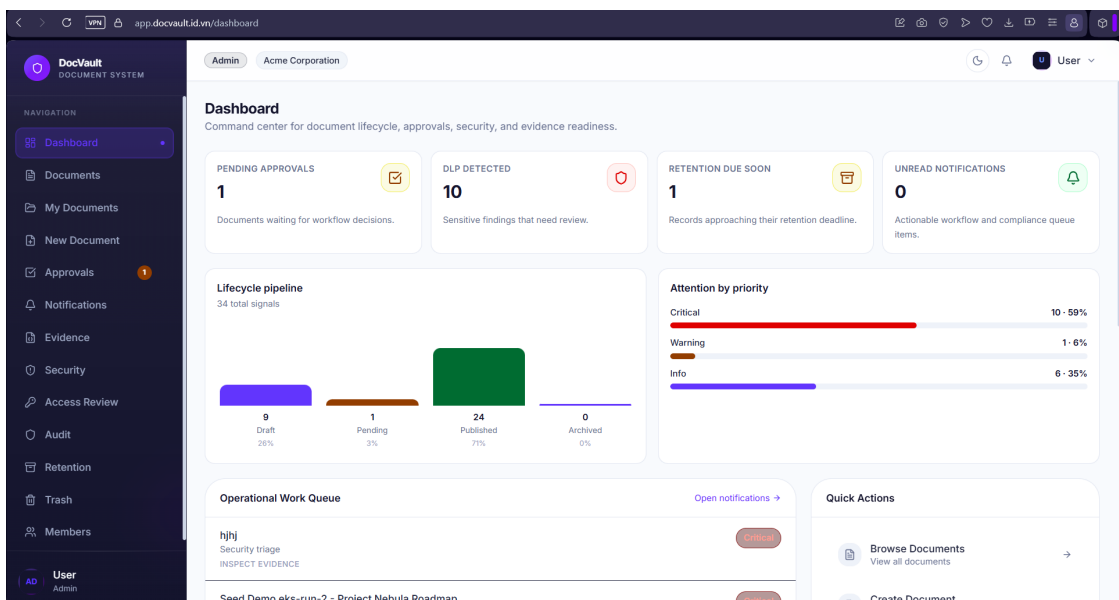
Monorepo giúp báo cáo, code, pipeline và evidence nằm chung một vòng đời. Khi một thay đổi chạm tới API contract hoặc security policy, nhóm có thể cập nhật code, test và tài liệu trong cùng một pull request.

4.1.2 Frontend Next.js

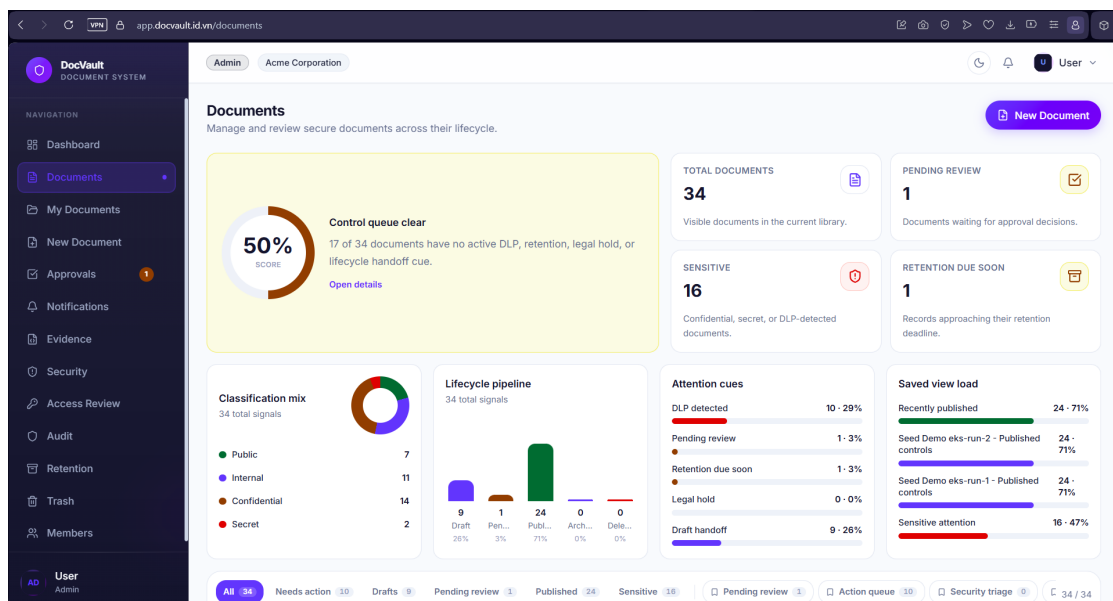
Frontend được xây dựng bằng Next.js, React, TanStack Query, React Hook Form, Zod, Tailwind CSS và Radix UI. Ứng dụng sử dụng App Router và tổ chức theo các nhóm route đã đăng nhập hoặc chưa đăng nhập. Frontend gọi gateway qua biến môi trường NEXT_PUBLIC_API_BASE_URL, mặc định trỏ đến `http://localhost:3000/api` trong môi trường local.

Frontend làm nhiều việc hơn là hiển thị danh sách tài liệu. Ứng dụng đã bao phủ nhiều luồng vận hành quan trọng:

- Đăng nhập SSO qua Keycloak và quản lý phiên.
- Dashboard tổng quan tài liệu, pending approval, DLP, retention và readiness.
- Documents và My Documents với lọc nâng cao, quick views, saved views, folder tree và phân trang.
- New Document và Edit Document cho tạo/chỉnh metadata và upload file.
- Document Detail với preview/download, version history, workflow timeline, ACL, comments, activity, share link, legal hold, DLP findings, AI guardrails và evidence links.
- Approvals với SLA summary, review drawer, approve/reject và reason preset.
- Audit, Security, Evidence Center, Retention và Access Review cho compliance/admin.
- Notifications, Trash, Members, Profile, Settings và Demo Kit.



Hình 4.1: Màn hình dashboard của DocVault



Hình 4.2: Màn hình danh sách tài liệu

4.1.2.1 Permission-aware UI

Frontend có helper phân quyền để xác định action nào nên hiển thị, action nào nên bị disable và lý do deny là gì. Chẳng hạn, Compliance Officer được thấy lý do bị chặn download thay vì chỉ thấy nút biến mất. Dù vậy, helper frontend không phải boundary bảo mật cuối cùng; backend mới là nơi quyết định policy, tránh việc người dùng tự gọi API để vượt qua UI.

4.1.3 Gateway

Gateway là entry point của hệ thống. Service này xác thực JWT từ Keycloak, kiểm tra role ở mức route, expose health endpoint, proxy request xuống service phía sau và truyền các header ngữ cảnh. Các header quan trọng gồm:

- authorization
- x-request-id
- x-user-id
- x-roles
- x-groups

Gateway proxy các nhóm route chính như `/metadata`, `/documents`, `/workflow`, `/audit` và `/notify`. Frontend vì thế chỉ cần biết một API base URL, trong khi bên trong hệ thống vẫn tách service theo bounded context.

4.1.4 Metadata-service

Metadata-service là nguồn sự thật cho tài liệu logic và policy. Service này sở hữu các dữ liệu và nghiệp vụ:

- document record;
- classification và tags;
- current version pointer;
- ACL entries;
- workflow history;
- download/preview authorization;
- retention evidence;
- access impact simulation;
- AI guardrails metadata-only.

Metadata-service là nơi quan trọng nhất để thực thi policy liên quan đến metadata và grant token. Khi document-service cần đăng ký version mới hoặc khi workflow-service cần cập nhật trạng thái, các service này gọi metadata-service thay vì ghi trực tiếp vào database.

Một ví dụ tiêu biểu cho nguyên tắc *backend authoritative* là quy tắc chặn Compliance Officer tải file. Quy tắc này được đặt ngay đầu hàm kiểm tra quyền download trong `policy.service.ts`, trước cả các bước kiểm tra trạng thái và ACL, để bảo đảm không có nhánh nào bỏ sót:

Listing 4.1: Quy tắc deny download của Compliance Officer trong `policy.service.ts`

```
private getDeniedReason(status: string, roles: string[]): string | null {
  if (roles.includes('compliance_officer')) {
    return 'Compliance officers are never allowed to download files';
  }
  if (status !== 'PUBLISHED') {
    return 'Only published documents can be downloaded';
  }
  return null;
}
```

Vì quyết định nằm ở backend chứ không ở frontend, người dùng compliance không thể lách bằng cách gọi thẳng API. Khi quy tắc trả về lý do từ chối, service ném `ForbiddenException` và ghi audit event tương ứng.

Grant token download/preview cũng do metadata-service cấp. Token có thời hạn 300 giây, được ký bằng HMAC-SHA256 và hỗ trợ xoay khóa qua `kid`, sau đó document-service xác minh trước khi phát file:

Listing 4.2: Cấp grant token ngắn hạn cho download

```
const encoded = Buffer.from(JSON.stringify(tokenPayload))
  .toString('base64url');
const signature = createHmac('sha256', signing.secret)
  .update(encoded)
  .digest('base64url');
return `${encoded}.${signature}`;
```

4.1.5 Document-service

Document-service xử lý phần blob file. Service này nhận upload multipart, tính checksum SHA-256, scan malware/DLP, sinh object key và upload file lên MinIO/S3. Sau khi upload thành công, service gọi metadata-service để đăng ký version mới.

Đối với download, document-service không cấp file một cách độc lập. Service xác minh grant token hoặc tái authorize thông qua metadata-service trước khi trả presigned URL hoặc stream endpoint. Với tài liệu nhạy cảm, service trả về stream endpoint để áp dụng posture kiểm soát tốt hơn.

4.1.6 Workflow-service

Workflow-service sở hữu logic chuyển trạng thái. Service đọc trạng thái hiện tại từ metadata-service, kiểm tra transition hợp lệ rồi yêu cầu metadata-service cập nhật trạng thái. Các transition chính:

- SUBMIT: DRAFT -> PENDING
- APPROVE: PENDING -> PUBLISHED
- REJECT: PENDING -> DRAFT
- ARCHIVE: PUBLISHED -> ARCHIVED

Sau mỗi transition, workflow-service phát audit event và gọi notification-service. Workflow vẫn là nơi điều phối nghiệp vụ, còn metadata-service vẫn là owner của dữ liệu trạng thái.

4.1.7 Audit-service

Audit-service ghi và truy vấn audit log. Service hỗ trợ append audit event từ các service tin cậy, query theo actor/action/resource/result/time, verify hash-chain và security summary. Endpoint audit query chỉ dành cho Compliance Officer hoặc Admin.

Hash-chain giúp phát hiện sửa đổi audit event ngoài luồng. Nếu một event cũ bị chỉnh trực tiếp trong database, verify-chain trả về trạng thái không hợp lệ. Trong phạm vi đồ án, đây là cơ chế tamper-evident phù hợp để chứng minh ý tưởng kiểm soát audit.

4.1.8 Notification-service

Notification-service hiện là notification sink cho môi trường dev/demo. Service nhận payload từ workflow hoặc các sự kiện liên quan và lưu/log để frontend hiển thị notification center. Đây chưa phải hệ thống email, SMS hoặc push notification hoàn chỉnh, nhưng đủ để mô phỏng luồng thông báo trong sản phẩm. Trạng thái chưa hoàn thiện này được ghi nhận lại trong phần hạn chế ở Mục 5.7.

4.1.9 Hạ tầng local

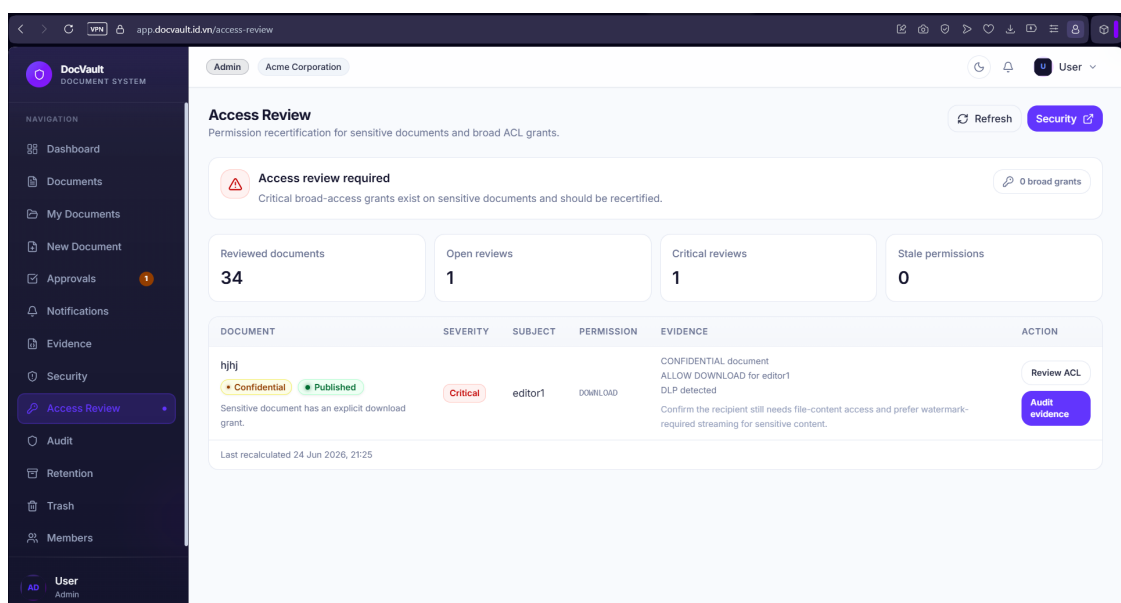
Môi trường local sử dụng Docker Compose để chạy các dependency:

- PostgreSQL cho metadata.
- MongoDB cho audit.
- MongoDB Express để quan sát dữ liệu audit trong dev.
- MinIO cho object storage.
- MinIO init job để tạo bucket và bật cấu hình cần thiết.
- Keycloak cho identity, realm, roles và demo users.

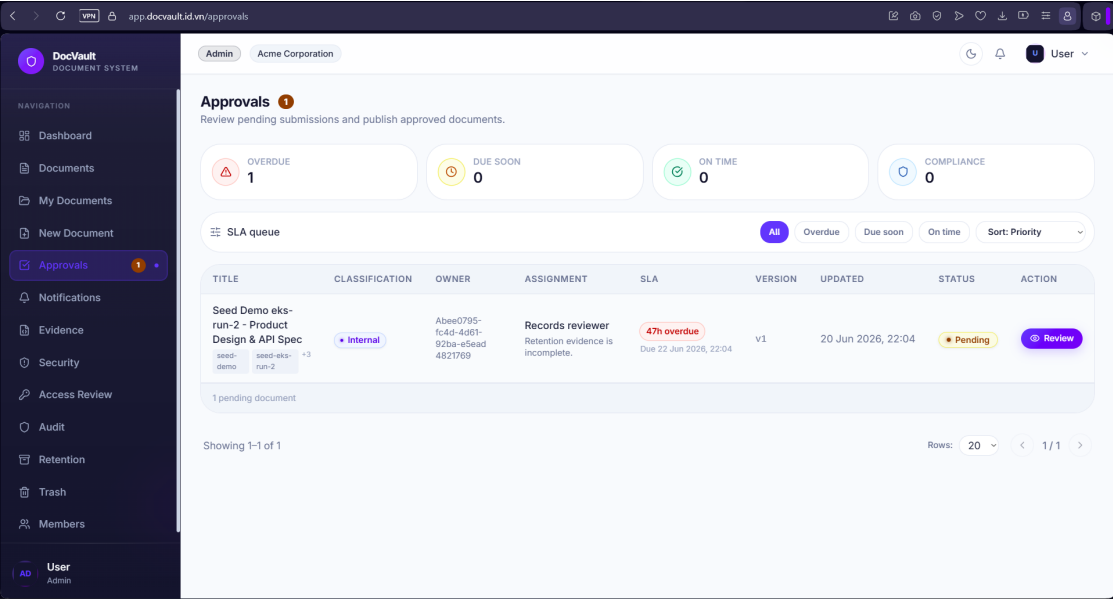
Các service backend chạy trên các port 3000 đến 3005; frontend chạy trên port 3006 (script `pnpm --filter web dev`) để tránh xung đột với gateway và các service backend.

4.1.10 Các màn hình minh họa

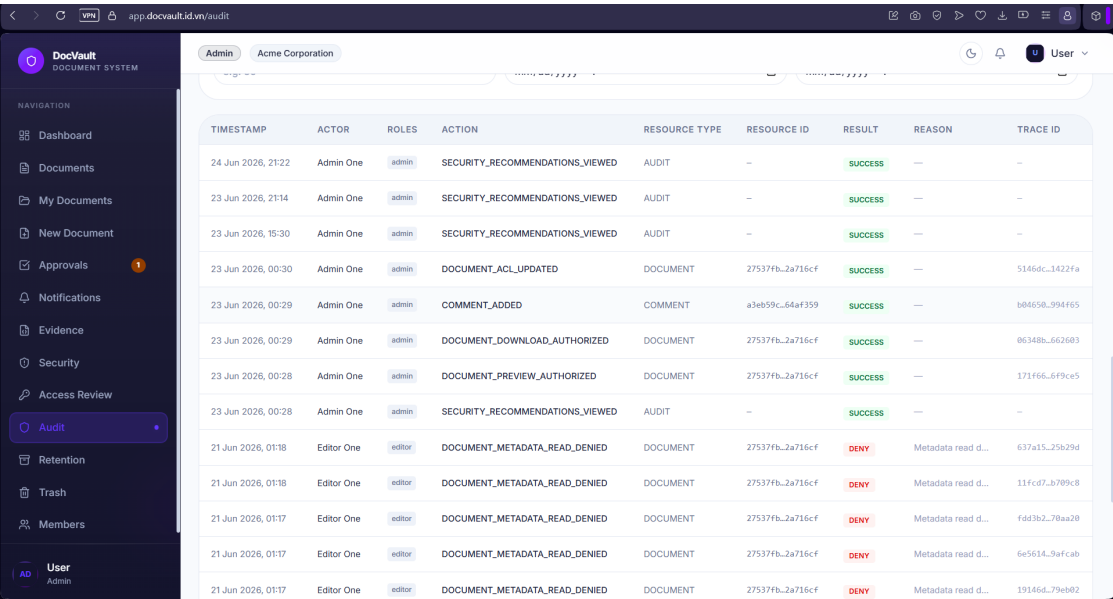
Các hình dưới đây thể hiện một số giao diện chính đã triển khai trong frontend.



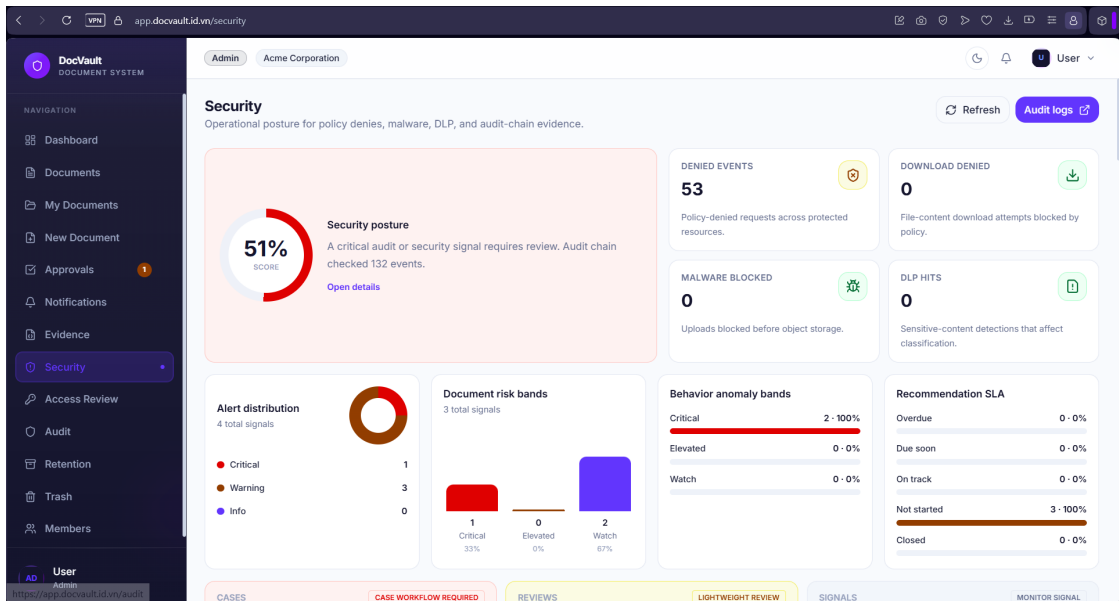
Hình 4.3: Màn hình rà soát quyền truy cập



Hình 4.4: Màn hình phê duyệt tài liệu



Hình 4.5: Màn hình audit log



Hình 4.6: Màn hình security posture

4.1.11 Luồng demo ứng dụng

Luồng demo chính được thiết kế để chứng minh cả nghiệp vụ và bảo mật:

1. Editor đăng nhập, tạo tài liệu và upload file.
2. Editor submit tài liệu để chuyển từ Draft sang Pending.
3. Approver đăng nhập, mở hàng đợi Approvals và approve tài liệu.
4. Viewer đăng nhập, xem tài liệu đã Published và download nếu policy cho phép.
5. Compliance Officer đăng nhập, mở Audit/Security/Evidence nhưng bị từ chối pre-view/download nội dung file.

Luồng này thể hiện đủ các thành phần chính của hệ thống: identity, gateway, metadata, document storage, workflow, audit, notification và frontend.

4.2 Hạ tầng triển khai trên AWS và EKS

4.2.1 Mục tiêu và phạm vi triển khai

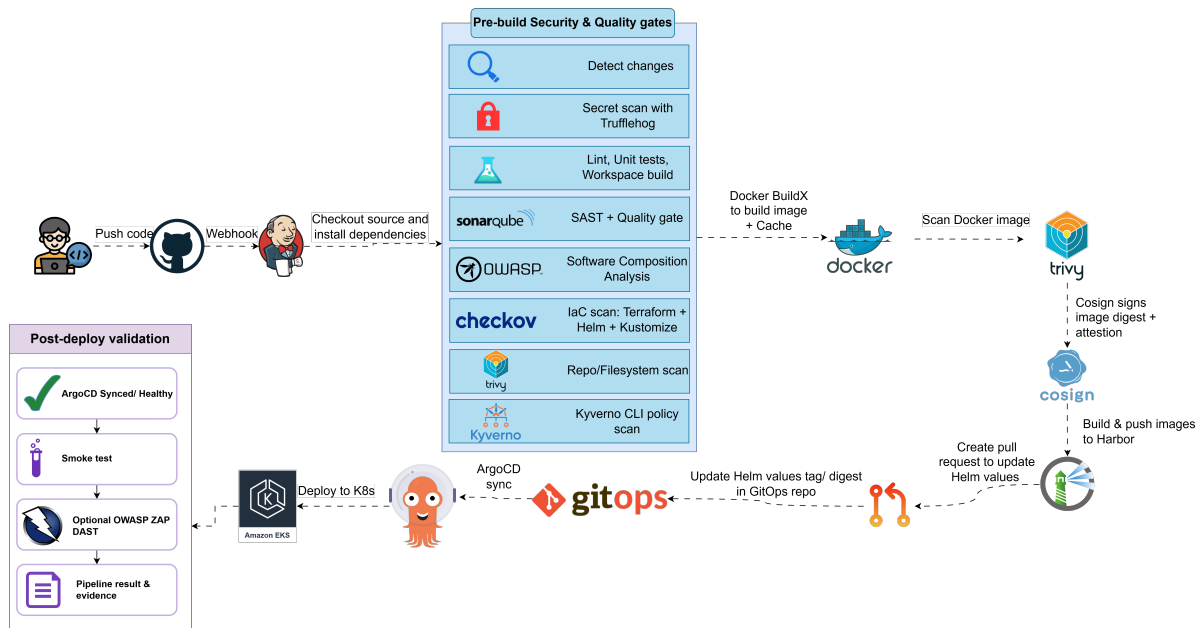
Phần DevSecOps là một trong hai phần chính của đề tài DocVault. Nếu các mục trước tập trung vào cách hệ thống quản lý tài liệu được hiện thực, thì phần này trình bày đường đi của một thay đổi từ repository đến môi trường cloud-native. Mục tiêu là tạo ra quy trình phát hành có kiểm soát, có thể lặp lại và có bằng chứng cho từng giai đoạn từ mã nguồn đến workload đang chạy.

Quy trình DevSecOps của DocVault được xây dựng để tự động hóa release lên AWS EKS và thu thập bằng chứng cần thiết cho việc kiểm tra lại. Jenkins chịu trách nhiệm chạy các gate CI, tạo container artifact và cập nhật trạng thái mong muốn trên nhánh GitOps. Argo CD tiếp tục reconcile trạng thái đó xuống cụm EKS. Cách làm này giúp mỗi lần phát hành có thể được đối chiếu lại thông qua commit, image tag/digest, trạng thái Argo CD, smoke test và các báo cáo scan.

Trong thiết kế này, Jenkins không triển khai trực tiếp tài nguyên lên Kubernetes. Thành phần này thực hiện kiểm tra mã nguồn, xây dựng và quét container image, đẩy artifact lên Harbor, sau đó cập nhật trạng thái mong muốn trên nhánh GitOps. Argo CD chịu trách nhiệm đối chiếu trạng thái trong Git với trạng thái thực tế và đồng bộ các thay đổi xuống cụm EKS.

Thiết kế trên hướng đến các mục tiêu sau:

- Hạ tầng AWS được mô tả bằng Terraform để có thể tái tạo, kiểm tra và quản lý thay đổi.
- Mã nguồn, thư viện phụ thuộc, secret, filesystem, container image và cấu hình IaC được kiểm tra tự động.
- Image triển khai được lưu trong registry riêng, gắn tag theo Git commit và cố định bằng digest.
- Trạng thái triển khai được lưu trong Git, qua đó hình thành lịch sử thay đổi và cơ sở rollback.
- Giá trị bí mật của ứng dụng không được ghi trực tiếp trong repository mà được đồng bộ từ AWS Secrets Manager.
- Trạng thái sau triển khai được xác minh bằng Argo CD health check, smoke test và DAST.
- Metric, log và trạng thái workload được tập trung để phục vụ giám sát và thu thập bằng chứng.



Hình 4.7: Luồng tổng quan CI, CD theo GitOps và kiểm tra sau triển khai

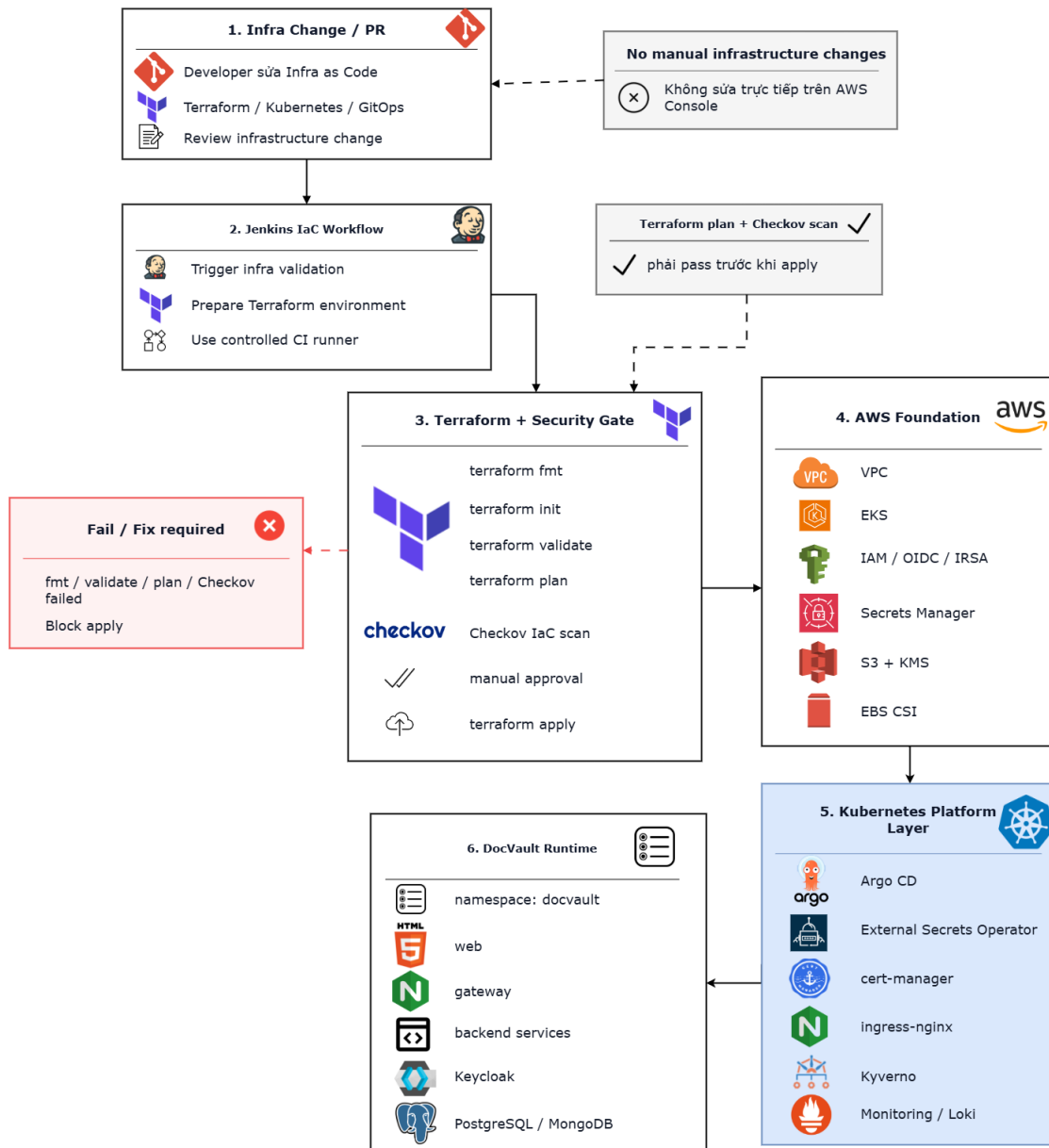
DocVault tổ chức quy trình phát hành thành ba phân đoạn nối tiếp, từ kiểm tra mã nguồn đến triển khai và xác minh sau triển khai (Hình 4.7). Phân đoạn CI bắt đầu khi mã nguồn được đẩy lên Git: Jenkins checkout, cài dependency, quét secret, chạy test/lint, rồi áp các security gate gồm SCA, Trivy, SonarQube và kiểm tra IaC. Phân đoạn đóng gói build Docker image, quét image bằng Trivy, đẩy lên Harbor, lấy digest và tùy chọn ký bằng Cosign; chỉ image vượt qua mọi gate mới được phát hành. Phân đoạn CD theo GitOps là nơi Jenkins cập nhật desired state trên nhánh GitOps, Argo CD đối chiếu và đồng bộ xuống EKS, sau đó pipeline chạy kiểm tra sau triển khai gồm Argo CD health check, smoke test và DAST.

Ranh giới quan trọng nhất trong thiết kế là Jenkins không trực tiếp `kubectl apply` lên cụm; nó chỉ ghi trạng thái mong muốn vào Git, còn Argo CD mới là thành phần thực thi thay đổi trên cluster. Trong phiên bản hiện tại, Jenkins tạo pull request để cập nhật nhánh GitOps (nhánh main) bằng commit có `[skip ci]` sau khi toàn bộ điều kiện phát hành được thỏa mãn.

4.2.2 Kiến trúc hạ tầng trên AWS EKS

Trước khi đi vào từng nhóm tài nguyên, Hình 4.8 mô tả luồng triển khai hạ tầng AWS EKS ở mức tổng thể. Thay đổi hạ tầng bắt đầu từ pull request trên các file IaC như Terraform, Kubernetes manifest hoặc GitOps values, được review trước khi Jenkins kích hoạt workflow kiểm tra. Jenkins chuẩn bị môi trường Terraform trên CI runner có kiểm soát, chạy `fmt`, `init`, `validate`, `plan` và Checkov scan; nếu một gate thất bại thì quá trình bị chặn và yêu cầu sửa lại trước khi apply.

AWS EKS Infrastructure Deployment Flow



Hình 4.8: Luồng triển khai hạ tầng AWS EKS bằng IaC

Sau khi kết quả `terraform plan` vượt qua kiểm tra và được phê duyệt, pipeline mới thực hiện `terraform apply` để tạo hoặc cập nhật nền tảng AWS gồm VPC, EKS, IAM/OIDC/IRSA, Secrets Manager, S3/KMS và EBS CSI. Trên nền tảng đó, lớp Kubernetes tiếp tục cài các controller vận hành như Argo CD, External Secrets Operator, cert-manager, ingress-nginx, Kyverno và monitoring/Loki. Runtime của DocVault sau cùng chạy trong namespace riêng với web, gateway, các backend service, Keycloak và cơ sở dữ liệu. Nguyên tắc chính của luồng này là không thay đổi hạ tầng trực tiếp trên AWS Console; mọi thay đổi đều đi qua mã nguồn, pipeline, kiểm tra bảo mật và lịch sử Git.

4.2.2.1 Hạ tầng dưới dạng mã với Terraform

Mã nguồn Terraform được đặt tại `infra/terraform/aws-eks` và được phân tách thành các module cục bộ theo từng nhóm trách nhiệm. Module `network` khởi tạo VPC, public subnet và private subnet trên hai Availability Zone. Module `eks-cluster` khởi tạo EKS control plane, managed node group, security group và các add-on nền tảng gồm CoreDNS, kube-proxy, VPC CNI và AWS EBS CSI Driver. Việc mô tả hạ tầng bằng mã giúp chuẩn hóa cấu hình, hỗ trợ kiểm tra thay đổi và tái tạo môi trường [26].

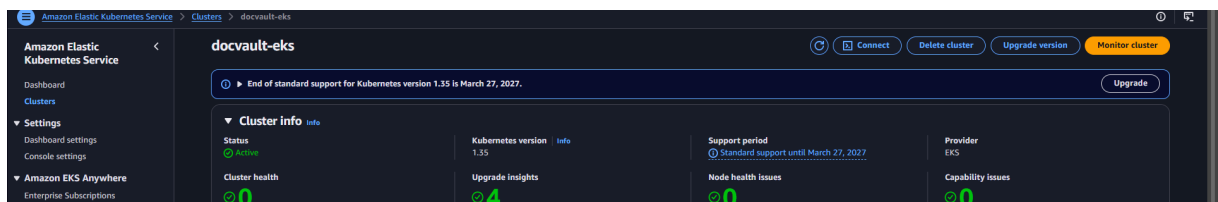
Cấu hình hỗ trợ hai phương án mạng. Phương án tối ưu chi phí đặt worker node trong public subnet và không sử dụng NAT Gateway, phù hợp với môi trường kiểm thử. Phương án triển khai production đặt worker node trong private subnet và sử dụng NAT Gateway. Endpoint công khai của Kubernetes API có thể được giới hạn bằng danh sách CIDR thay vì cho phép truy cập từ toàn bộ Internet.

Các biện pháp gia cố bảo mật chính được thể hiện trong Terraform gồm:

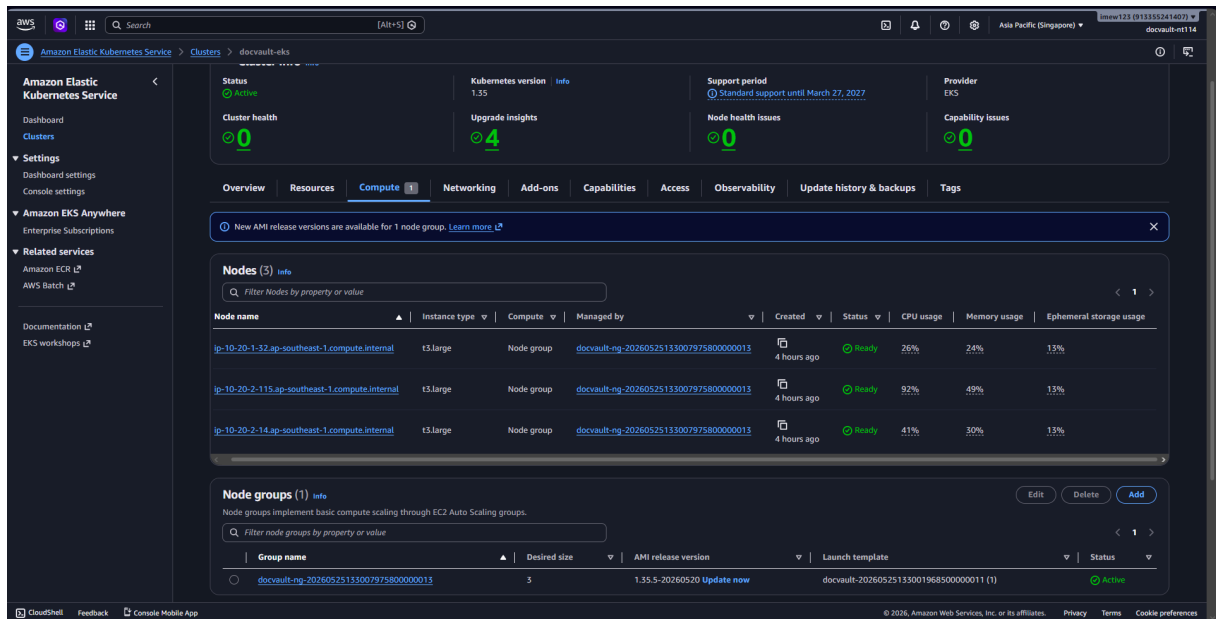
- Bật đầy đủ năm nhóm log EKS control plane: API, audit, authenticator, controller manager và scheduler [27].
- Bắt buộc IMDSv2 trên EC2 node.
- Mã hóa volume gốc của node bằng EBS gp3.
- Bật OIDC provider để cấp quyền AWS theo IRSA cho từng Kubernetes ServiceAccount [24].
- Tạo StorageClass gp3 và cài EBS CSI Driver để hỗ trợ PersistentVolume.
- Tách module lưu trữ tài liệu, backup cơ sở dữ liệu, External Secrets và quyền Jenkins.

Các cấu hình này bám theo các nhóm khuyến nghị thường gặp trong Kubernetes Security Checklist như kiểm soát truy cập API, network security, pod security, secrets, image và admission controller [28].

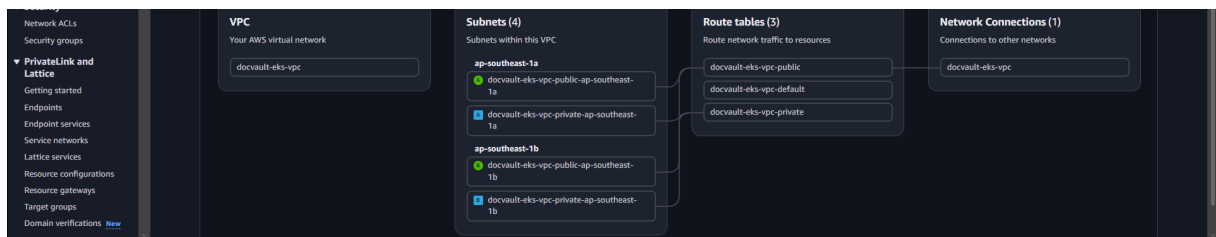
Cấu hình hiện tại sử dụng managed node group có khả năng co giãn từ một đến ba node, trong đó số lượng mong muốn của môi trường kiểm thử là hai node. Loại máy, dung lượng đĩa, phiên bản Kubernetes và số lượng node đều được khai báo dưới dạng biến, cho phép điều chỉnh mà không thay đổi logic của module.



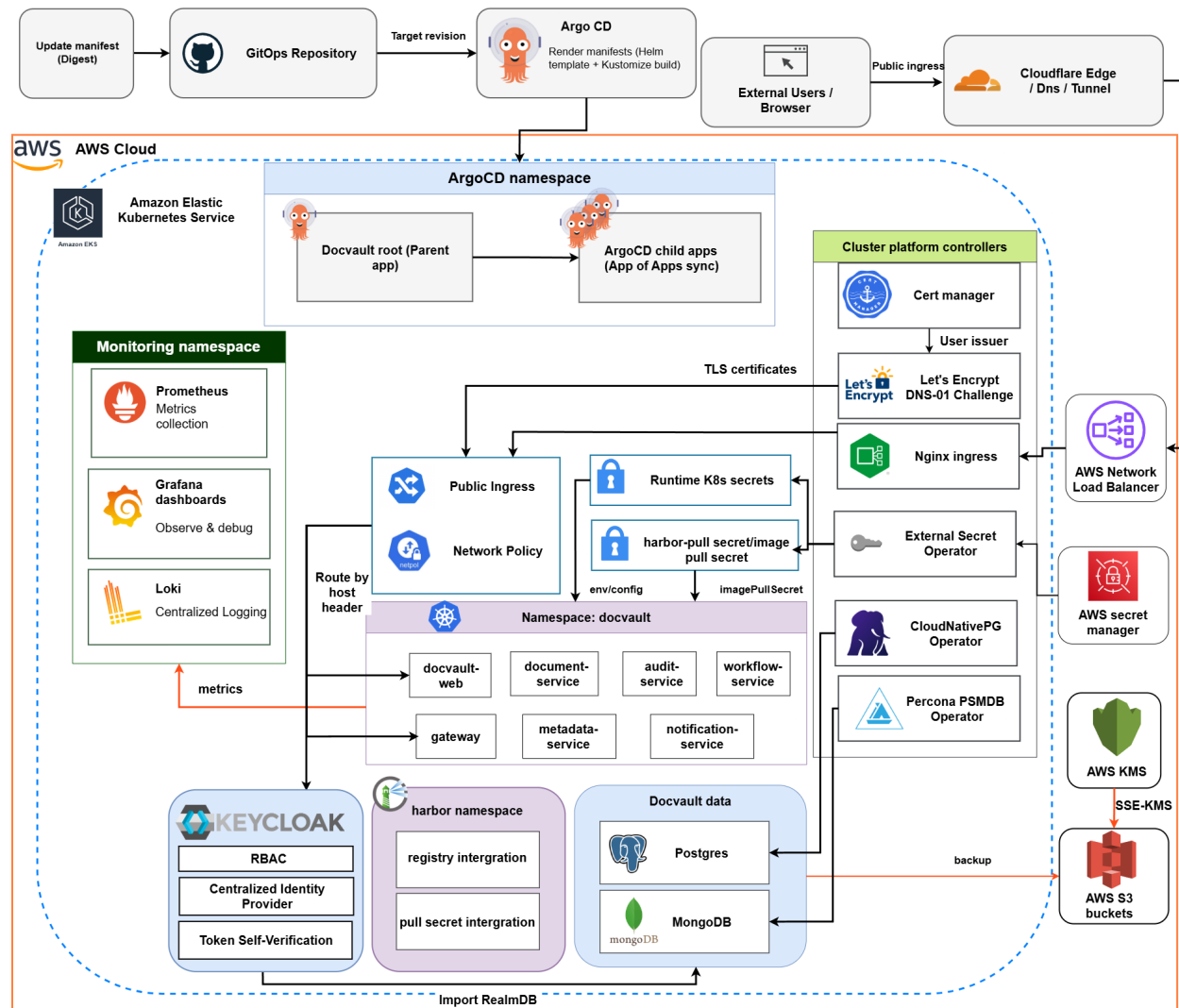
Hình 4.9: EKS cluster `docvault-eks` đang hoạt động trên khu vực Singapore



Hình 4.10: Managed node group và các worker node ở trạng thái sẵn sàng



Hình 4.11: Resource map của VPC gồm public/private subnet trên hai Availability Zone



Hình 4.12: Kiến trúc triển khai DocVault trên AWS EKS

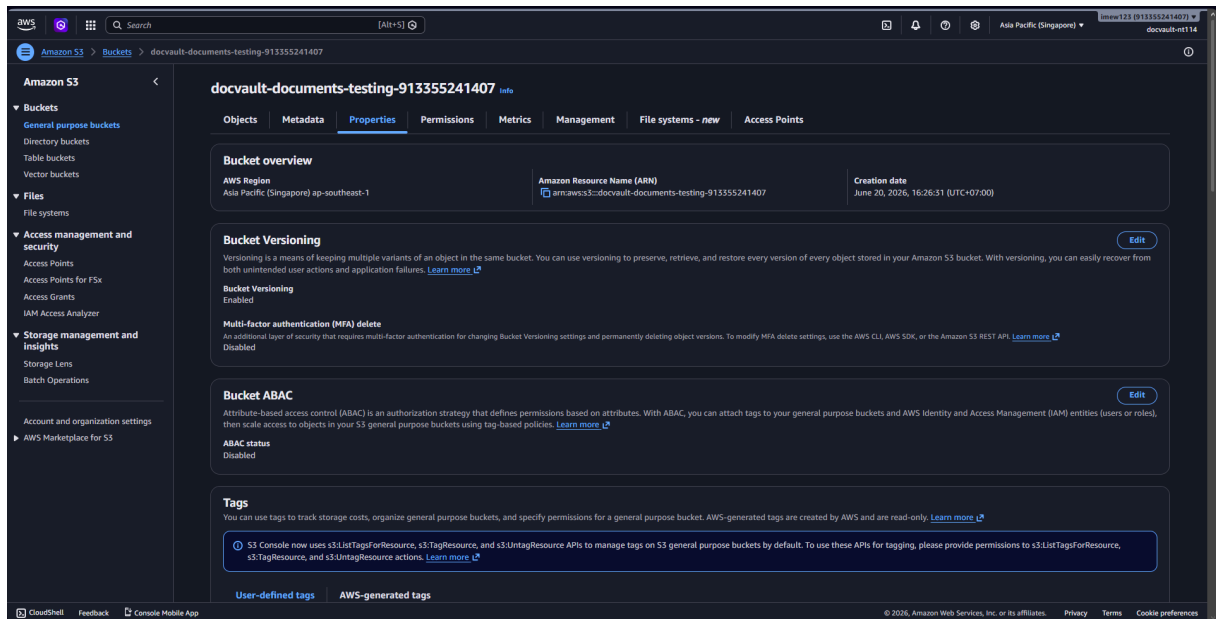
DocVault được triển khai trên AWS EKS theo bốn cụm thành phần, liên kết GitOps repository, Argo CD, các namespace ứng dụng và giám sát, nhóm controller nền tảng, Harbor, Keycloak cùng các dịch vụ AWS (Hình 4.12). Cụm điều khiển triển khai gồm GitOps repository và Argo CD: Argo CD theo dõi nhánh GitOps và đồng bộ desired state vào cụm, đóng vai trò cầu nối duy nhất giữa Git và Kubernetes. Cụm workload ứng dụng là các namespace chạy gateway, web và sáu backend service, được đóng gói bằng Helm. Cụm nền tảng gồm các controller dùng chung như ingress-nginx, cert-manager, External Secrets Operator và CloudNativePG. Cụm dịch vụ AWS bên ngoài cluster gồm S3 (file và backup), KMS (khóa mã hóa), Secrets Manager (khóa bí mật) và IAM/IRSA (cấp quyền cho ServiceAccount).

Thiết kế hạ tầng dựa trên nguyên tắc không để workload lộ trực tiếp ra Internet và không lưu khóa bí mật trong cụm. Lưu lượng truy cập từ người dùng đi từ Cloudflare đến AWS Load Balancer rồi vào ingress-nginx trước khi tới web/gateway. Các khóa bí mật đi từ AWS Secrets Manager qua External Secrets Operator để tạo Kubernetes Secret tại runtime, nên repository chỉ chứa tham chiếu chứ không chứa các giá trị khóa bí mật. Theo cùng nguyên tắc đó, file tài liệu được lưu trên S3 và mã hóa bằng KMS thay vì nằm trong cụm.

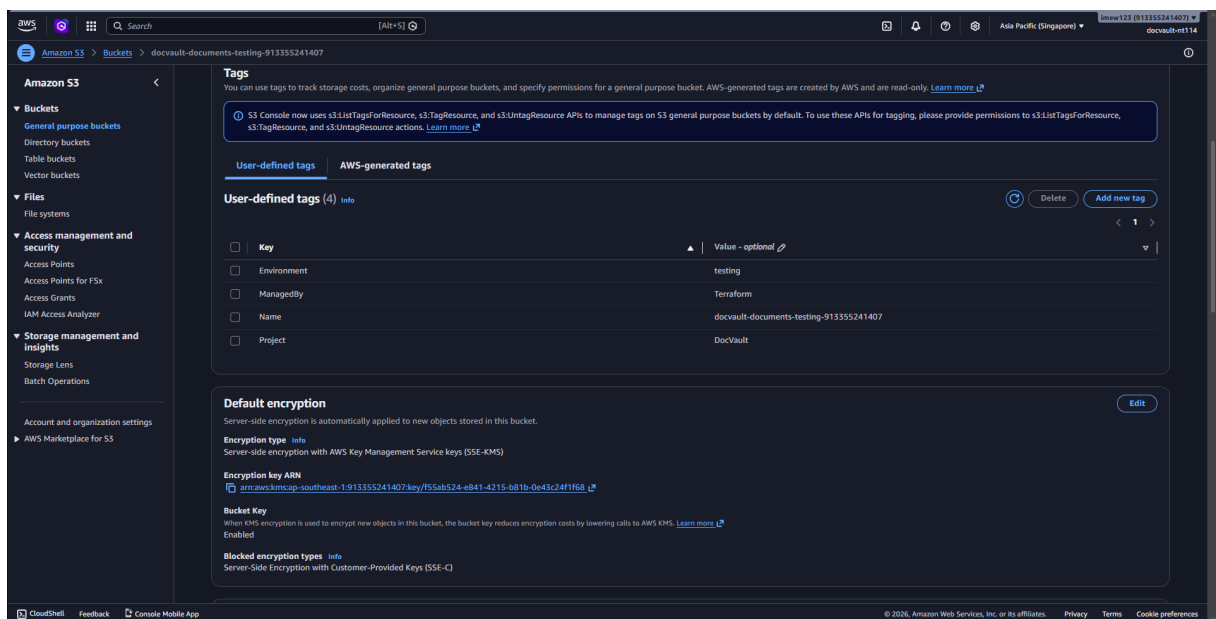
4.2.2.2 Lưu trữ tài liệu trên S3

Terraform khởi tạo bucket tài liệu, customer-managed KMS key và IAM role dành cho ServiceAccount của document-service. Bucket được chặn truy cập công khai, bật versioning, mã hóa mặc định bằng SSE-KMS, ghi access log sang bucket riêng, áp dụng lifecycle và phát sự kiện qua EventBridge. IAM policy chỉ cấp các thao tác cần thiết trên prefix object của document-service và KMS key tương ứng. Cách tổ chức này cho phép quản lý tập trung khóa mã hóa, chính sách sử dụng khóa và dấu vết truy cập [25].

DocVault dùng customer-managed KMS key thay vì key mặc định của AWS để tự kiểm soát key policy và tách khóa theo từng mục đích: một khóa cho EKS, một cho tài liệu, một cho backup PostgreSQL và một cho backup MongoDB. Nhờ vậy mỗi nhóm dữ liệu có ranh giới quyền giải mã riêng và sự cố ở một khóa không lan sang phần còn lại. Mỗi khóa bật xoay vòng tự động hằng năm: AWS thay vật liệu khóa nền theo chu kỳ trong khi ARN của khóa giữ nguyên, nên ứng dụng không phải cập nhật tham chiếu và dữ liệu mã hóa trước đó vẫn giải mã được bằng phiên bản khóa cũ. Cơ chế này giảm rủi ro khi một phiên bản khóa bị lộ mà không gây gián đoạn dịch vụ. Ngoài ra mỗi khóa đặt cửa sổ chờ xóa 30 ngày để tránh việc xóa nhầm làm mất vĩnh viễn dữ liệu đã mã hóa [25].



Hình 4.13: S3 bucket lưu tài liệu đã bật versioning trong môi trường testing



Hình 4.14: S3 bucket tài liệu được Terraform quản lý và mã hóa mặc định bằng SSE-KMS

4.2.2.3 Sao lưu và khôi phục cơ sở dữ liệu

DocVault có hai cơ sở dữ liệu trạng thái cần bảo vệ với mức ưu tiên khác nhau. Metadata PostgreSQL giữ ánh xạ object-key, ACL và lịch sử trạng thái tài liệu. Audit MongoDB giữ nhật ký kiểm toán có chuỗi băm SHA-256, là bằng chứng tuân thủ nên việc mất hoặc sai lệch dữ liệu ở đây nghiêm trọng hơn. Vì vậy hai cơ sở dữ liệu được sao lưu vào hai bucket S3 riêng, dùng hai KMS key riêng và hai IAM role IRSA riêng. Tách biệt như vậy giúp ranh giới quyền truy cập, lifecycle và quy trình xử lý sự cố của từng cơ sở dữ liệu độc lập với nhau, đồng thời tách hẳn khỏi bucket tài liệu.

Metadata PostgreSQL được triển khai bằng CloudNativePG với volume gp3. Cấu hình backup dùng plugin Barman Cloud ghi vào bucket S3 và KMS key riêng, xác thực bằng IRSA thay vì access key tĩnh. Tài nguyên `ScheduledBackup` chạy base backup hằng ngày kèm lưu trữ WAL liên tục và giữ 30 ngày, nhờ đó hệ thống có thể khôi phục về một thời điểm bất kỳ trong cửa sổ lưu giữ thay vì chỉ về mốc backup gần nhất [29].

```
PS D:\projects\docvaultNewest\docvault> kubectl get cluster,scheduledbackup,backup -n docvault
```

NAME	AGE	INSTANCES	READY	STATUS	PRIMARY
cluster.postgresql.cnpg.io/keycloak-postgres	6d23h	2	2	Cluster in healthy state	keycloak-postgres-2
cluster.postgresql.cnpg.io/metadata-postgres	4d19h	2	2	Cluster in healthy state	metadata-postgres-1

NAME	AGE	CLUSTER	LAST BACKUP
scheduledbackup.postgresql.cnpg.io/metadata-postgres-daily	4d19h	metadata-postgres	17h

NAME	AGE	CLUSTER	METHOD	PHASE	ERROR
backup.postgresql.cnpg.io/metadata-postgres-daily-20260622165741	4d19h	metadata-postgres	barmanObjectStore	completed	
backup.postgresql.cnpg.io/metadata-postgres-daily-20260622180000	4d17h	metadata-postgres	barmanObjectStore	completed	
backup.postgresql.cnpg.io/metadata-postgres-daily-20260623180000	2d23h	metadata-postgres	barmanObjectStore	completed	
backup.postgresql.cnpg.io/metadata-postgres-daily-20260624180000	2d17h	metadata-postgres	barmanObjectStore	completed	
backup.postgresql.cnpg.io/metadata-postgres-daily-20260625180000	41h	metadata-postgres	barmanObjectStore	completed	
backup.postgresql.cnpg.io/metadata-postgres-daily-20260626180000	11m	metadata-postgres	barmanObjectStore	completed	

Hình 4.15: Trạng thái backup CloudNativePG metadata với base backup hằng ngày và WAL đang được archive lên S3

Audit MongoDB được triển khai bằng Percona Server for MongoDB Operator dưới dạng replica set `rs0` ba thành viên, đặt anti-affinity theo hostname để các thành viên không nằm chung node. Sao lưu do Percona Backup for MongoDB (PBM) đảm nhận: một bản backup logical hằng ngày được nén gzip và giữ 30 bản, đẩy lên bucket MongoDB riêng với SSE-KMS [30]. Bên cạnh đó, PBM bật point-in-time recovery qua oplog với chu kỳ cắt lát 10 phút, nên ngoài các mốc backup hằng ngày hệ thống còn có thể khôi phục về gần như mọi thời điểm trong khoảng giữa hai lần backup. Đây là khả năng phục hồi chặt hơn so với chỉ snapshot định kỳ, phù hợp với yêu cầu toàn vẹn của dữ liệu audit.

```
PS D:\projects\docvaultNewest\docvault> kubectl exec -it audit-mongodb-rs0-0 -n docvault -c backup-agent -- pbm status
```

```
Cluster:
=====
rs0:
- audit-mongodb-rs0-0.audit-mongodb-rs0.docvault.svc.cluster.local:27017 [S]: pbm-agent [v2.12.0] OK
- audit-mongodb-rs0-1.audit-mongodb-rs0.docvault.svc.cluster.local:27017 [P]: pbm-agent [v2.12.0] OK
- audit-mongodb-rs0-2.audit-mongodb-rs0.docvault.svc.cluster.local:27017 [S]: pbm-agent [v2.12.0] OK

PITR incremental backup:
=====
Status [ON]
Running members: rs0/audit-mongodb-rs0-0.audit-mongodb-rs0.docvault.svc.cluster.local:27017;

Currently running:
=====
(none)
```

Hình 4.16: Trạng thái Percona MongoDB replica set audit với một PRIMARY và hai SECONDARY

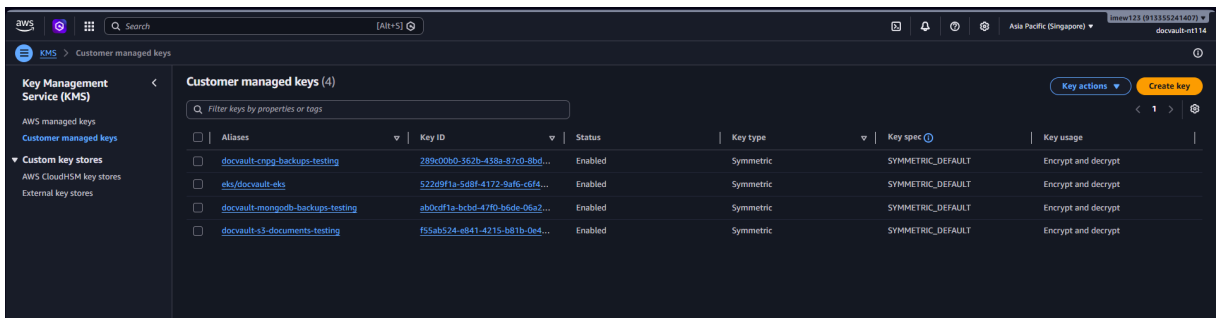
```
PS D:\projects\docvaultNewest\docvault> kubectl exec -it audit-mongodb-rs0-0 -n docvault -c backup-agent -- pbm status

Backups:
=====
S3 ap-southeast-1 s3://docvault-mongodb-backups-testing-913355241407/audit-mongodb
  Snapshots:
    2026-06-23T08:10:41Z 80.11KB <logical> success [restore_to_time: 2026-06-23T08:10:49]
    2026-06-22T19:16:21Z 45.81KB <logical> success [restore_to_time: 2026-06-22T19:16:26]
  PITR chunks [52.10MB]:
    2026-06-23T08:10:50 - 2026-06-25T18:14:27
PS D:\projects\docvaultNewest\docvault> kubectl exec -it audit-mongodb-rs0-0 -n docvault -c backup-agent -- pbm list
Backup snapshots:
  2026-06-22T19:16:21Z <logical> [restore_to_time: 2026-06-22T19:16:26]
  2026-06-23T08:10:41Z <logical> [restore_to_time: 2026-06-23T08:10:49]

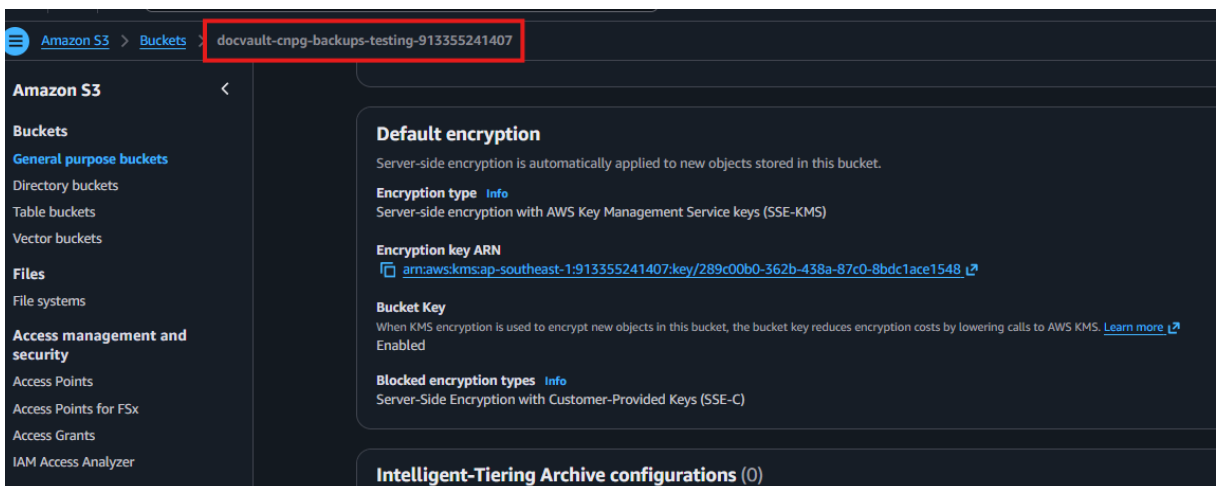
PITR <on>:
  2026-06-23T08:10:50 - 2026-06-27T12:00:49
```

Hình 4.17: Trạng thái backup và PITR của PBM

Cả hai cơ chế hiện đã cấu hình và chạy theo lịch. Việc diễn tập khôi phục để đo RPO và RTO thực tế được bàn ở phần vận hành (Mục 4.5.2), nơi khép lại mạch giám sát, log và khôi phục.



Hình 4.18: Các customer-managed KMS key dùng cho EKS, tài liệu và backup



Hình 4.19: S3 bucket lưu backup cơ sở dữ liệu

4.2.3 Tổ chức workload trên Kubernetes

4.2.3.1 Helm chart và Kustomize overlay

Các service DocVault sử dụng Helm chart dùng chung tại `infra/k8s/charts/docvault-service`. Mỗi workload có một file values riêng cho image, port, biến môi trường, resource request/limit, migration và seed job. Việc dùng một chart chung giúp các service áp dụng đồng nhất những kiểm soát sau:

- Chạy với UID/GID không phải root và `seccompProfile=RuntimeDefault`.
- Tắt privilege escalation, đặt root filesystem chỉ đọc và loại bỏ toàn bộ Linux capability.
- Không tự động mount ServiceAccount token nếu workload không cần truy cập Kubernetes API.
- Cấu hình readiness probe, liveness probe và resource request/limit.
- Mount `/tmp` bằng `emptyDir` cho tiến trình cần vùng ghi tạm.
- Hỗ trợ image reference theo dạng `repository:tag@sha256:digest`.
- Tạo NetworkPolicy cho từng release.

Các kiểm soát như chạy non-root, tắt privilege escalation, dùng seccomp mặc định, loại bỏ Linux capability và áp dụng NetworkPolicy được đối chiếu với Kubernetes Pod Security Standards và Security Checklist [28], [31].

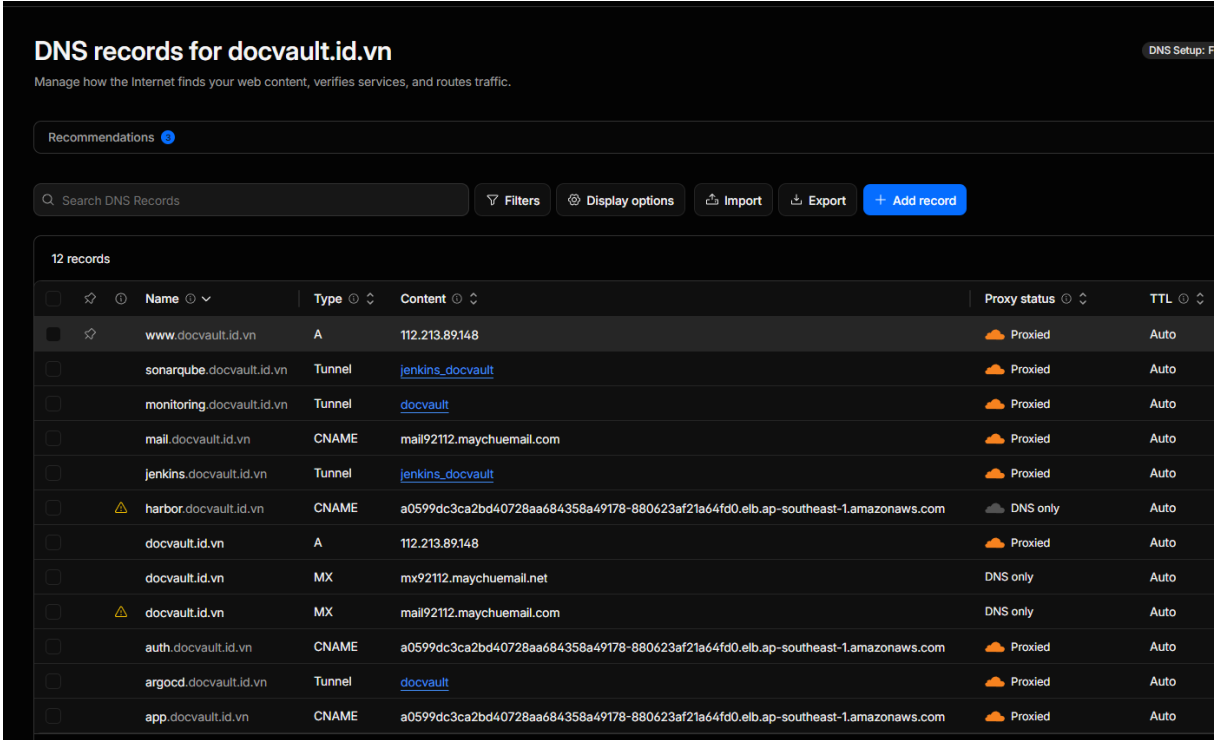
Các dependency nền tảng được quản lý bằng Kustomize tại `infra/k8s/infra-deps`. Overlay hiện dùng cho EKS triển khai namespace, External Secrets, CloudNativePG, MongoDB và Keycloak; các tài nguyên MinIO được loại khỏi desired state sau khi document-service chuyển sang S3. Cách tách Helm cho application và Kustomize cho dependency giúp giảm lặp cấu hình nhưng vẫn giữ được khả năng tùy biến theo môi trường.

4.2.3.2 Ingress, DNS và TLS

Lưu lượng public đi qua ingress-nginx. Các dịch vụ công khai của hệ thống gồm ứng dụng chính (`app.docvault.id.vn`), máy chủ xác thực Keycloak (`auth.docvault.id.vn`), và private registry Harbor (`harbor.docvault.id.vn`) được cấu hình DNS trỏ trực tiếp đến địa chỉ IP Public của AWS Network Load Balancer (NLB) thông qua bản ghi trên Cloudflare (Hình 4.20). Để cấp phát chứng chỉ SSL/TLS bảo mật, cert-manager sử dụng ClusterIssuer `letsencrypt-cloudflare` và giao thức xác thực DNS-01 challenge qua API Token của Cloudflare.

Để có thể cung cấp một cách truy cập công khai bằng URL cho các giao diện như Argo CD (`argocd.docvault.id.vn`), hệ thống giám sát Grafana (`monitoring.docvault.id.vn`) và private registry Harbor (`harbor.docvault.id.vn`), chúng được định tuyến bảo

mật qua giải pháp Cloudflare Tunnel thông qua một dịch vụ `cloudflared` chạy độc lập bằng docker container trong namespace `cloudflare-tunnel` (Hình 4.22). Bằng cách này, các dịch vụ không cần lộ trực tiếp ra Internet mà vẫn có thể truy cập công khai thông qua Cloudflare Tunnel. Tuy nhiên, chỉ riêng Harbor thì cần phải sử dụng IP Public của AWS NLB (như đã nói ở trên) và tắt dịch vụ Proxy của Cloudflare. Lý do cần tắt dịch vụ đó đi là vì Proxy của Cloudflare giới hạn dung lượng upload chỉ 100MB (ở bản free) và các image khi được upload lên Harbor có thể lên đến 200MB hoặc 500MB nếu dành cho phần cache image.



DNS records for docvault.id.vn

Manage how the Internet finds your web content, verifies services, and routes traffic.

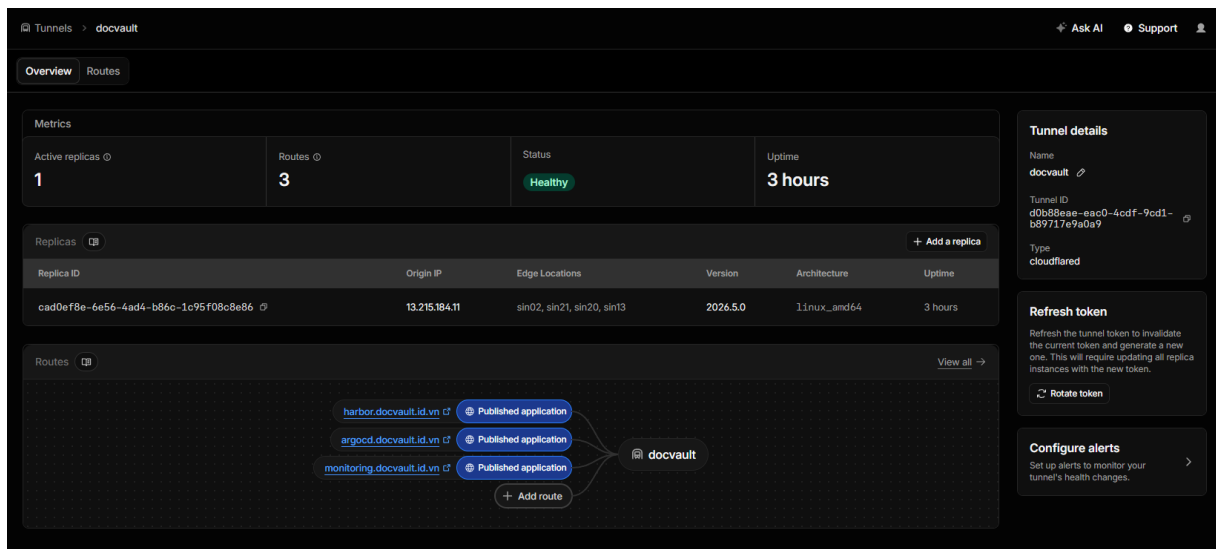
Recommendations ⓘ

Search DNS Records Filters Display options Import Export + Add record

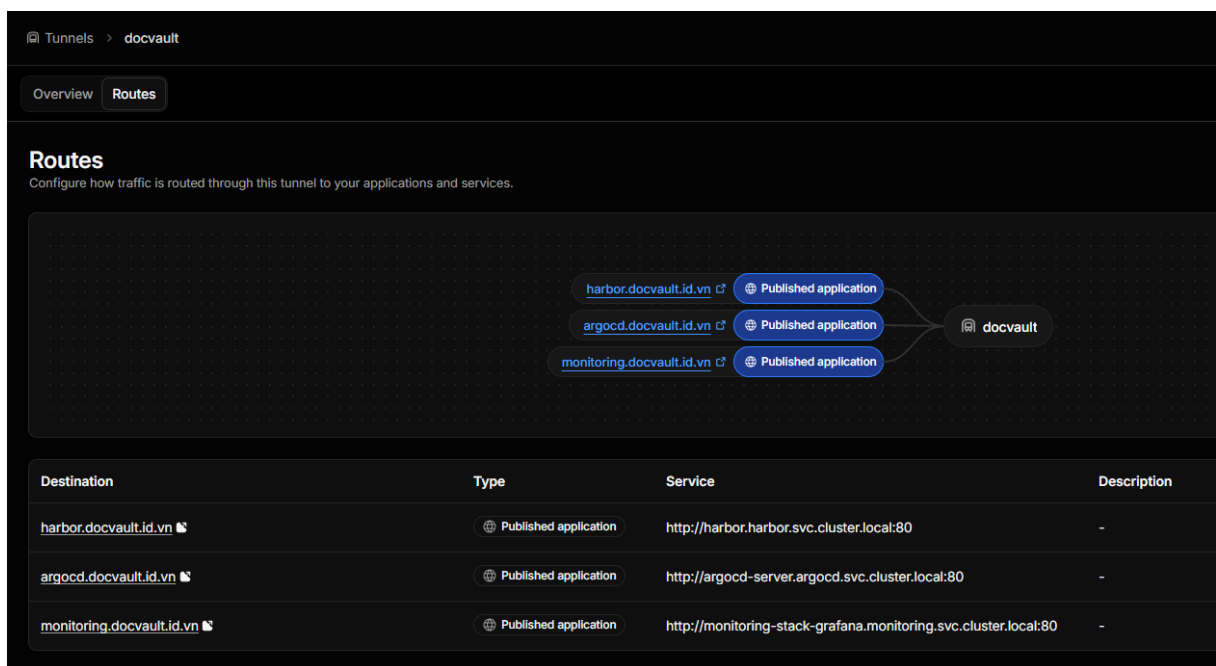
12 records

Name	Type	Content	Proxy status	TTL
www.docvault.id.vn	A	112.213.89.148	Proxied	Auto
sonarqube.docvault.id.vn	Tunnel	jenkins_docvault	Proxied	Auto
monitoring.docvault.id.vn	Tunnel	docvault	Proxied	Auto
mail.docvault.id.vn	CNAME	mail92112.maychuemail.com	Proxied	Auto
jenkins.docvault.id.vn	Tunnel	jenkins_docvault	Proxied	Auto
harbor.docvault.id.vn	CNAME	a0599dc3ca2bd40728aa684358a49178-880623af21a64fd0.elb.ap-southeast-1.amazonaws.com	DNS only	Auto
docvault.id.vn	A	112.213.89.148	Proxied	Auto
docvault.id.vn	MX	mx92112.maychuemail.net	DNS only	Auto
docvault.id.vn	MX	mail92112.maychuemail.com	DNS only	Auto
auth.docvault.id.vn	CNAME	a0599dc3ca2bd40728aa684358a49178-880623af21a64fd0.elb.ap-southeast-1.amazonaws.com	Proxied	Auto
argocd.docvault.id.vn	Tunnel	docvault	Proxied	Auto
app.docvault.id.vn	CNAME	a0599dc3ca2bd40728aa684358a49178-880623af21a64fd0.elb.ap-southeast-1.amazonaws.com	Proxied	Auto

Hình 4.20: Cấu hình bản ghi DNS của DocVault trỏ tên miền dịch vụ về Load Balancer hoặc Tunnel trên Cloudflare

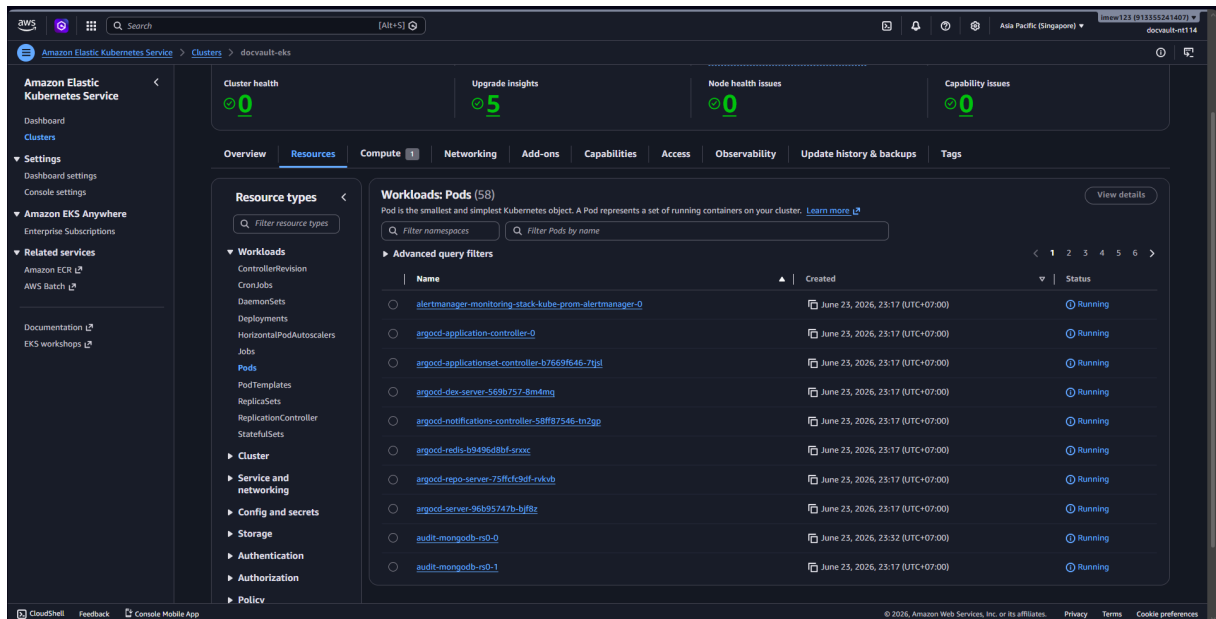


Hình 4.21: Trạng thái hoạt động của Cloudflare Tunnel trên trang quản trị Cloudflare Zero Trust



Hình 4.22: Application route của Cloudflare Tunnel cho Argo CD, Grafana, Harbor trong cụm EKS

NetworkPolicy hiện tạo ranh giới truy cập cơ bản cho workload và ingress công khai. Tuy nhiên, egress của chart ứng dụng vẫn được mở để bảo đảm khả năng giao tiếp giữa các dịch vụ trong môi trường kiểm thử. Do đó, cấu hình hiện tại chưa đạt mô hình zero-trust đầy đủ. Giai đoạn tiếp theo cần giới hạn egress theo dịch vụ đích, DNS, cơ sở dữ liệu, S3 endpoint và identity provider [32].



Hình 4.23: Danh sách workload trên cụm EKS và trạng thái vận hành tại thời điểm thu thập bằng chứng

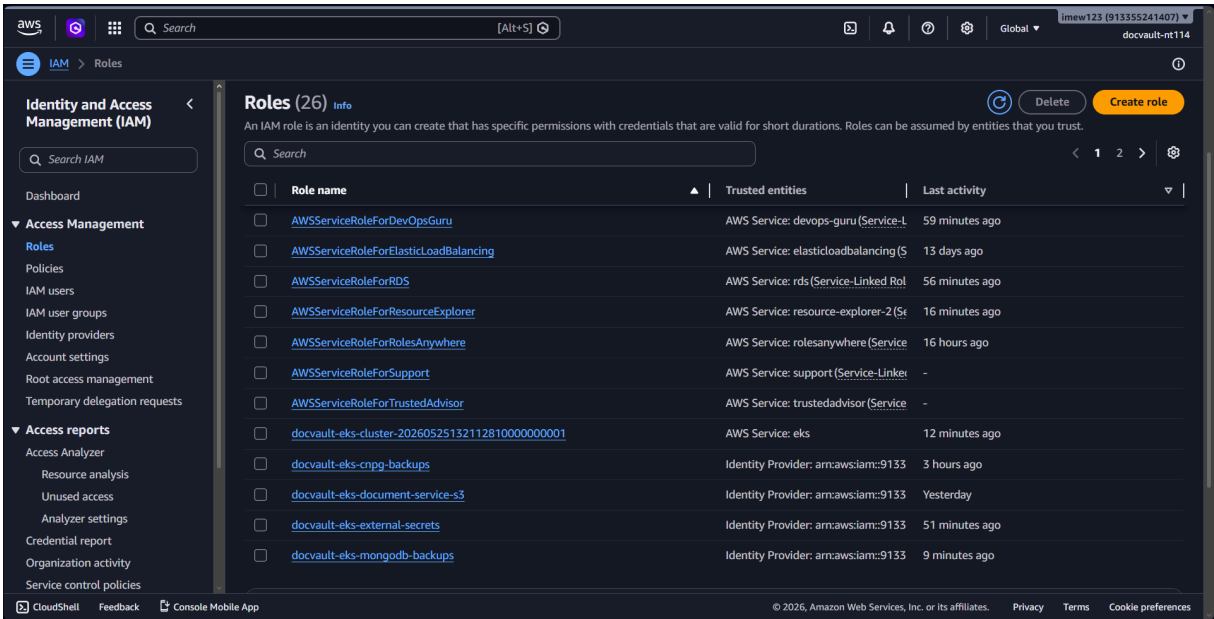
Hình 4.23 ghi nhận 58 pod được quản lý trên cụm và các workload tiêu biểu của Argo CD, monitoring và cơ sở dữ liệu đang ở trạng thái Running.

4.2.4 Quản lý secret và quyền truy cập AWS

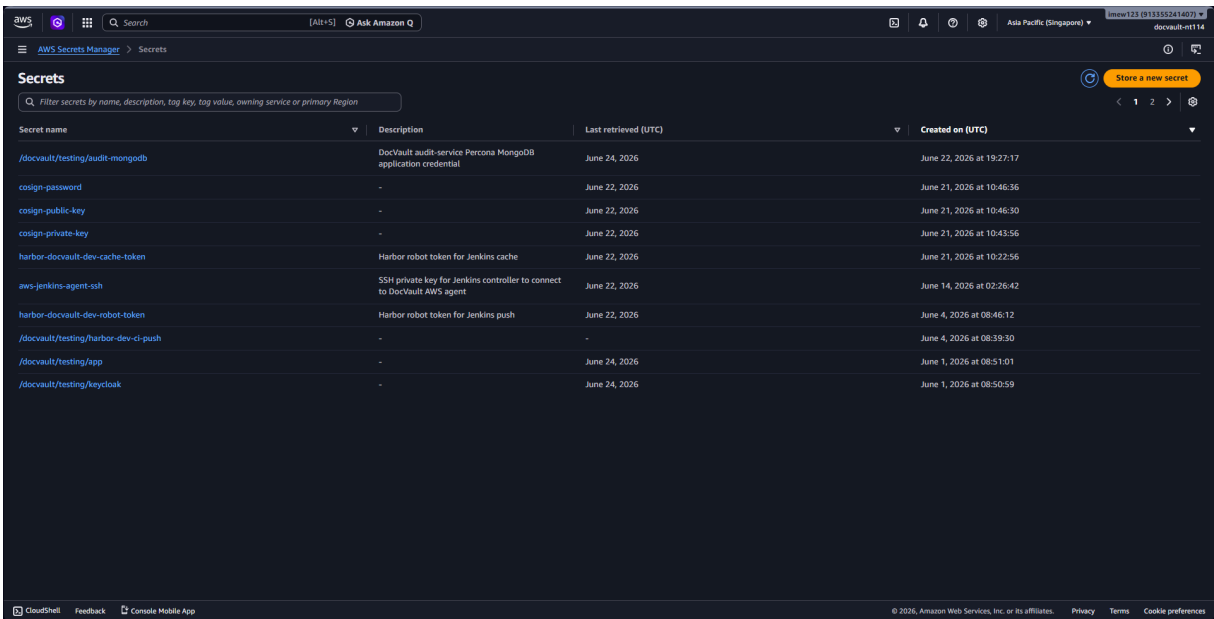
DocVault sử dụng External Secrets Operator làm cầu nối giữa Kubernetes và AWS Secrets Manager. `SecretStore aws-secrets-manager` đọc secret từ khu vực triển khai của cluster là `ap-southeast-1`. ServiceAccount của operator nhận IAM role thông qua IRSA và chỉ có quyền đọc các secret thuộc prefix của môi trường DocVault. Kubernetes Secret tại runtime được tạo từ tài nguyên `ExternalSecret`; vì vậy repository chỉ lưu tham chiếu và cấu trúc ánh xạ, không lưu giá trị bí mật [24], [33].

Pipeline cũng hỗ trợ AWS IAM Roles Anywhere cho Jenkins controller được triển khai local, chạy ngoài AWS. Cơ chế này sử dụng trust anchor, profile và IAM role để Jenkins nhận credential tạm thời bằng certificate thay vì lưu access key dài hạn. Quyền đọc secret của Jenkins được giới hạn theo danh sách tài nguyên cho phép.

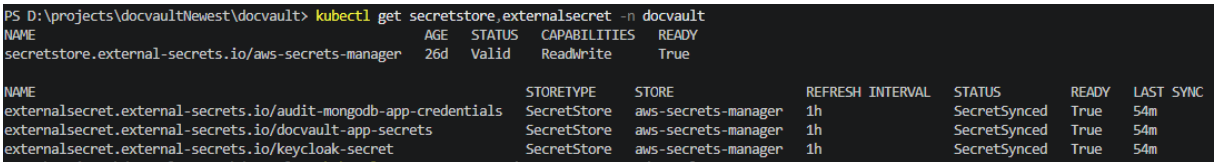
Đối với việc kiểm tra trạng thái triển khai, ServiceAccount `jenkins-argocd-reader` chỉ có các quyền `get`, `list` và `watch` trên tài nguyên Argo CD Application. Jenkins không có quyền sửa hoặc xóa application. Thiết kế này áp dụng nguyên tắc đặc quyền tối thiểu cho bước health check sau triển khai.



Hình 4.24: Các IAM role cho document-service, External Secrets và tiến trình backup qua IRSA



Hình 4.25: Các secret của DocVault được quản lý tập trung trong AWS Secrets Manager



Hình 4.26: Trạng thái đồng bộ Secret của External Secrets Operator (ESO)

Hình 4.25 thể hiện các nhóm secret dành cho ứng dụng, Keycloak, MongoDB, Harbor, Cosign và Jenkins. Ảnh chỉ hiển thị tên và metadata quản lý, không hiển thị giá trị secret.

4.3 Quy trình DevSecOps với GitOps và CI/CD

4.3.1 Mô hình GitOps với Argo CD

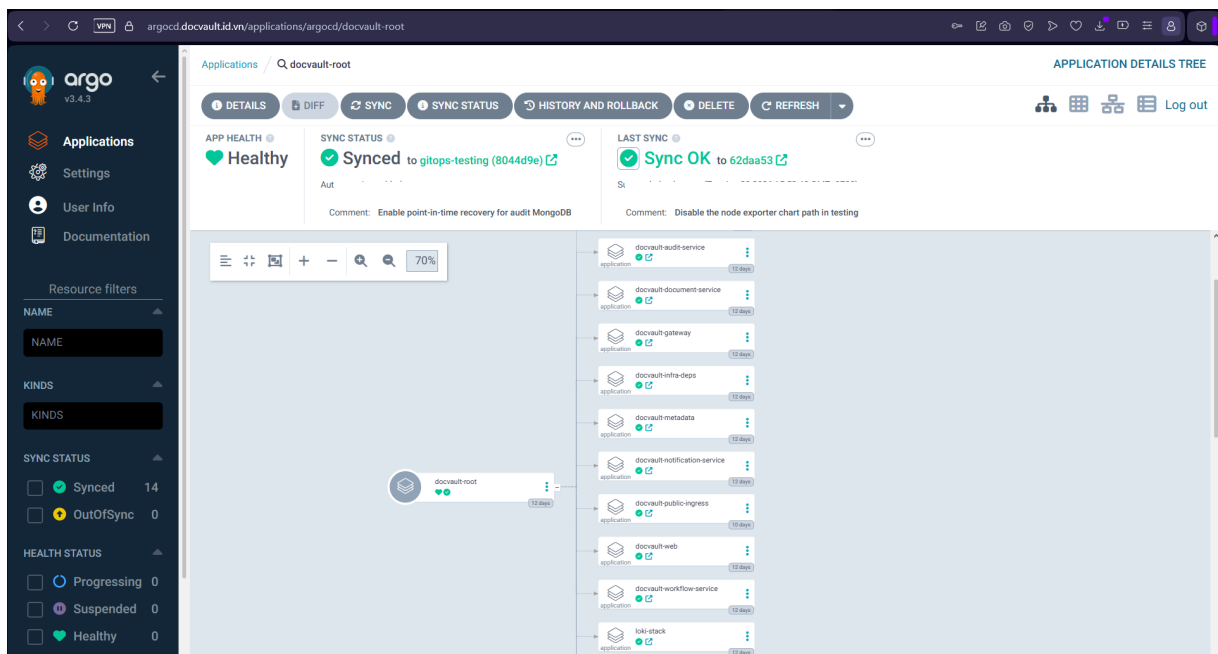
4.3.1.1 App-of-apps và thứ tự đồng bộ

Argo CD triển khai theo mô hình App-of-apps, khởi tạo các application con bằng application gốc là `docvault-root`. Application này theo dõi thư mục `infra/argocd-apps` trên repo GitOps và tạo các application con. Mô hình App-of-apps cho phép quản lý thống nhất hạ tầng phụ trợ, dịch vụ ứng dụng, ingress và observability [18].

Thứ tự triển khai được kiểm soát bằng sync wave:

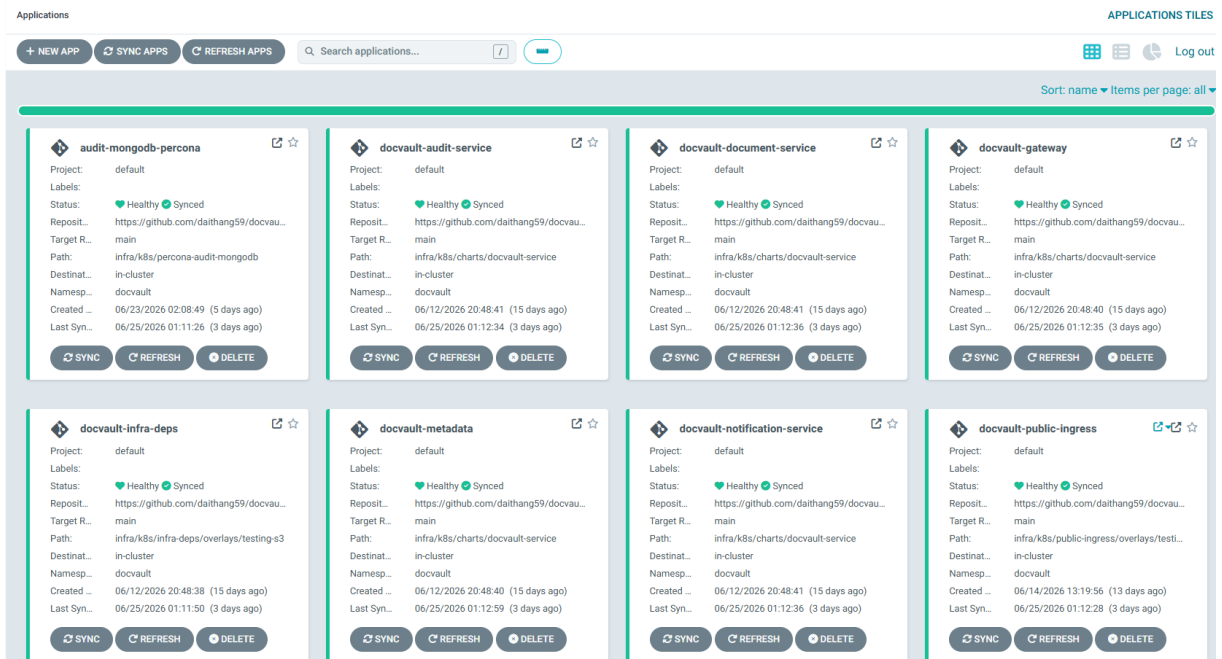
1. Sync wave 0 triển khai dependency hạ tầng qua `docvault-infra-deps` (Kustomize base/overlay).
2. Sync wave 1 triển khai gateway, web và sáu backend service bằng Helm.
3. Sync wave 2 triển khai public ingress, Prometheus/Grafana và Loki/Promtail.

Các application DocVault bật `selfHeal=true` để Argo CD tự động xử lý sai lệch trạng thái giữa cụm EKS và Git. `prune=false` được duy trì đối với application nghiệp vụ nhằm giảm nguy cơ xóa ngoài ý muốn trong môi trường kiểm thử. Riêng monitoring và logging bật prune để quản lý toàn bộ vòng đời chart. Khi triển khai production, chính sách prune cần đi kèm quy trình rà soát, backup và cơ chế bảo vệ tài nguyên quan trọng.

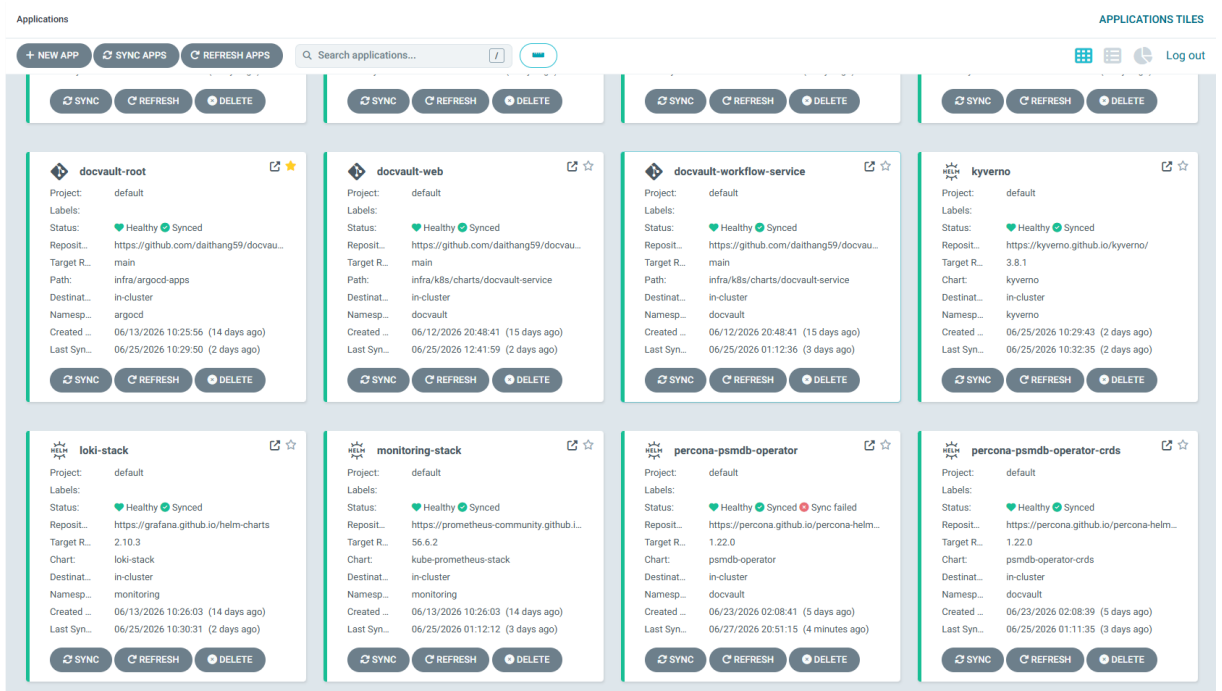


Hình 4.27: Application gốc `docvault-root` đạt trạng thái Healthy và Synced

Hình 4.27 xác nhận application gốc đang theo dõi nhánh GitOps (main), đã đồng bộ revision Git và quản lý các application con của DocVault.



Hình 4.28: Hình chụp dashboard ArgoCD hiển thị các ứng dụng (Ảnh 1)



Hình 4.29: Hình chụp dashboard ArgoCD hiển thị các ứng dụng (Ảnh 2)

4.3.1.2 Cập nhật trạng thái mong muốn (desired state) bằng GitOps

Sau khi image được push, Jenkins lấy digest từ registry và cập nhật ba trường `repository`, `tag`, `digest` trong Helm values tương ứng. Tag mặc định là 12 ký tự đầu của Git commit SHA. Pipeline clone riêng nhánh GitOps, commit thay đổi với `[skip ci]` để tránh vòng lặp và retry push tối đa ba lần nếu có cập nhật đồng thời.

Nếu thay đổi chỉ liên quan đến manifest Kubernetes, pipeline đồng bộ thư mục `infra/k8s` sang nhánh GitOps nhưng giữ lại image reference đang chạy, nên thay đổi cấu hình hạ tầng không vô tình hạ phiên bản image. Argo CD sau đó phát hiện commit mới và reconcile cluster về desired state.

4.3.2 Harbor và bảo mật chuỗi cung ứng phần mềm

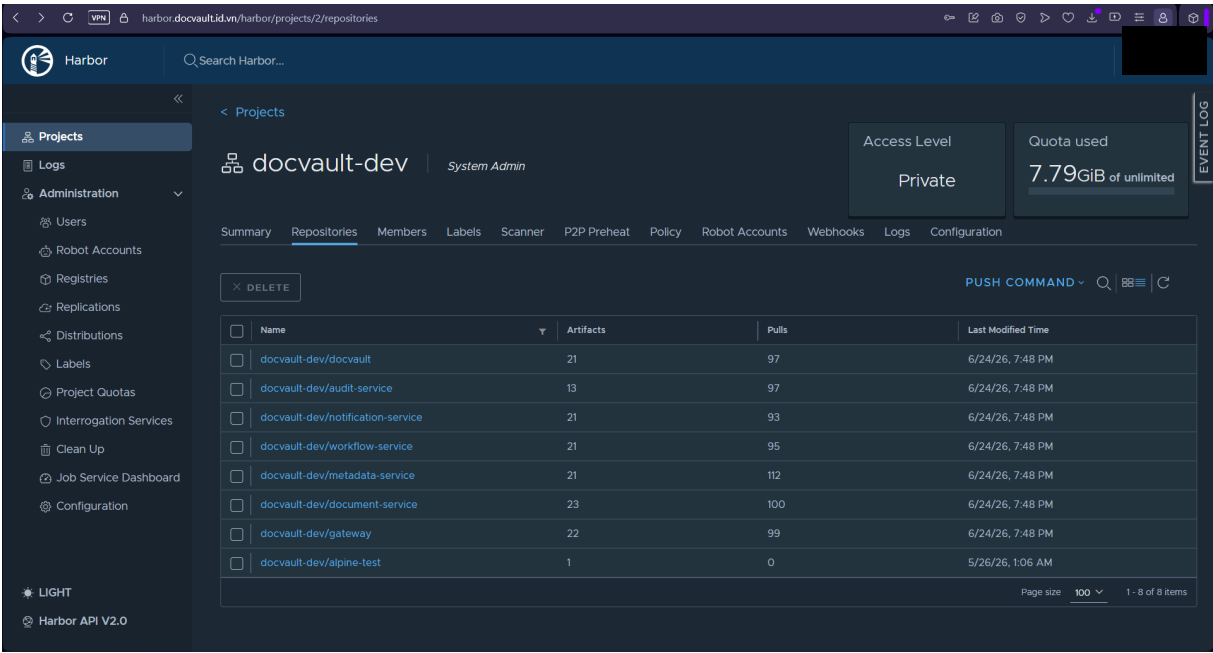
Harbor được triển khai trên EKS với vai trò private container registry của DocVault. Registry sử dụng PersistentVolume gp3, TLS, robot account dành cho Jenkins và image pull secret dành cho workload. Trivy scanner tích hợp hỗ trợ quản lý kết quả quét lỗ hổng của artifact lưu trong registry [20].

Pipeline mặc định không đẩy tag `latest`; mỗi image được định danh bằng Git SHA và digest. BuildKit registry cache sử dụng tag riêng `buildcache` để rút ngắn thời gian build nhưng tag này không được sử dụng khi triển khai. Sau khi push, Jenkins ký image digest bằng Cosign khi tham số `SIGN_IMAGES` được bật và có thể xác minh lại chữ ký bằng public key đã cấu hình [34]. Bằng chứng Harbor thu thập trong đề tài cho thấy nhiều artifact đã có trạng thái được ký bằng Cosign và có liên kết `SBOM details` của Harbor.

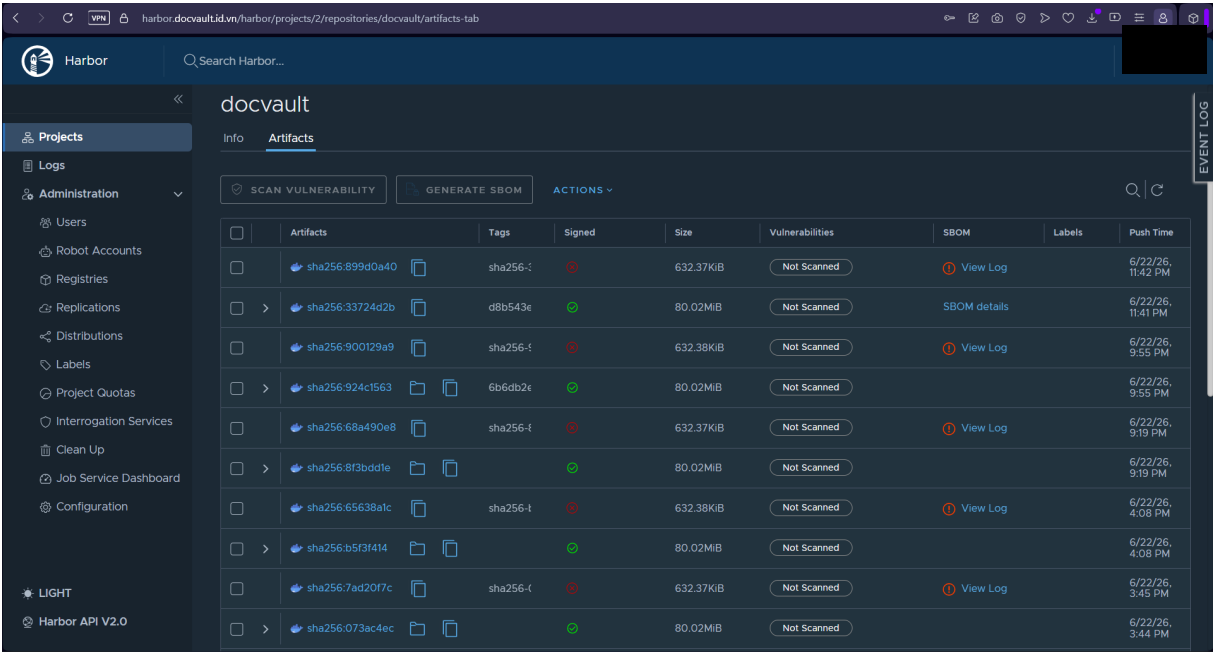
Policy-as-code yêu cầu workload DocVault:

- chỉ dùng image từ project DocVault trên Harbor;
- pin image bằng SHA-256 digest;
- khai báo Harbor image pull secret;
- không chạy privileged hoặc root;
- dùng read-only root filesystem, tắt privilege escalation và drop capability;
- có CPU/memory request và limit;
- không dùng tag `latest`;
- không commit Kubernetes Secret có `data` hoặc `stringData`.

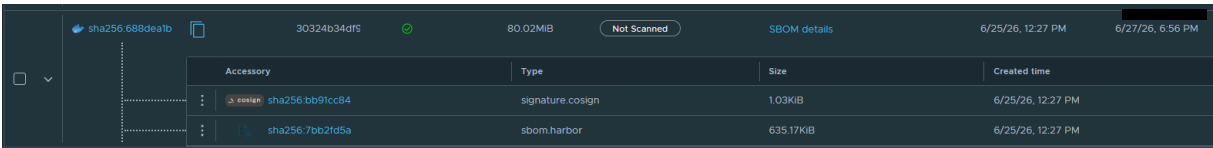
Các policy Kyverno được chạy bằng Kyverno CLI trong Jenkins trên manifest đã render. Báo cáo kết quả hiện có ghi nhận 85 lượt kiểm tra đạt, không có trường hợp không đạt, cảnh báo hoặc lỗi. Đây là cổng kiểm soát trước triển khai [35]. Việc cài Kyverno admission controller trong cụm và xác minh chữ ký Cosign tại thời điểm tiếp nhận tài nguyên là bước gia cố tiếp theo để kiểm soát cả các thay đổi được tạo ngoài pipeline.



Hình 4.30: Project riêng docvault-dev và các repository dịch vụ trên Harbor



Hình 4.31: Artifact Harbor được định danh bằng digest, có trạng thái ký và thông tin SBOM



Hình 4.32: Chữ ký Cosign và SBOM của artifact trên Harbor

Hình 4.30 xác nhận các image của web, gateway và microservice được lưu trong project riêng. Hình 4.31 và Hình 4.32 cung cấp bằng chứng về digest, chữ ký và SBOM của artifact.

Một số artifact trong ảnh vẫn hiển thị trạng thái **Not Scanned**; lý do là khi pipeline thực hiện Cosign signing, pipeline tạo signature cho image và SBOM attestation, với SBOM được tạo ra từ Trivy SBOM generator. Attestation này bị quét không thành công bởi Trivy scanner của Harbor, dẫn đến tình trạng nó là một artifact nằm riêng. Tuy vậy, nó vẫn được lưu với dạng `.att`, minh chứng cho việc ký SBOM của pipeline.

4.3.3 Jenkins pipeline

4.3.3.1 Phân tách CI và CD

Pipeline chính nằm trong `Jenkinsfile`, còn logic tái sử dụng được tổ chức thành Jenkins Shared Library trong thư mục `vars`. Pipeline phân biệt hai chế độ:

- **CI validation**: chạy trên branch phát triển (như `feat`, `chore`, `fix`, ...) hoặc pull request, thực hiện secret scan, lint, unit test, workspace build và các security gate.
- **CD release**: chỉ branch tin cậy được khai báo bởi `RELEASE_BRANCH` mới được push image và cập nhật GitOps. Hiện tại `RELEASE_BRANCH` là nhánh `main`, nó chạy khi thực hiện merge pull request từ nhánh phát triển.

Cách phân tách này ngăn việc branch phát triển tạo ra các artifact. Trước khi chạy các stage, pipeline xác định file thay đổi để chỉ chạy phần liên quan. Thay đổi ở service chỉ build service bị ảnh hưởng; thay đổi ở shared library, lockfile hoặc Dockerfile có thể kích hoạt nhiều image. Khi không xác định được diff range tin cậy, pipeline build toàn bộ để ưu tiên tính đúng đắn.

4.3.3.2 Các stage chính

Luồng pipeline hiện tại gồm:

1. **Checkout & Initialize Config**: checkout source, hợp nhất parameter và xác định chế độ CI/CD.
2. **Prevent Loop**: bỏ qua commit GitOps có `[skip ci]`.
3. **Detect Changes**: phân loại thay đổi ứng dụng, bảo mật, IaC và image build.
4. **System Check**: kiểm tra agent và công cụ cần thiết.
5. **Install**: cài dependency bằng `pnpm` và sinh Prisma client khi cần.
6. **Secret Scan**: dùng TruffleHog phát hiện credential bị commit.
7. **Pre-build Quality**: chạy unit test và lint song song, sau đó build toàn workspace.
8. **Pre-build Security Gates**: chạy SCA, Trivy filesystem, SonarQube và kiểm tra IaC song song.
9. **Build & Scan Services**: build image theo batch, dùng BuildKit cache và quét image bằng Trivy.

- The screenshot displays the Jenkins web interface for a pipeline named 'fix/dependency-check'. The top navigation bar shows the Jenkins logo, the pipeline name, and various tabs like 'Runs', 'Builds', 'Configuration', and 'Logs'. The main area features a 'Pipeline View' showing a sequence of steps: Checkout SCM, Checkout & Initialize, Prevent Loop, Detect Changes, System Check, Install, Secret Scan, Pre-build Quality, Lint & Unit Tests, Workspace Build, Pre-build Security..., Build & Scan Services, Publish, Post-build Security..., Push to GitOps, Argo CD Health Check, Post-deploy Smoke..., and Post-deploy ZAP. The pipeline is in a 'Completed' state, indicated by a green checkmark and the text 'Build has been executing for 1 min 16 sec on built-in'. Below the pipeline view, a 'Post Actions' section lists several tasks, including 'Archiving Security Reports...' and 'Archive the artifacts', all of which are completed. The bottom left shows a 'Build History' table with columns for build number, duration, and status, listing recent builds for the same pipeline.

72

4.4 Kiểm soát bảo mật trong pipeline

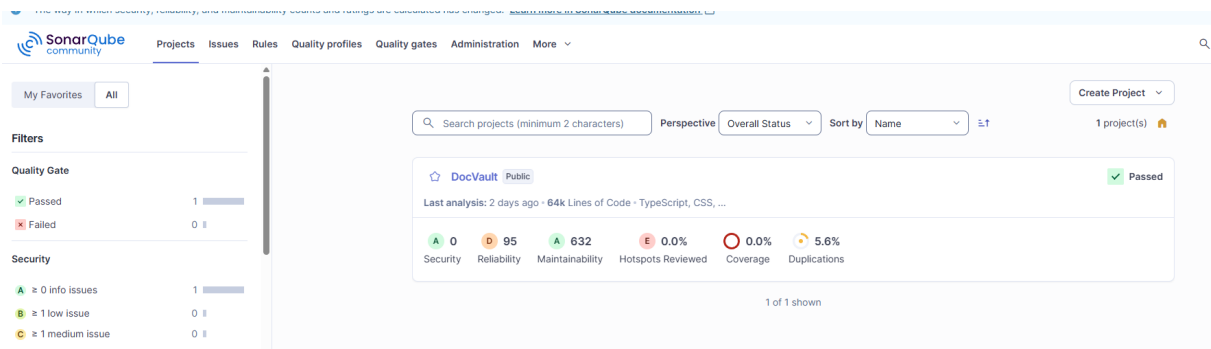
4.4.1 Các cổng kiểm soát chất lượng và bảo mật

4.4.1.1 Secret scanning

TruffleHog quét filesystem repository trước các bước build. Các thư mục dependency, cache, Terraform working directory và file state/plan được loại trừ để giảm nhiễu. Nếu phát hiện secret, stage fail và report được archive. Cơ chế này giảm nguy cơ credential bị đóng gói vào image hoặc lưu trong lịch sử phát hành.

4.4.1.2 Kiểm thử và chất lượng mã nguồn

Unit test và lint chạy song song để giảm thời gian phản hồi; workspace build chạy sau đó để kiểm tra khả năng biên dịch của toàn monorepo. SonarQube phân tích source trong apps, services và libs. Tham số `ENFORCE_SONAR_QG` mặc định bật, vì vậy pipeline fail nếu Quality Gate không đạt hoặc hết thời gian chờ [21].



Hình 4.34: SonarQube Quality Gate của DocVault

4.4.1.3 SCA và filesystem scanning

OWASP Dependency-Check phân tích dependency và tạo report HTML/JSON/log. Chính sách kỹ thuật đặt ngưỡng fail ở CVSS từ 7 trở lên.

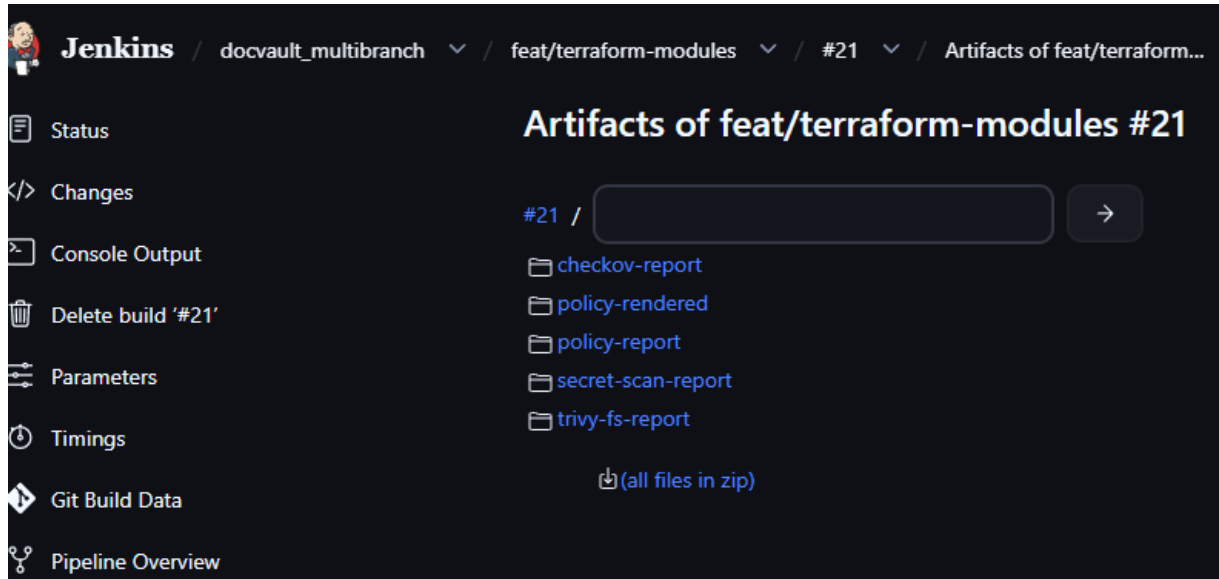
Trivy filesystem quét vulnerability, secret và misconfiguration trên snapshot sạch tạo từ Git. Các finding HIGH/CRITICAL làm fail gate. Phạm vi misconfiguration tập trung vào Dockerfile, Kubernetes và Helm; Terraform được kiểm tra riêng bằng Checkov và Terraform Validate để tránh finding trùng từ module tải về [22].

4.4.1.4 IaC và policy-as-code

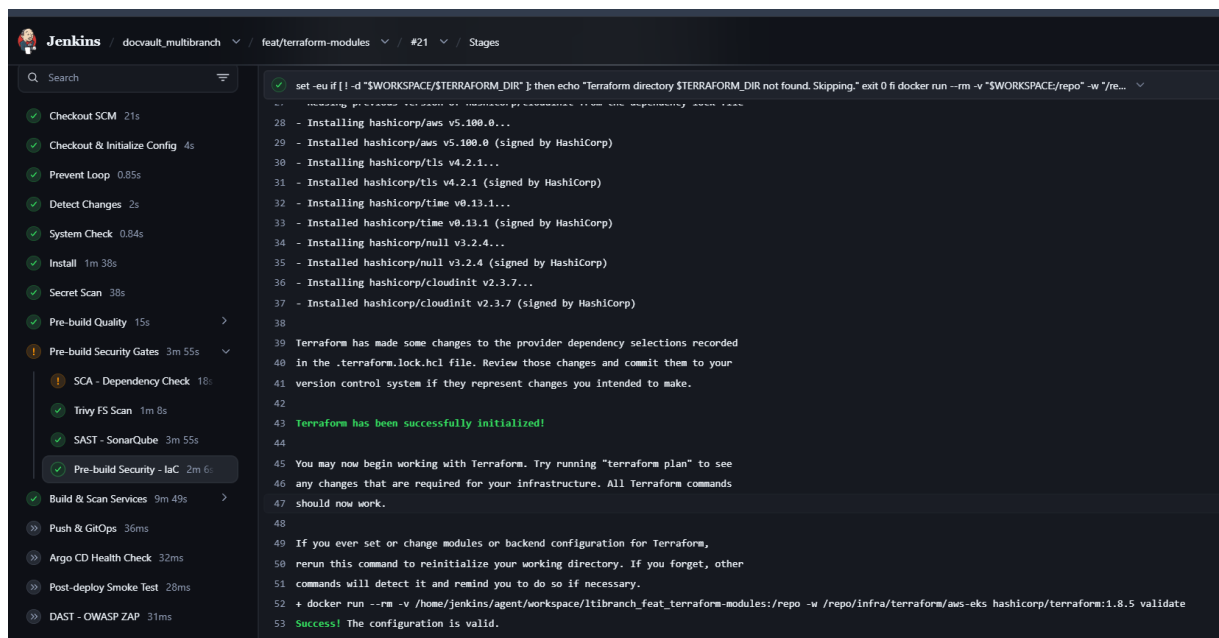
Checkov quét Terraform, Kubernetes và Helm để phát hiện cấu hình rủi ro [36]. Terraform Validate chạy `fmt -check, init -backend=false và validate`. Kyverno CLI render toàn

bộ Helm values thực tế, sau đó áp dụng policy lên manifest Kubernetes trước khi image được phát hành [35].

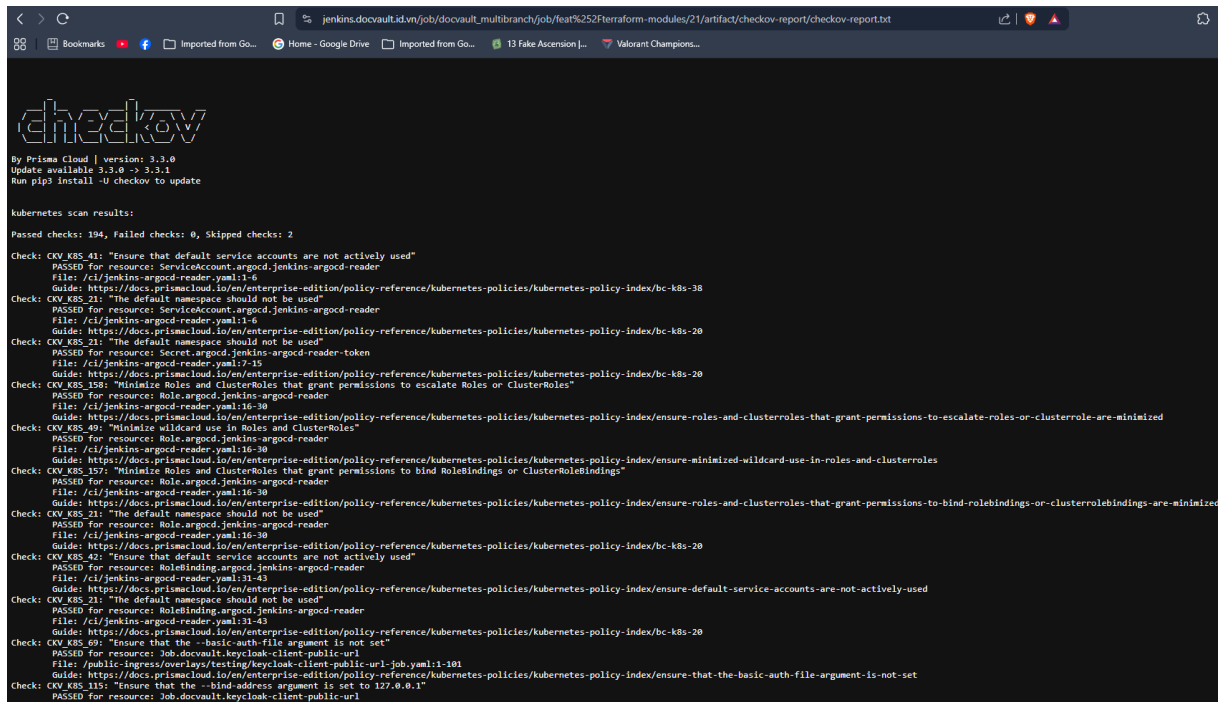
Ba lớp kiểm tra bổ sung cho nhau: Terraform Validate kiểm tra tính hợp lệ cú pháp và provider schema; Checkov kiểm tra best practice của hạ tầng; Kyverno kiểm tra policy runtime trên manifest cuối cùng. Artifact gồm Checkov report, Kyverno report và manifest đã render.



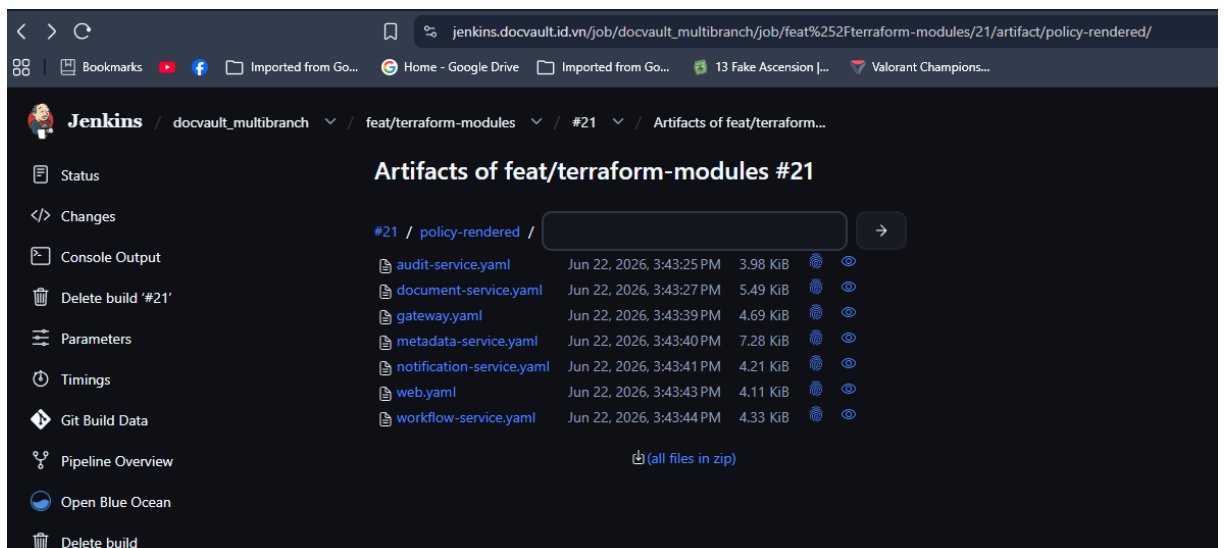
Hình 4.35: Log và artifact cho ba bước IaC



Hình 4.36: Kết quả Terraform Validate



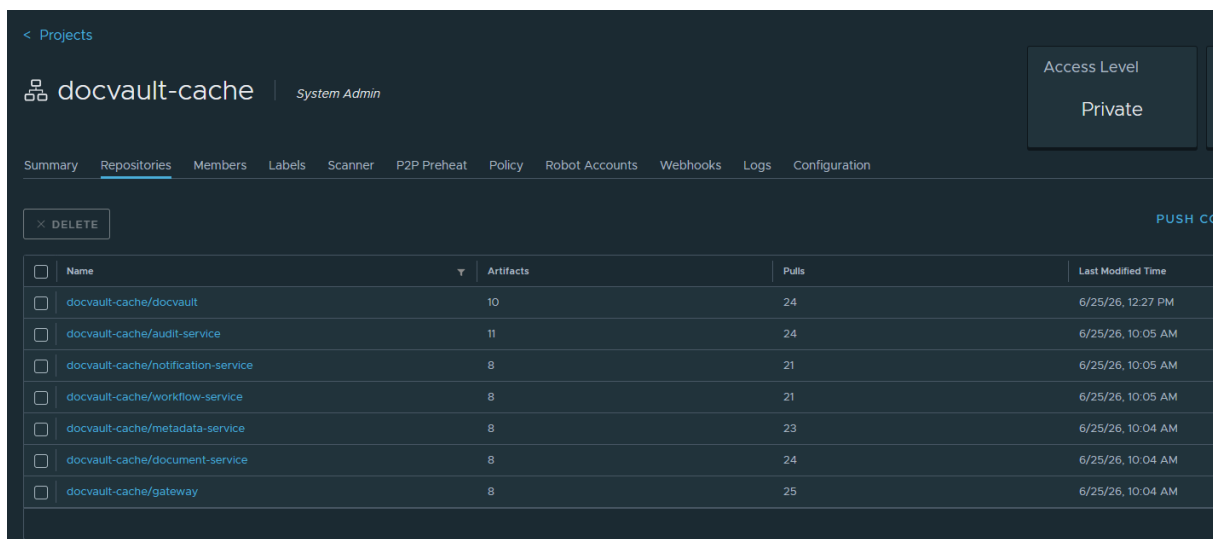
Hình 4.37: Kết quả Checkov



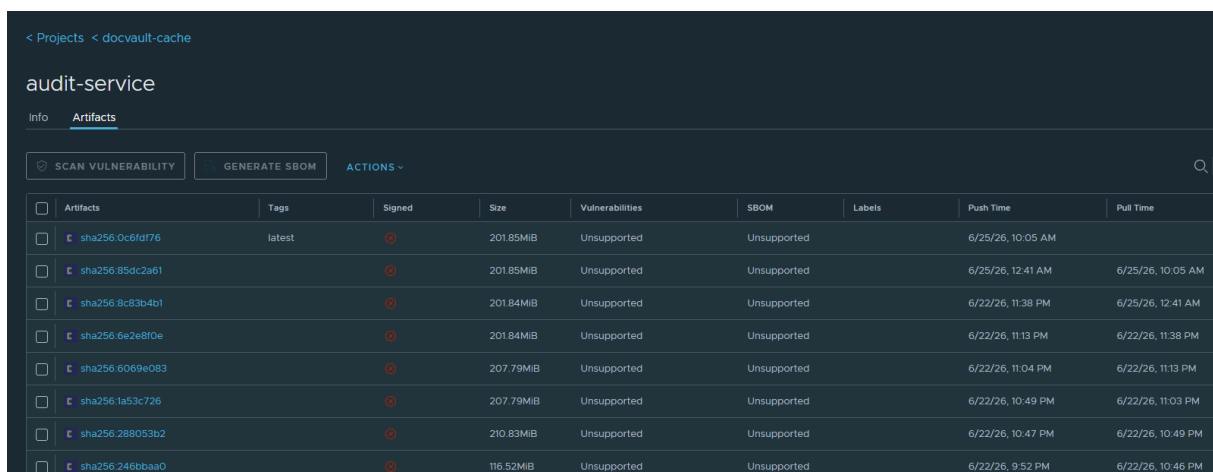
Hình 4.38: Kết quả Kyverno

4.4.2 Build, quét và phát hành container image

Backend sử dụng Dockerfile.backend với build argument xác định service; frontend sử dụng Dockerfile riêng trong apps/web. Các image được build theo batch để tận dụng song song nhưng vẫn giới hạn tải trên Jenkins agent. BuildKit lấy và cập nhật cache trong Harbor, giúp giảm thời gian build lại dependency không đổi.



Hình 4.39: Repo Harbor có tên docvault-cache, dùng để lưu cache của các image



Hình 4.40: Các image cache được đánh dấu bằng chữ C ở cột artifact, có dung lượng lớn do chứa cả các lớp build trung gian.

Sau build, Trivy quét từng image và fail nếu có lỗi hỏng CRITICAL. Chỉ các image vượt qua gate mới được push. Pipeline đăng nhập registry bằng credential tạm trong thư mục Docker config riêng và xóa sau khi hoàn tất. Image triển khai không phụ thuộc tag mutable mà được cập nhật cả tag Git SHA lẫn registry digest.

So với triển khai chỉ dựa trên tag, digest pin bảo đảm Kubernetes kéo đúng artifact đã được quét. Nếu cùng một tag bị ghi đè, digest trong Helm values vẫn không thay đổi. Đây là kiểm soát quan trọng đối với tính toàn vẹn của chuỗi cung ứng phần mềm và là bước nền trước khi chuẩn hóa provenance/attestation theo SLISA [17].

4.4.3 Kiểm tra sau triển khai

4.4.3.1 Argo CD health gate

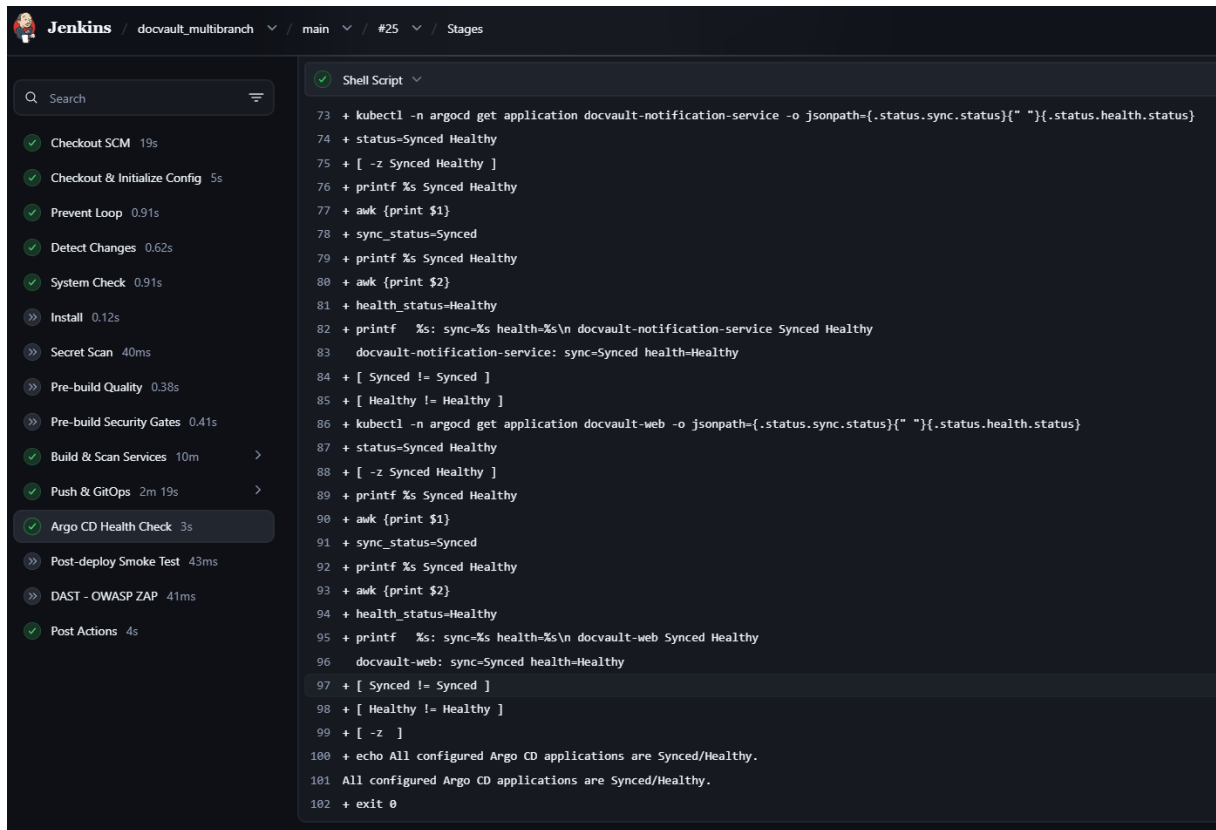
Khi `RUN_ARGO_HEALTH_CHECK=true`, Jenkins dùng kubeconfig credential hoặc context hiện tại để đọc các Argo CD Application. Pipeline chờ đến khi gateway, metadata, document, workflow, audit, notification và web đồng thời đạt **Synced** và **Healthy**. Nếu quá timeout, release fail. Vai trò của Argo CD ở đây vượt ra ngoài việc thực hiện CD: nó còn cung cấp trạng thái reconcile có thể kiểm chứng [18].

4.4.3.2 Smoke test

Smoke test gọi web root và endpoint `/api/health`. Web root chấp nhận HTTP 2xx hoặc 3xx, còn health endpoint yêu cầu 2xx. Mỗi request được retry để hấp thụ độ trễ rollout ngắn. Bước này không thay thế E2E nghiệp vụ, nhưng phát hiện nhanh lỗi ingress, service routing, container startup hoặc cấu hình runtime.

4.4.3.3 DAST bằng OWASP ZAP

OWASP ZAP baseline scan chạy khi `RUN_ZAP=true` và target có thể truy cập từ Jenkins. Pipeline tạo report HTML và JSON, archive làm bằng chứng, đồng thời fail khi report chứa finding High/Critical. Cảnh báo mức thấp hơn được giữ để review thay vì bỏ qua [23].



Hình 4.41: Kết quả Argo CD health check

Jenkins / docvault_multibranch / main / #44 / Stages

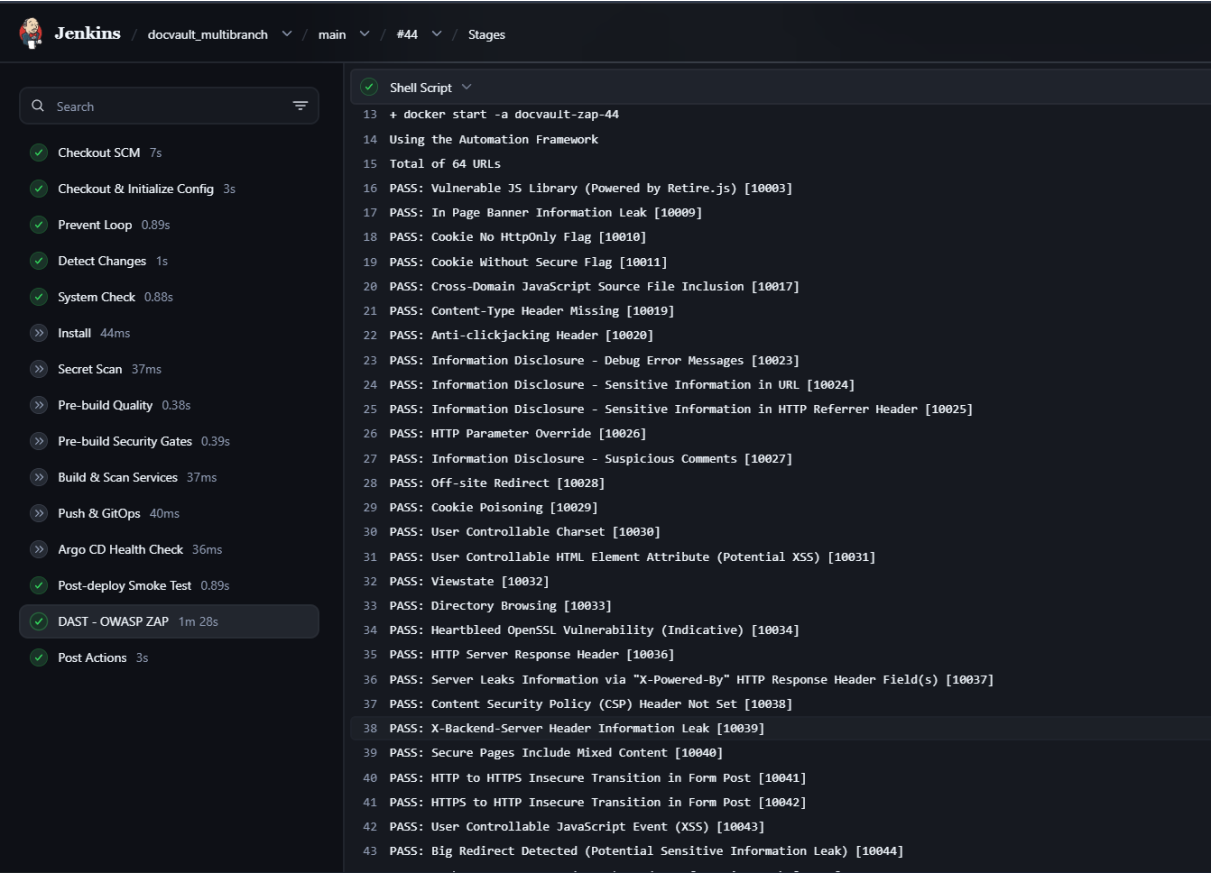
Search

- ✓ Checkout SCM 7s
- ✓ Checkout & Initialize Config 3s
- ✓ Prevent Loop 0.89s
- ✓ Detect Changes 1s
- ✓ System Check 0.88s
- » Install 44ms
- » Secret Scan 37ms
- » Pre-build Quality 0.38s
- » Pre-build Security Gates 0.39s
- » Build & Scan Services 37ms
- » Push & GitOps 40ms
- » Argo CD Health Check 36ms
- ✓ Post-deploy Smoke Test 0.89s
- ✓ DAST - OWASP ZAP 1m 28s
- ✓ Post Actions 3s

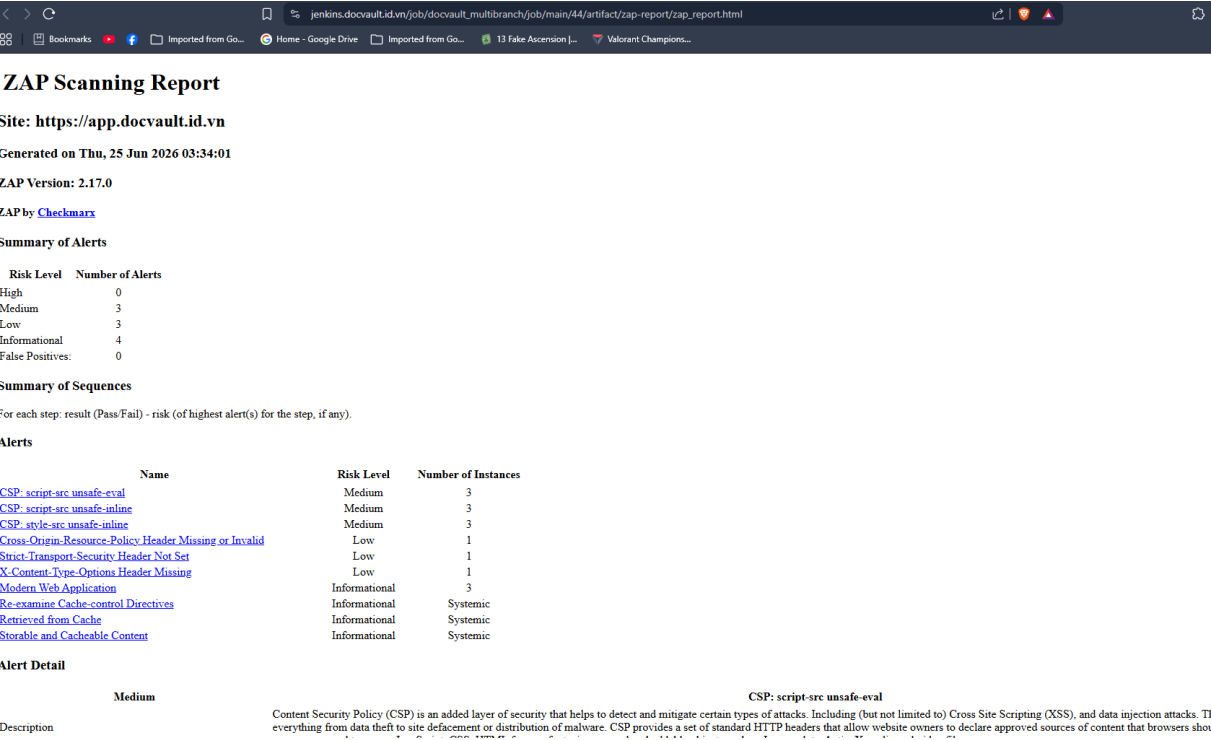
Shell Script

```
0 + set -eu
1 + check_url web-root https://app.docvault.id.vn root
2 + name=web-root
3 + url=https://app.docvault.id.vn
4 + expected=root
5 + attempt=1
6 + [ 1 -le 30 ]
7 + curl -ss -o /dev/null -w %{http_code} --max-time 20 https://app.docvault.id.vn
8 + status=200
9 + echo web-root attempt 1/30: https://app.docvault.id.vn -> HTTP 200
10 web-root attempt 1/30: https://app.docvault.id.vn -> HTTP 200
11 + [ root = root ]
12 + return 0
13 + check_url api-health https://app.docvault.id.vn/api/health health
14 + name=api-health
15 + url=https://app.docvault.id.vn/api/health
16 + expected=health
17 + attempt=1
18 + [ 1 -le 30 ]
19 + curl -ss -o /dev/null -w %{http_code} --max-time 20 https://app.docvault.id.vn/api/health
20 + status=200
21 + echo api-health attempt 1/30: https://app.docvault.id.vn/api/health -> HTTP 200
22 api-health attempt 1/30: https://app.docvault.id.vn/api/health -> HTTP 200
23 + [ health = root ]
24 + [ health = health ]
25 + return 0
26 + echo Post-deploy smoke test passed.
27 Post-deploy smoke test passed.
```

Hình 4.42: Log post deploy smoke test



Hình 4.43: Log OWASP ZAP



Hình 4.44: Kết quả OWASP ZAP

4.5 Vận hành, giám sát và khôi phục

Phần này gom các năng lực vận hành của hệ thống sau khi triển khai: giám sát số liệu, tập trung log và khả năng khôi phục dữ liệu. Hai năng lực đầu được trình bày ngay dưới đây; năng lực khôi phục dựa trên cơ chế sao lưu cơ sở dữ liệu đã mô tả ở Mục 4.2.2.3.

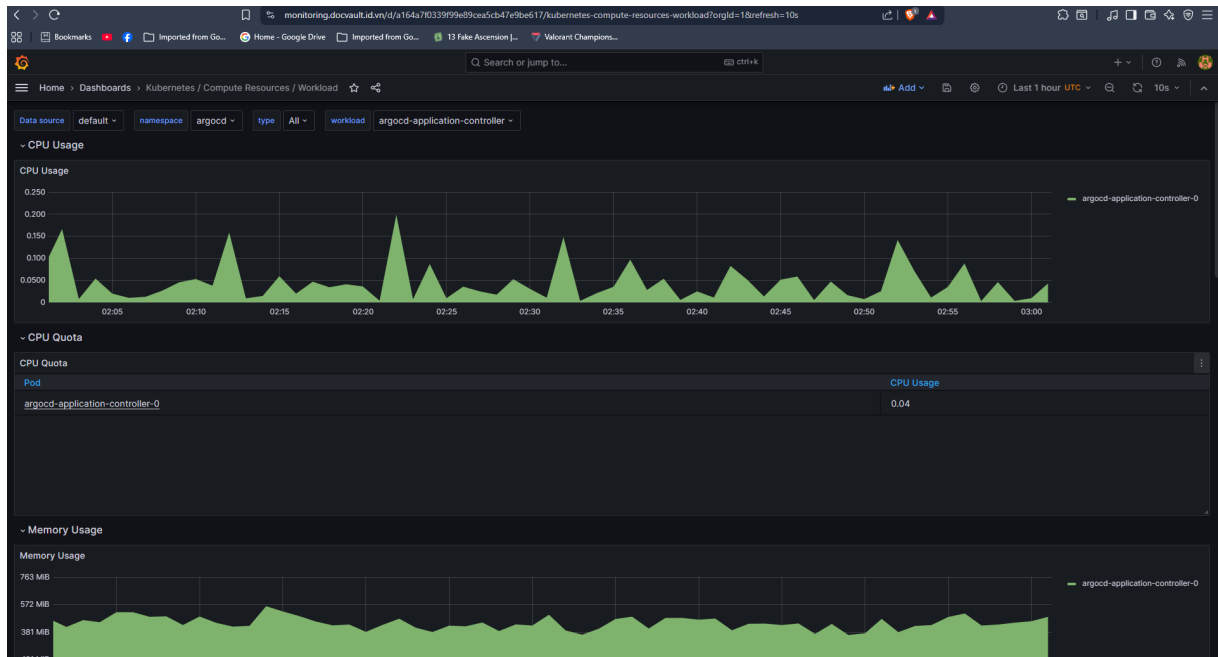
4.5.1 Giám sát và tập trung log

Khả năng quan sát được triển khai qua hai Argo CD Application trong namespace `monitoring`. `kube-prometheus-stack` cung cấp Prometheus, Grafana và Alertmanager; `loki-stack` cung cấp Loki và Promtail. Promtail thu thập log container trong cụm, còn Grafana sử dụng cả Prometheus và Loki làm nguồn dữ liệu [37], [38].

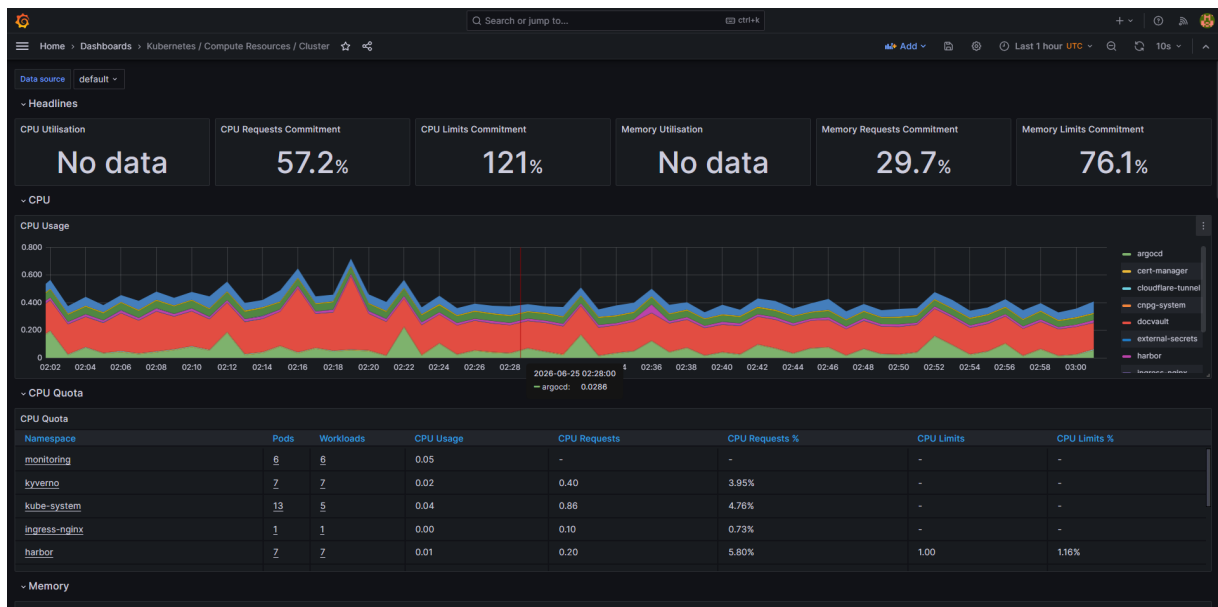
Các nhóm thông tin cần theo dõi gồm:

- Trạng thái pod, deployment, node và số lần restart.
- CPU, memory, saturation và resource request/limit.
- Lỗi HTTP, latency và health endpoint của gateway/service.
- Log theo namespace, pod, container, request ID và thời gian.
- Cảnh báo về pod crash, thiếu tài nguyên, volume hoặc endpoint không sẵn sàng.

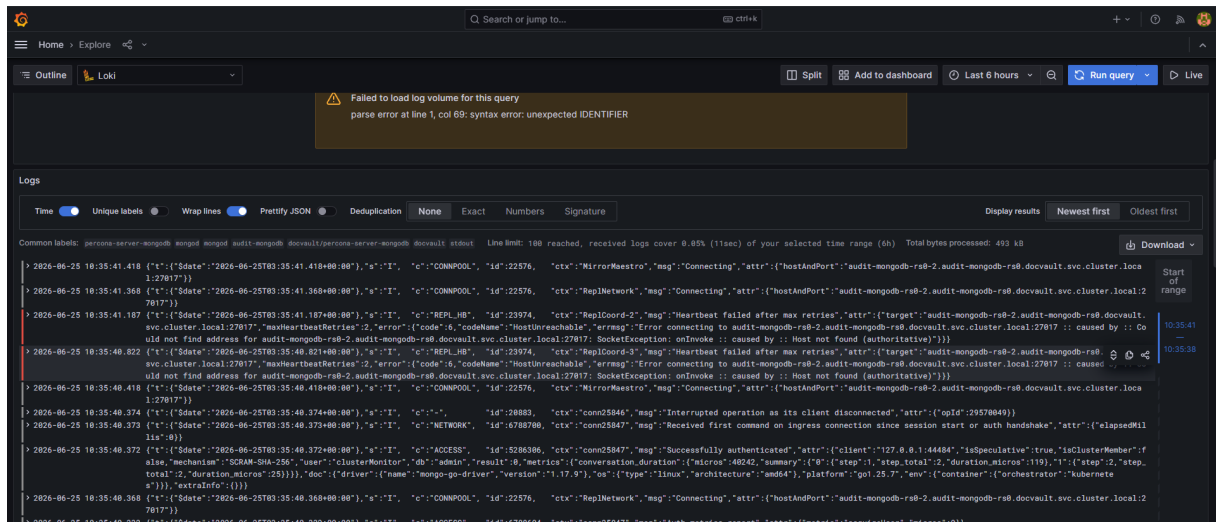
Cấu hình hiện tại đáp ứng nhu cầu trình diễn và thu thập bằng chứng. Tuy nhiên, Loki chưa bật persistence và thông tin quản trị Grafana cần được quản lý hoàn toàn qua External Secrets trước khi vận hành dài hạn. Hệ thống cũng cần bổ sung dashboard nghiệp vụ, alert rule, chính sách retention và kênh tiếp nhận cảnh báo để đáp ứng yêu cầu production.



Hình 4.45: Grafana dashboard giám sát workload DocVault trong namespace docvault



Hình 4.46: Grafana dashboard giám sát EKS cluster



Hình 4.47: Loki tập trung log của namespace DocVault

4.5.2 Khôi phục dữ liệu và sẵn sàng vận hành

Giám sát và log cho biết hệ thống đang chạy ra sao; khôi phục quyết định chuyện gì xảy ra khi nó hỏng. Hai mảnh ghép này cùng tạo nên năng lực vận hành. Cơ chế sao lưu ở Mục 4.2.2.3 đã đặt nền cho khôi phục: PostgreSQL có base backup hằng ngày kèm WAL liên tục, MongoDB có backup hằng ngày kèm PITR qua oplog, cả hai đều ghi vào S3 mã hóa KMS.

Phần còn thiếu để khép vòng là diễn tập khôi phục định kỳ. Backup chỉ có giá trị khi đã chứng minh khôi phục được, nên bước tiếp theo là khôi phục vào môi trường cô lập, đối chiếu số bản ghi và index, chạy lại xác minh chuỗi băm audit, rồi ghi nhận RPO và RTO đo được. Cho tới khi có kết quả diễn tập, các tham số như chu kỳ oplog 10 phút chỉ phản ánh năng lực đã cấu hình. Tương tự, Loki chưa bật persistence nên log lịch sử chưa bền vững qua restart, và alert rule nghiệp vụ vẫn cần bổ sung trước khi vận hành dài hạn.

4.6 Đánh giá trạng thái triển khai

4.6.1 Mapping kiểm soát, công cụ và bằng chứng

Bảng 4.1: Mapping kiểm soát DevSecOps với công cụ và bằng chứng

Khu vực	Công cụ	Bằng chứng chính	Điều kiện không đạt
Secret scanning	TruffleHog	Report và log Jenkins	Phát hiện secret xác thực
Quality	pnpm, Jest, ESLint, Turbo	Unit test, lint và build log	Test, lint hoặc build lỗi
SAST	SonarQube	Dashboard và Quality Gate	Quality Gate fail/timeout
SCA	Dependency-Check	HTML/JSON/log	CVSS vượt ngưỡng hoặc ngoại lệ chưa duyệt
Filesystem scan	Trivy	JSON report	HIGH/CRITICAL finding
IaC syntax	Terraform	Fmt/validate log	Terraform không hợp lệ
IaC security	Checkov	Text report	Policy nghiêm trọng fail
Policy-as-code	Kyverno CLI	Policy report và manifest render	Workload vi phạm policy
Container image	Docker Build-Kit, Trivy	Build log, image scan	Build lỗi hoặc CRITICAL CVE
Registry	Harbor	Repository, tag, digest, scan	Push lỗi hoặc artifact thiếu digest
Artifact integrity	in-Cosign	Sign/verify log	Ký hoặc xác minh thất bại khi được bật
GitOps	Git, Argo CD	GitOps commit, revision, sync status	Push lỗi hoặc app OutOf-Sync/Degraded

Khu vực	Công cụ	Bằng chứng chính	Điều kiện không đạt
Post-deploy	Smoke test, ZAP	HTTP log, HTML/JSON report	Target lỗi hoặc High/Critical finding
Observability	Prometheus, Grafana, Loki	Dashboard, alert và log query	Mất metric/log hoặc workload unhealthy
Cloud security	IRSA, Secrets Manager, KMS	IAM role, ExternalSecret, encrypted bucket	Quyền quá rộng hoặc secret bị lộ

4.6.2 Nhận xét trạng thái triển khai

Kết quả đối chiếu mã nguồn, cấu hình hạ tầng và bằng chứng triển khai cho thấy DocVault đã triển khai các thành phần cốt lõi của quy trình DevSecOps: IaC cho EKS, các cổng kiểm soát trong CI, private registry, image digest, SBOM, ký artifact bằng Cosign, GitOps, kiểm tra sau triển khai, quản lý bí mật, thu thập metric/log và backup cơ sở dữ liệu.

Các giới hạn hiện tại gồm:

- Môi trường EKS ưu tiên chi phí kiểm thử; worker node công khai và phạm vi CIDR rộng của API chỉ nên được sử dụng tạm thời.
- Ký image bằng Cosign đã có bằng chứng trên Harbor nhưng vẫn chưa sử dụng Kyverno trong cluster để xác minh chữ ký.
- Kyverno hiện kiểm tra manifest trong CI; cụm chưa bắt buộc admission policy theo bằng chứng hiện có.
- NetworkPolicy chưa giới hạn egress theo từng dependency.
- Loki chưa có persistence và cấu hình Grafana cần được đưa hoàn toàn sang secret manager.
- Backup PostgreSQL và MongoDB đã được cấu hình nhưng cần diễn tập restore và ghi nhận RPO/RTO.
- Harbor đã lưu SBOM cho artifact, nhưng provenance chưa được chuẩn hóa theo SLSA/in-toto (chưa xác thực SBOM attestation bằng Kyverno) [17].

Những giới hạn trên xác định ranh giới giữa môi trường đồ án có kiểm soát và hệ thống production. Các bước gia cố tiếp theo gồm remote Terraform state có locking, private node group, admission control, provenance chuẩn hóa, bắt buộc xác minh chữ ký image, NetworkPolicy theo nguyên tắc đặc quyền tối thiểu, cảnh báo vận hành và kiểm thử phục hồi định kỳ.

Sau khi hệ thống đã được hiện thực và triển khai, bước tiếp theo là kiểm tra mức độ

đáp ứng của các luồng nghiệp vụ, kiểm soát bảo mật và chuỗi phát hành. Nội dung này được trình bày trong chương kiểm thử và đánh giá hệ thống.

CHƯƠNG 5

KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày chiến lược kiểm thử và đánh giá hệ thống DocVault sau khi hiện thực và triển khai. Các nội dung đánh giá bao gồm kiểm thử runtime, kiểm thử giao diện, kiểm thử bảo mật, kiểm tra pipeline DevSecOps, đối chiếu với yêu cầu ban đầu và phân tích các hạn chế hiện tại của hệ thống.

5.1 Chiến lược kiểm thử

Kiểm thử DocVault được tổ chức theo nhiều lớp để kiểm tra cả nghiệp vụ tài liệu và quy trình phát hành. Ở lớp mã nguồn, các service backend có unit test cho policy, controller, guard bảo mật và các helper quan trọng. Frontend có kiểm thử cho permission helper, form, component và một số behavior giao diện. Ở lớp runtime, nhóm sử dụng kiểm thử E2E qua API thật để kiểm tra các luồng nghiệp vụ và bảo mật chính. Ở lớp pipeline, Jenkins kiểm tra chuỗi release từ mã nguồn đến môi trường triển khai.

Mục tiêu của chương này không chỉ là chứng minh hệ thống chạy được, mà còn chứng minh các luồng quan trọng đã được kiểm tra. Với nghiệp vụ tài liệu, kiểm thử cần cho thấy tài liệu có thể đi qua các bước tạo mới, upload version, submit, approve, preview/download, retention và audit/evidence. Với release, kiểm thử cần cho thấy pipeline kiểm soát được đường đi từ mã nguồn đến image, GitOps revision, workload trên Kubernetes và kiểm tra sau triển khai.

Chiến lược kiểm thử này phù hợp với đặc thù của hệ thống. Một unit test có thể kiểm tra một rule policy cụ thể, nhưng chưa đủ để chứng minh toàn bộ luồng từ frontend, gateway, metadata-service, document-service đến audit-service. Ngược lại, E2E runtime có thể chứng minh luồng chính hoạt động, nhưng khó bao phủ tất cả nhánh nhỏ của policy. Vì vậy, nhóm kết hợp nhiều loại kiểm thử để có cái nhìn đầy đủ hơn về hệ thống.

5.2 Kiểm thử runtime

Để kiểm tra các luồng runtime chính, nhóm sử dụng lệnh kiểm thử E2E:

Listing 5.1: Lệnh kiểm thử E2E runtime

```
pnpm test:e2e
```

Bộ kiểm thử này gọi trực tiếp vào các API của hệ thống để kiểm tra các luồng quan trọng như xác thực, phân quyền, quản lý tài liệu, workflow, preview/download, malware scanning, DLP, retention, Compliance Officer boundary và audit verify-chain. Kết quả kiểm thử gần nhất ghi nhận toàn bộ các kiểm tra bắt buộc đều đạt.

Các nhóm kiểm thử runtime gồm:

- **Authentication boundary:** request thiếu token hoặc token hết hạn phải bị từ chối.
- **Create policy:** viewer không được tạo tài liệu, editor được phép tạo tài liệu.
- **Metadata policy:** truy cập trái phép vào confidential document bị từ chối ở các API detail, history, comments và ACL.
- **GROUP ACL:** quyền READ theo group được normalize và kiểm tra đúng.
- **Confidential download posture:** viewer bị từ chối, editor nhận stream-only response hoặc controlled stream theo policy.
- **Malware scanning:** file EICAR bị chặn trước khi tạo object/version.
- **DLP:** file có dữ liệu nhạy cảm bị phát hiện và nâng classification.
- **Workflow:** submit, approve và duplicate approve conflict được kiểm tra.
- **Retention:** retention job có thể archive tài liệu đến hạn và ghi nhận workflow history/audit.
- **Compliance Officer boundary:** Compliance Officer được xem metadata, audit, retention và evidence nhưng bị chặn preview, presign, stream và download nội dung file.
- **Audit:** hỗ trợ query audit, verify-chain, security summary và chặn người dùng thường giả mạo audit event.

Bảng 5.1 tóm tắt các nhóm kiểm thử runtime, bằng chứng tương ứng và trạng thái kết quả.

Bảng 5.1: Tóm tắt bằng chứng kiểm thử runtime

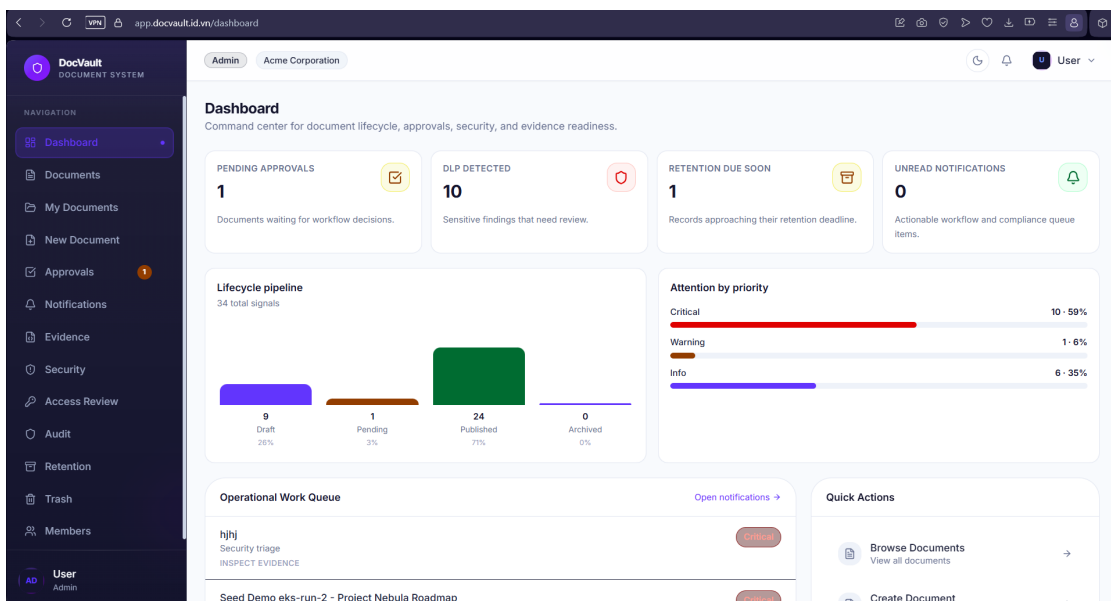
Khu vực	Bằng chứng	Kết quả
Authentication	Missing/expired token rejected	PASS
Authorization	Viewer create denied, editor create allowed	PASS
Metadata policy	Viewer guessed confidential access denied	PASS
ACL	GROUP READ ACL metadata access	PASS
Download posture	Confidential presign withheld direct URL	PASS

Khu vực	Bằng chứng	Kết quả
Malware	EICAR upload blocked before object/version creation	PASS
DLP	Sensitive upload escalated classification	PASS
Workflow	Submit/approve/duplicate approve conflict	PASS
Retention	Auto-archive due record and workflow history	PASS
Compliance boundary	Compliance file content access denied	PASS
Audit	Verify-chain and security summary available	PASS
Audit ingest	Viewer cannot ingest fake audit event	PASS

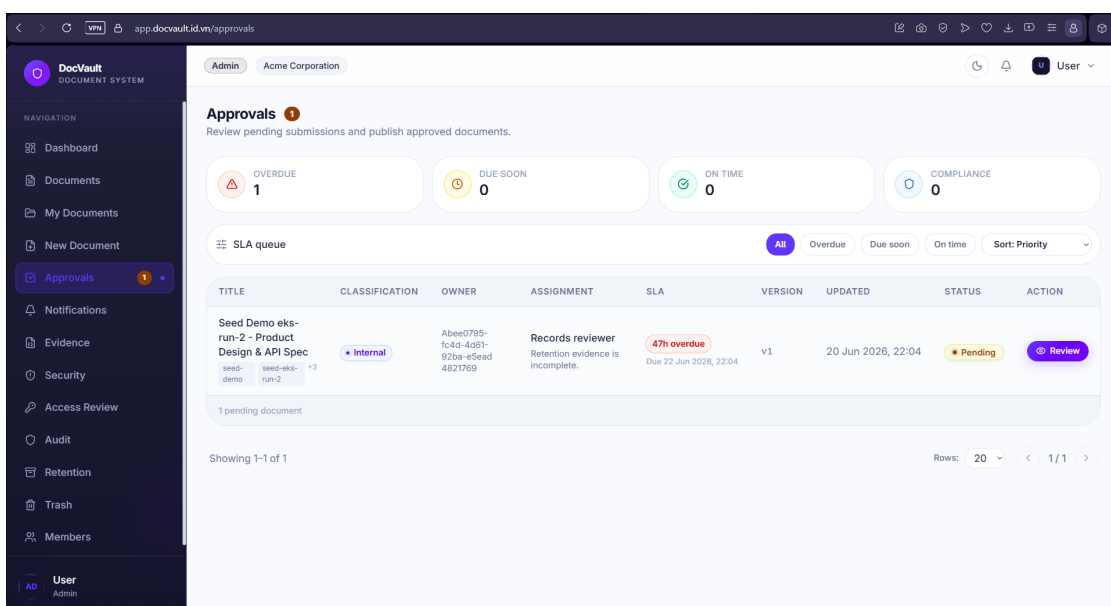
5.3 Kiểm thử giao diện và bằng chứng UI

Frontend được kiểm tra bằng các bước lint, build, typecheck và test component/helper. Các bước này giúp phát hiện lỗi cú pháp, lỗi kiểu dữ liệu, lỗi build và một số lỗi ở tầng giao diện trước khi triển khai.

Bên cạnh kiểm thử tự động, nhóm cũng thu thập ảnh chụp các màn hình chính của hệ thống sau khi triển khai. Các màn hình được kiểm tra gồm đăng nhập, xác thực Keycloak, Dashboard, Documents, My Documents, New Document, Document Detail, Preview, Approvals, Audit, Security, Evidence Center, Retention và Access Review. Những ảnh chụp này được dùng để minh họa giao diện và làm bằng chứng trực quan rằng các màn hình chính có thể render đúng, hiển thị dữ liệu phù hợp và phản ánh trạng thái hệ thống tại thời điểm kiểm thử.

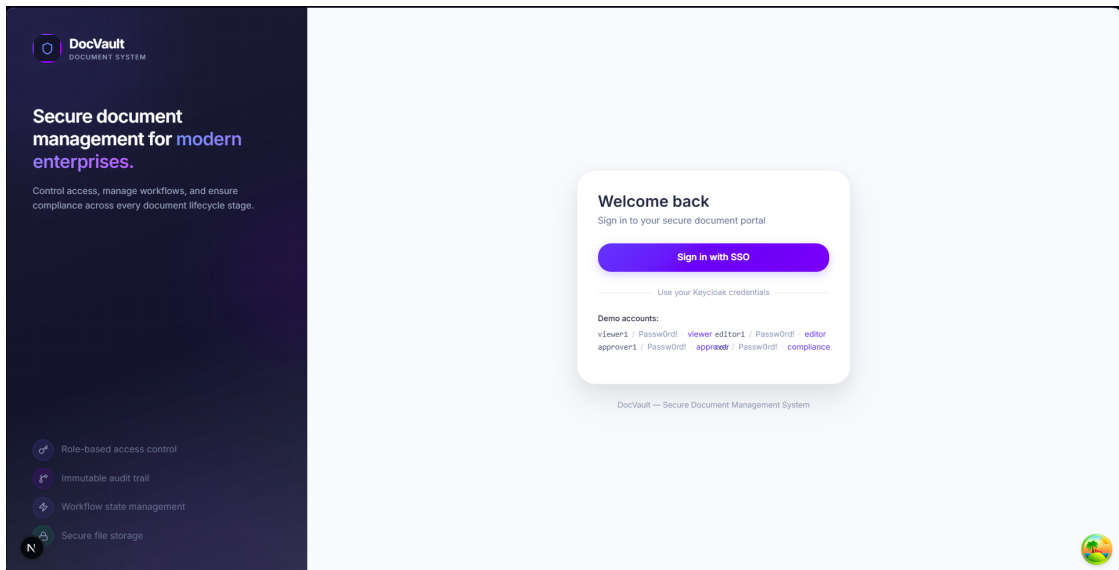


Hình 5.1: Bảng chứng giao diện dashboard command center

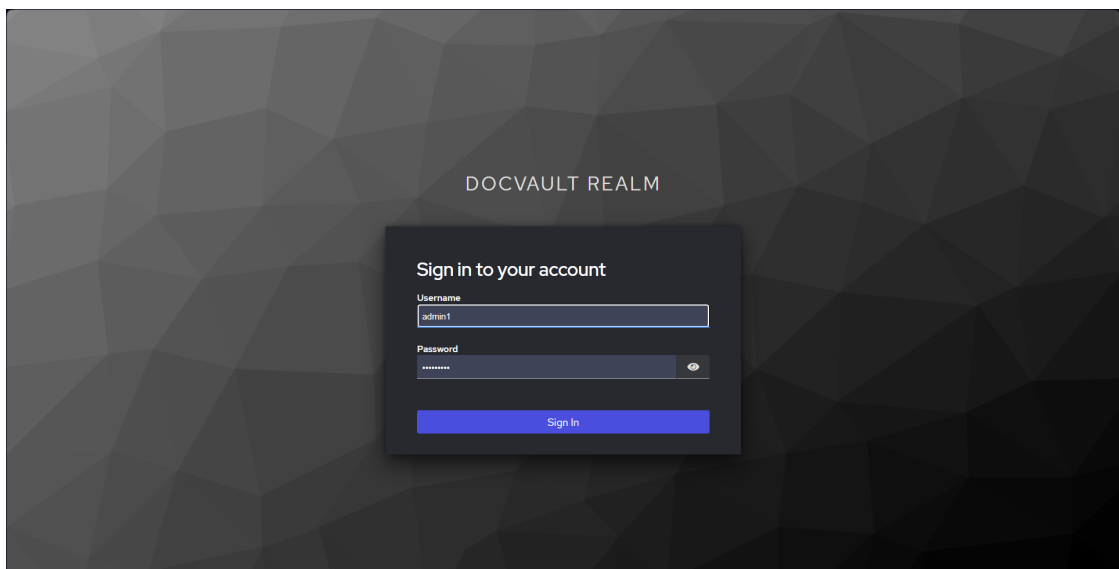


Hình 5.2: Bảng chứng giao diện hàng đợi phê duyệt và trạng thái SLA

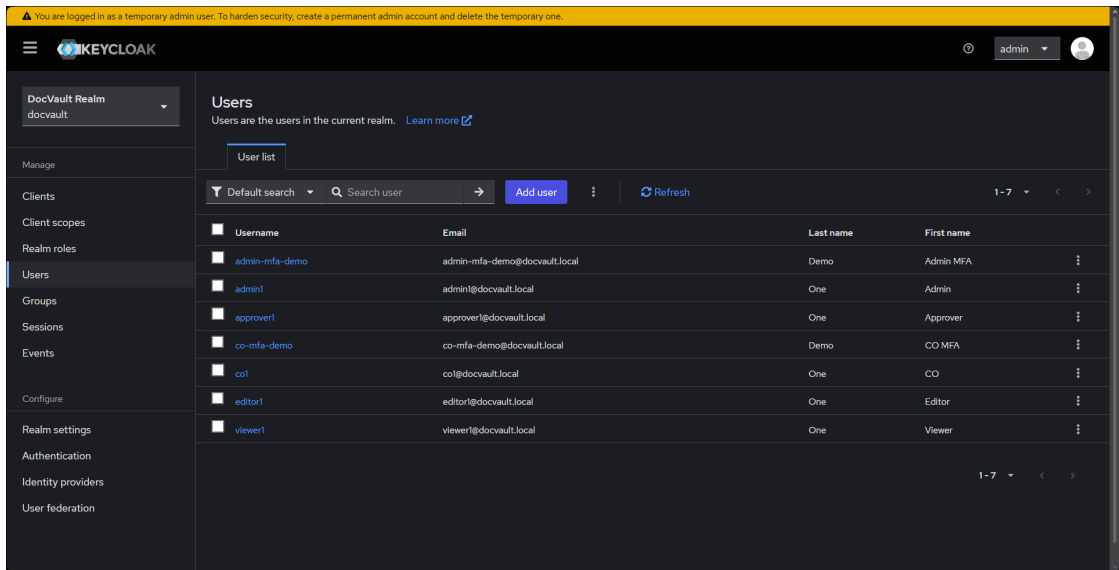
Nhóm ảnh bổ sung dưới đây làm rõ hơn đường đi của người dùng từ đăng nhập, tạo tài liệu, xem chi tiết, preview nội dung đến đối soát evidence và retention.



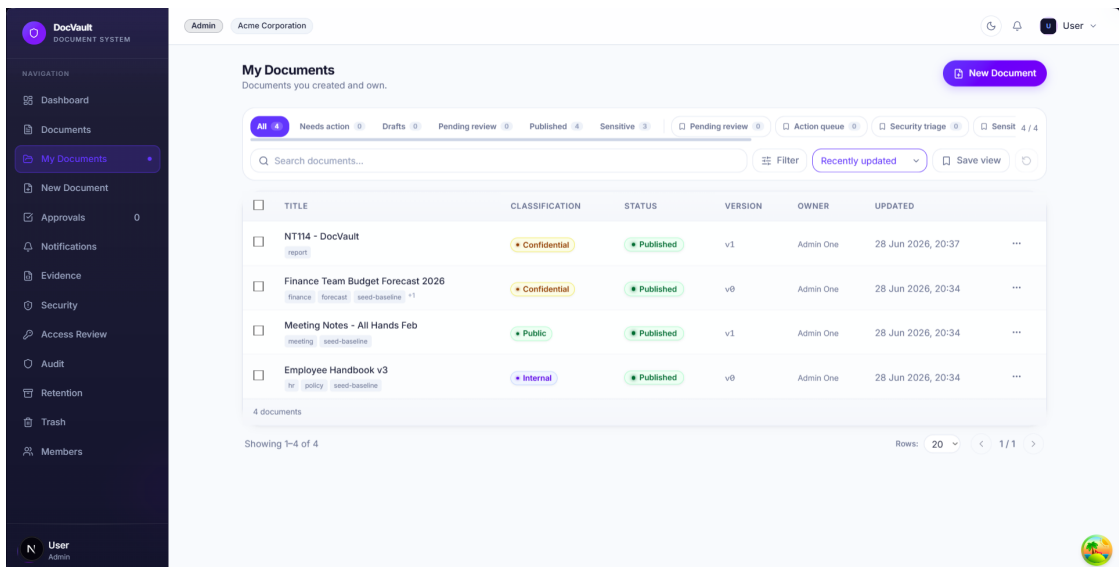
Hình 5.3: Bảng chứng trang đăng nhập ứng dụng DocVault



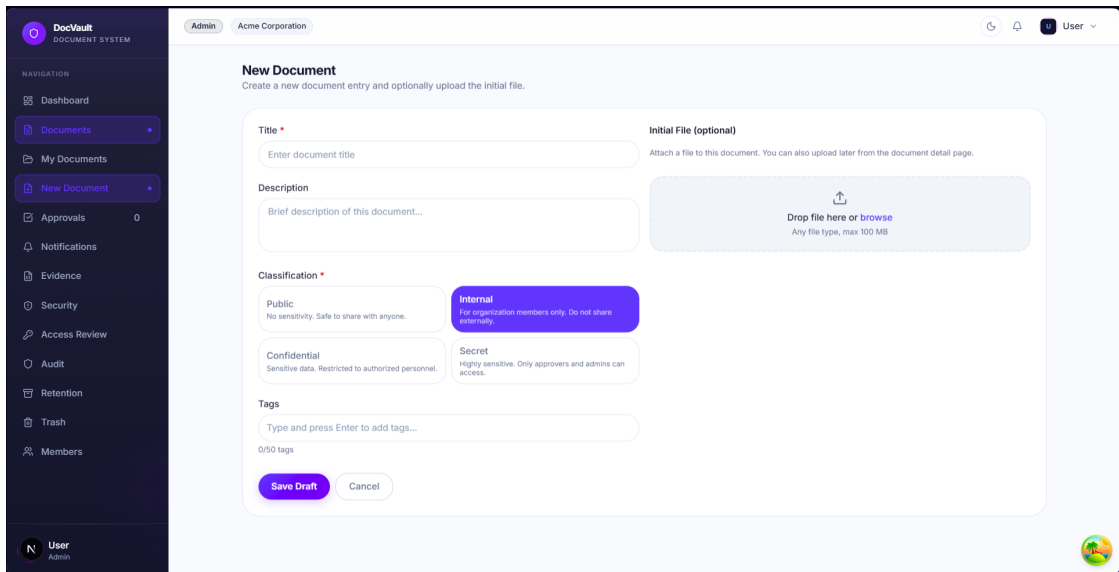
Hình 5.4: Bảng chứng trang xác thực Keycloak



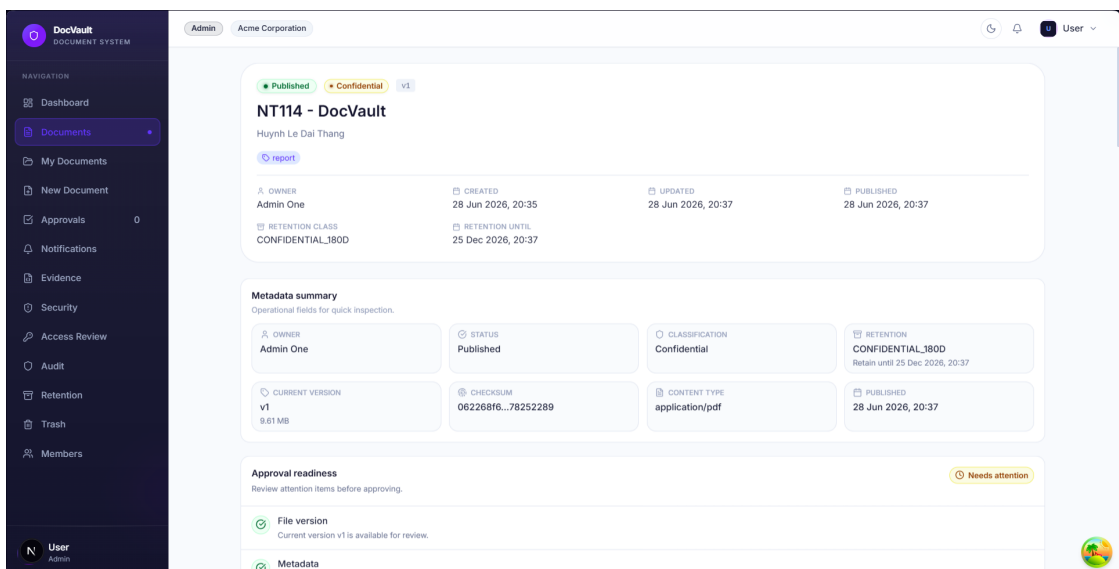
Hình 5.5: Bảng chứng quản trị user trong realm Keycloak



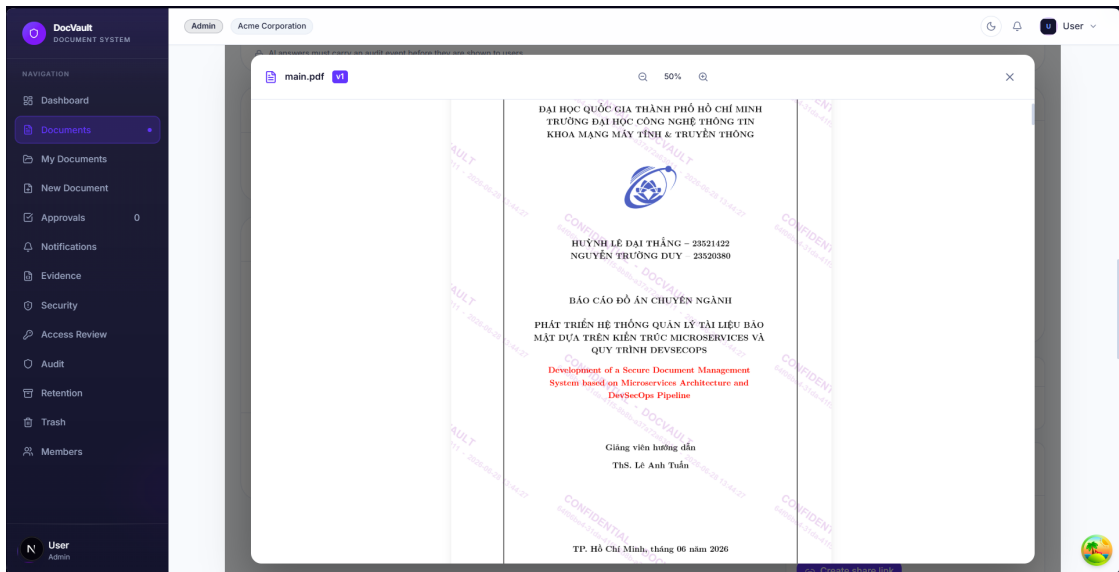
Hình 5.6: Bảng chứng danh sách tài liệu của người dùng



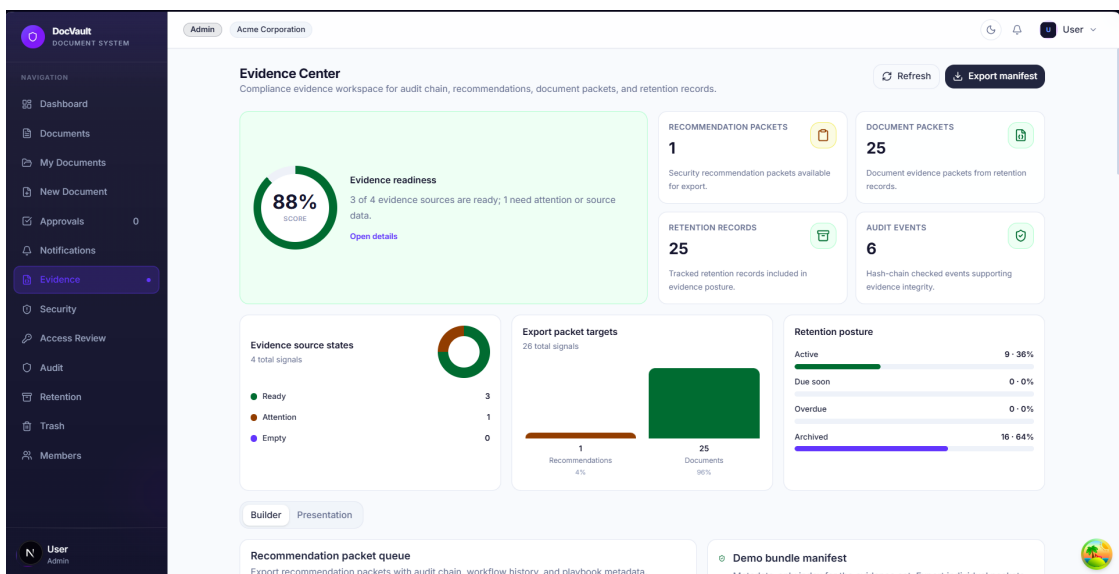
Hình 5.7: Bảng chứng tạo tài liệu mới và upload file



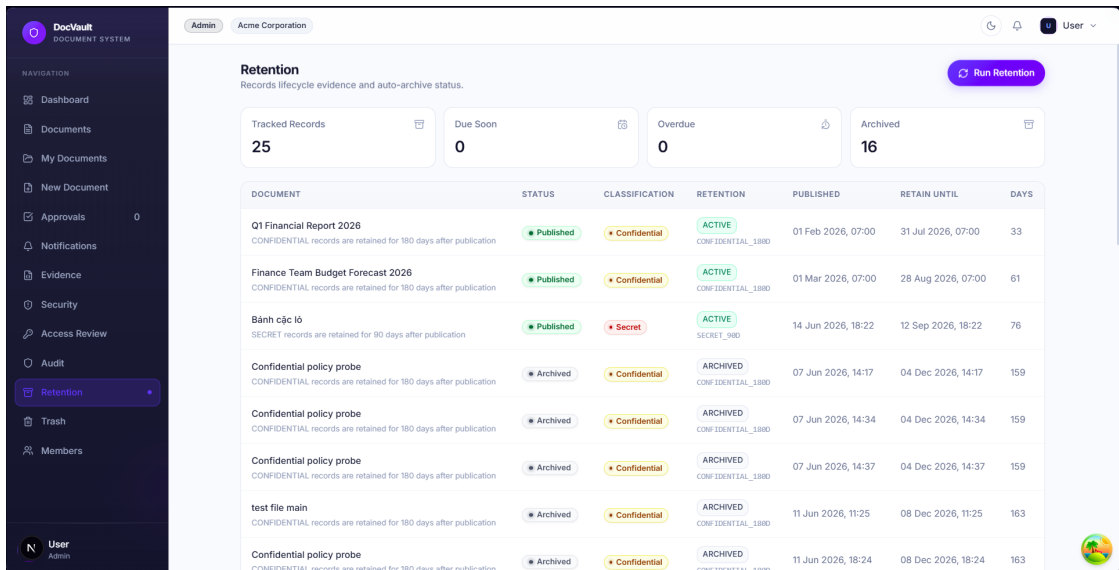
Hình 5.8: Bảng chứng chi tiết tài liệu và metadata



Hình 5.9: Bảng chứng preview tài liệu qua luồng kiểm soát



Hình 5.10: Bảng chứng giao diện Evidence Center



Hình 5.11: Bảng chứng giao diện Retention và trạng thái lưu giữ

Tuy nhiên, phạm vi kiểm thử frontend hiện tại chủ yếu tập trung vào build, component/helper và bảng chứng giao diện. Kiểm thử end-to-end ở cấp trình duyệt đầy đủ vẫn là hướng cần bổ sung thêm nếu hệ thống được phát triển tiếp.

5.4 Bảng chứng kiểm thử bảo mật

Các kiểm soát bảo mật runtime được nhóm đối chiếu thông qua kiểm thử API, kết quả E2E và bảng chứng giao diện. Mục tiêu của phần này là xác nhận các quyết định bảo mật quan trọng không chỉ được thể hiện trên giao diện, mà còn được thực thi ở backend.

Các điểm chính đã được kiểm tra gồm:

- Compliance Officer bị chặn preview, stream, presign và download nội dung file.
- Metadata access boundary được áp dụng cho detail, workflow history, comments và ACL list.
- Frontend permission helper được căn chỉnh với backend policy ở mức UI hint, nhưng backend vẫn là nơi quyết định cuối cùng.
- Grant token cho download/preview có TTL ngắn hạn và hỗ trợ rotation bằng `kid`.
- Malware scanning chặn file EICAR trước khi ghi object hoặc tạo version.
- DLP phát hiện pattern nhạy cảm và nâng classification.
- Audit ingest yêu cầu service token, người dùng thường không thể tự ghi audit event giả.
- Audit verify-chain và security summary được expose qua gateway để kiểm tra trạng thái audit.
- Evidence packet không chứa nội dung file, presigned URL, grant token hoặc object

key nhạy cảm.

Kết quả này cho thấy các kiểm soát bảo mật chính của phần ứng dụng đã được kiểm tra ở mức runtime. Một số nội dung vẫn cần phát triển thêm nếu hướng đến production, chẳng hạn kiểm thử nhiều trường hợp biên hơn cho policy, kiểm thử trình duyệt đầy đủ và kiểm tra tải lớn với nhiều tài liệu/audit event.

Bên cạnh các kiểm thử trực tiếp trên chức năng tài liệu, nhóm cũng kiểm tra việc ghi nhận audit event cho một số hoạt động bảo mật và vận hành. Mục tiêu của phần này là xác nhận rằng các thao tác quan trọng trong hệ thống đều để lại dấu vết đủ rõ để phục vụ truy vết và kiểm tra sau này.

Bảng 5.2: Kiểm chứng audit event cho hoạt động bảo mật và vận hành

Nhóm hoạt động	Nội dung kiểm chứng	Ý nghĩa
Truy cập nội dung	Ghi nhận các lần pre-view/download thành công hoặc bị từ chối.	Kiểm tra hệ thống có lưu lại actor, thời điểm, kết quả xử lý và lý do từ chối khi có.
Kiểm soát nội dung	Ghi nhận các trường hợp liên quan đến DLP, malware scanning hoặc guardrail bảo mật.	Cho thấy hệ thống có lưu lại các tình huống có rủi ro về nội dung hoặc file.
Retention và thao tác nhạy cảm	Ghi nhận việc chạy retention job hoặc thực hiện thao tác cần xác nhận bổ sung.	Cung cấp dấu vết cho các thao tác có ảnh hưởng đến dữ liệu hoặc trạng thái tài liệu.
Quản trị người dùng	Ghi nhận thay đổi thành viên, vai trò hoặc quyền truy cập.	Hỗ trợ kiểm tra lại các thay đổi có thể ảnh hưởng đến quyền trong hệ thống.
Comment và share link	Ghi nhận thao tác bình luận, chia sẻ, thu hồi hoặc sử dụng liên kết chia sẻ.	Theo dõi các hoạt động cộng tác và chia sẻ liên quan đến tài liệu.
Khuyến nghị bảo mật	Ghi nhận quá trình tạo hoặc cập nhật trạng thái xử lý khuyến nghị.	Hỗ trợ quản trị viên theo dõi quá trình kiểm tra một hành vi đáng chú ý.

Kết quả kiểm thử cho thấy audit log trong DocVault không chỉ phục vụ việc xem lại lịch sử của từng tài liệu, mà còn ghi nhận được các hoạt động bảo mật và vận hành ở phạm vi hệ thống. Các dữ liệu này là cơ sở để quản trị viên kiểm tra lại hành vi người dùng, đánh giá các khuyến nghị bảo mật và đưa ra hướng xử lý phù hợp.

5.5 Bảng chứng quy trình pipeline

Các bảng chứng pipeline được thu thập từ quá trình chạy Jenkins và môi trường triển khai. Nhóm sử dụng log pipeline, kết quả scan, trạng thái artifact trên Harbor, GitOps commit, Argo CD health check, smoke test, ZAP report và dashboard quan sát vận hành để đối chiếu quá trình phát hành hệ thống.

Cụ thể, bảng chứng DevSecOps gồm:

- Jenkins build và Stage View cho thấy pipeline chạy qua các stage chính.
- TruffleHog secret scan để kiểm tra credential bị commit trong repository.
- Lint, unit test và workspace build để kiểm tra chất lượng cơ bản của mã nguồn.
- SonarQube Quality Gate để phân tích mã nguồn tĩnh.
- Dependency Check để quét lỗ hổng thư viện phụ thuộc.
- Trivy filesystem/image scan để quét filesystem và Docker image.
- Checkov, Terraform Validate và Kyverno report để kiểm tra IaC và policy Kubernetes.
- Harbor image tag/digest, SBOM, trạng thái ký và kết quả quét lỗ hổng.
- Log ký và xác minh Cosign tương ứng với image digest.
- GitOps commit cập nhật repository, image tag và digest trong Helm values.
- Argo CD application đạt trạng thái Synced/Healthy.
- Smoke test cho web root và `/api/health`.
- OWASP ZAP report sau triển khai.
- Grafana/Loki dashboard cho metric, log và quan sát workload.
- Bảng chứng hạ tầng như EKS, S3/KMS, IAM/IRSA, Secrets Manager và backup cơ sở dữ liệu.

Các bảng chứng này giúp nhóm đối chiếu được quá trình phát hành từ mã nguồn đến container artifact, GitOps revision và workload trên Kubernetes. Nhờ đó, phần DevSecOps không chỉ dừng ở việc có pipeline, mà còn có bằng chứng cho từng giai đoạn quan trọng của quy trình triển khai.

5.6 Đánh giá theo yêu cầu

Bảng mapping yêu cầu với hiện thực và bằng chứng được dùng để kiểm tra mức độ đáp ứng của hệ thống so với mục tiêu ban đầu. Các bằng chứng ứng dụng cho thấy hệ thống đã quản lý được vòng đời tài liệu, kiểm soát quyền truy cập, ghi audit log, xuất evidence và xử lý retention. Các bằng chứng DevSecOps cho thấy release đã đi qua pipeline, container registry, GitOps, Argo CD và các bước kiểm tra sau triển khai.

Bảng 5.3 giúp người đọc đối chiếu nhanh giữa yêu cầu, hiện thực và bằng chứng.

Bảng 5.3: Mapping yêu cầu với hiện thực và bằng chứng

Yêu cầu	Hiện thực	Bằng chứng
Đăng nhập và phân quyền	Keycloak, JWT, gateway role guard, frontend permission helper	E2E auth checks, kiểm tra route bảo vệ
Quản lý vòng đời tài liệu	Documents, versions, workflow history, workflow-service transitions	E2E workflow checks, workflow history
Kiểm soát pre-view/download	Metadata authorization, grant token, document-service pre-sign/stream	Runtime security checks, E2E confidential posture
Compliance boundary	Compliance metadata/audit/evidence allowed, file content denied	Runtime compliance checks, E2E compliance checks
Audit tamper-evident	audit-service hash-chain, verify-chain endpoint	Kết quả verify-chain, audit events
Malware/DLP	Malware scanner, DLP scanner, classification escalation	E2E malware/DLP checks
Retention evidence	Retention fields, retention run, audit event	E2E retention checks
DevSecOps pipeline	Jenkins gates, Harbor/digest, Kyverno, GitOps, Argo CD, smoke, ZAP	Pipeline log, Harbor, Argo CD, ZAP evidence

5.7 Hạn chế hiện tại

Kết quả kiểm thử và đối chiếu cấu hình cho thấy hệ thống đã đáp ứng được các luồng chính, nhưng vẫn còn một số giới hạn cần tiếp tục hoàn thiện.

Ở phần ứng dụng, notification-service hiện vẫn chủ yếu phục vụ môi trường phát

triển/demo, chưa tích hợp đầy đủ email, webhook hoặc push notification. Kiểm thử end-to-end ở cấp trình duyệt chưa phải bằng chứng chính cho toàn bộ luồng nghiệp vụ; phạm vi kiểm chứng hiện tập trung nhiều hơn vào API và runtime. Ngoài ra, một số thư viện xác thực và contract dùng chung giữa các service vẫn cần tiếp tục hợp nhất để giảm trùng lặp.

Ở phần DevSecOps, Harbor đã có SBOM và trạng thái ký của artifact, nhưng provenance theo chuẩn SLSA/in-toto và chính sách bắt buộc xác minh chữ ký bằng Kyverno trong cluster chưa được triển khai đầy đủ. Kyverno hiện đã được dùng như công policy-as-code trong CI với manifest đã render, nhưng cụm EKS chưa có bằng chứng về admission controller chặn trực tiếp tài nguyên vi phạm. NetworkPolicy vẫn cần giới hạn egress chi tiết hơn, Loki cần bật persistence, và cấu hình monitoring nên được chuyển đầy đủ sang External Secrets.

Backup PostgreSQL và MongoDB đã được cấu hình trên S3/KMS và chạy theo lịch, nhưng quy trình khôi phục vẫn cần được diễn tập và đo lường để xác minh RPO/RTO. Đây là phần quan trọng nếu hệ thống muốn tiến gần hơn đến môi trường production.

5.8 Nhận xét chung

Qua các kết quả kiểm thử, DocVault đã đáp ứng được các mục tiêu cốt lõi của một hệ thống quản lý tài liệu bảo mật trong phạm vi đề án chuyên ngành. Các luồng chính như tạo tài liệu, upload version, workflow phê duyệt, kiểm soát preview/download, audit, evidence, retention, malware scanning và DLP đều có kiểm thử hoặc bằng chứng tương ứng.

Ở phần DevSecOps, hệ thống đã có pipeline kiểm soát chuỗi phát hành từ source code đến workload. Các bằng chứng từ Jenkins, Harbor, Argo CD, ZAP, Grafana/Loki và AWS giúp đối chiếu lại quá trình release của hệ thống.

Tuy vậy, kết quả kiểm thử hiện tại chủ yếu chứng minh hệ thống ở mức đề án chuyên ngành và môi trường kiểm thử có kiểm soát. Nếu phát triển tiếp theo hướng production, nhóm cần bổ sung kiểm thử trình duyệt đầy đủ, kiểm thử tải lớn, admission policy trong cluster, NetworkPolicy chi tiết, log persistence và diễn tập khôi phục dữ liệu.

Các kết quả kiểm thử và đánh giá trên là cơ sở để tổng kết mức độ hoàn thành đề tài, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

Sau quá trình thực hiện, nhóm đã xây dựng được DocVault theo đúng phạm vi đặt ra: một hệ thống quản lý tài liệu bảo mật có các luồng nghiệp vụ chính và một môi trường triển khai cloud-native có pipeline kiểm soát phát hành. Kết quả quan trọng nhất không nằm ở việc ghép nhiều công cụ lại với nhau, mà ở việc các thao tác trên tài liệu và các bước release đều có dấu vết để kiểm tra lại.

Với nghiệp vụ tài liệu, DocVault đã xử lý được các bước từ tạo draft, cập nhật metadata, upload version, gửi duyệt, approve/reject, xuất bản, preview/download đến retention và archive. Mỗi tài liệu đi kèm metadata, checksum phiên bản, trạng thái workflow, lịch sử phê duyệt và audit event. Khi cần kiểm tra lại, hệ thống có thể tổng hợp các dữ liệu này thành evidence thay vì chỉ trả về một file rời rạc.

Bên cạnh truy vết vòng đời tài liệu, hệ thống còn ghi nhận các hoạt động bảo mật và vận hành quan trọng, từ đó hỗ trợ quản trị viên kiểm tra hành vi người dùng và xử lý các khuyến nghị bảo mật khi cần.

Về bảo mật ứng dụng, nhóm đã hiện thực các lớp kiểm soát chính gồm RBAC, ACL, classification, ranh giới metadata-only cho Compliance Officer, grant token ngắn hạn, malware scanning, DLP và audit hash-chain. Trong quá trình kiểm thử, nhóm đặc biệt chú ý các điểm dễ bị bỏ sót như viewer cố truy cập tài liệu confidential, Compliance Officer bị chặn nội dung file, file EICAR bị từ chối trước khi tạo version và audit event giả bị chặn ở backend.

Với DevSecOps, nhóm đã hoàn thiện được chuỗi phát hành từ mã nguồn đến workload trên EKS. Jenkins giữ vai trò điều phối kiểm thử và các security gate, Harbor lưu artifact cùng digest/SBOM/trạng thái ký, GitOps ghi nhận desired state, còn Argo CD cung cấp trạng thái đồng bộ và health sau triển khai. Smoke test, ZAP report và dashboard vận hành giúp đối chiếu thêm rằng bản phát hành không chỉ được build thành công mà còn chạy được ở môi trường triển khai.

Nhìn chung, hệ thống đã đáp ứng được mục tiêu chính của đề tài ở mức đồ án chuyên ngành. DocVault chưa phải là một hệ thống production hoàn chỉnh, nhưng đã thể hiện được cách kết hợp giữa một ứng dụng quản lý tài liệu bảo mật và một quy trình DevSecOps có kiểm soát.

6.2 Mức độ đáp ứng mục tiêu

Đối chiếu với các mục tiêu ban đầu, đề tài đã đạt được các kết quả sau:

- Xây dựng kiến trúc microservices với ranh giới trách nhiệm tương đối rõ giữa các dịch vụ.
- Hoàn thiện các luồng nghiệp vụ chính của hệ thống quản lý tài liệu và áp dụng kiểm soát truy cập ở backend.
- Cung cấp bằng chứng kiểm thử cho xác thực, RBAC/ACL, truy cập tài liệu nhạy cảm, malware, DLP, retention, audit và ranh giới quyền của Compliance Officer.
- Triển khai hạ tầng cloud-native trên EKS bằng Terraform, Helm, Kustomize và GitOps.
- Tích hợp các cổng chất lượng và bảo mật vào pipeline, bao gồm kiểm tra mã nguồn, dependency, secret, IaC, policy Kubernetes và container artifact.
- Áp dụng S3/KMS, IRSA, External Secrets, SBOM, ký artifact, backup và observability trong môi trường triển khai.

6.3 Hạn chế

Do thời gian và tài nguyên triển khai còn giới hạn, DocVault hiện tập trung vào việc chứng minh các luồng chính thay vì tối ưu đầy đủ cho production. Nhóm ưu tiên các điểm có thể kiểm thử và có bằng chứng rõ ràng như vòng đời tài liệu, phân quyền, audit, evidence, retention và chuỗi phát hành. Vì vậy, một số phần mang tính gia cố vận hành được ghi nhận là hạn chế thay vì cố trình bày như đã hoàn thiện.

Ở phần ứng dụng, hệ thống vẫn cần bổ sung thêm kiểm thử trình duyệt đầy đủ, tối ưu trải nghiệm người dùng, hoàn thiện thêm các trường hợp biên của workflow và mở rộng khả năng xử lý khi số lượng tài liệu lớn hơn. Các cơ chế như DLP, malware scanning và retention hiện đã được tích hợp ở mức phục vụ đồ án, nhưng vẫn cần được đánh giá sâu hơn nếu áp dụng trong môi trường thực tế.

Ở phần DevSecOps, một số nội dung production hardening như NetworkPolicy chi tiết, admission controller bắt buộc xác minh chữ ký image, provenance chuẩn hóa, log persistence, backup/restore và diễn tập khôi phục dữ liệu vẫn chưa được hoàn thiện đầy đủ. Pipeline hiện đã chứng minh được chuỗi kiểm soát từ code đến deployment, nhưng thời gian chạy scan, cách xử lý CVE ngoại lệ và quy trình review kết quả vẫn cần tối ưu nếu áp dụng lâu dài.

6.4 Hướng phát triển

Trong thời gian tới, DocVault có thể được phát triển thêm ở cả phần ứng dụng và phần hạ tầng triển khai. Với phần ứng dụng, hệ thống có thể bổ sung các chức năng thực tế hơn như tìm kiếm nâng cao, dashboard theo trạng thái tài liệu, nhắc nhở tài liệu chờ duyệt, báo cáo hoạt động và cải thiện evidence packet. Ngoài ra, nhóm cũng có thể tối ưu giao diện, hiệu năng truy vấn và khả năng xử lý khi số lượng tài liệu, version file và audit event tăng lên.

Một hướng mở rộng đáng chú ý là tích hợp AI để hỗ trợ phân loại tài liệu. Khi người dùng upload hoặc cập nhật tài liệu, AI có thể phân tích nội dung và gợi ý mức classification phù hợp kèm độ tin cậy. Nếu kết quả AI khác với lựa chọn của người dùng, hệ thống chỉ nên cảnh báo hoặc yêu cầu xác nhận lại, không tự động thay đổi một cách im lặng. Các đề xuất của AI và phản hồi từ người dùng hoặc Compliance Officer sẽ được ghi audit để phục vụ truy vết và cải thiện mô hình về sau.

Về bảo mật và DevSecOps, hệ thống có thể tiếp tục hoàn thiện DLP, malware scanning, kiểm soát tải xuống theo ngữ cảnh, audit bền vững hơn và kiểm thử thêm các trường hợp biên của policy. Ở phần hạ tầng, nhóm có thể bổ sung các kiểm soát gần với production như NetworkPolicy chi tiết, admission policy, xác minh chữ ký image, provenance chuẩn hóa, backup/restore và diễn tập khôi phục. Bên cạnh đó, autoscaling cho Kubernetes, blue-green deployment hoặc canary deployment cũng là các hướng nên nghiên cứu để hệ thống ổn định hơn khi tải tăng và giảm rủi ro khi phát hành phiên bản mới.

6.5 Kết luận chung

Qua quá trình thực hiện đề tài, nhóm đã xây dựng và triển khai được DocVault như một hệ thống có cả nghiệp vụ tài liệu và quy trình phát hành kèm bằng chứng. Ở mức ứng dụng, hệ thống quản lý tài liệu theo vòng đời, kiểm soát truy cập và ghi nhận audit/evidence. Ở mức triển khai, hệ thống đi qua pipeline, container registry, GitOps và Argo CD trước khi được kiểm tra lại trên môi trường EKS.

Ở phần ứng dụng, DocVault đã thể hiện được cách quản lý một tài liệu từ lúc tạo mới, upload version, gửi duyệt, phê duyệt, xuất bản cho đến retention hoặc archive. Tài liệu trong hệ thống không chỉ là một file lưu trữ, mà còn có metadata, phiên bản, trạng thái workflow, quyền truy cập, lịch sử phê duyệt, audit log và evidence. Nhờ đó, hệ thống có thể kiểm tra lại quá trình xử lý tài liệu khi cần.

Với DevSecOps, nhóm đã xây dựng được pipeline có các cổng kiểm soát chính và có thể truy vết release qua log pipeline, report scan, image digest, SBOM, GitOps revision, trạng thái Argo CD, smoke test và ZAP report. Các bằng chứng này giúp báo cáo không chỉ nói rằng hệ thống đã được triển khai, mà còn chỉ ra bản triển khai đó đi qua những bước nào.

Trong quá trình thực hiện, nhóm cũng gặp nhiều khó khăn thực tế. Khi chạy local, các service có thể hoạt động tương đối đơn giản, nhưng khi đưa lên Kubernetes thì phát sinh nhiều vấn đề về biến môi trường, secret, kết nối nội bộ giữa các service và trạng thái readiness của workload. Việc cấu hình External Secrets, IRSA, Harbor, Argo CD và các manifest triển khai cũng mất nhiều thời gian để kiểm tra và debug.

Pipeline DevSecOps cũng phát sinh nhiều vấn đề trong quá trình chạy thử. Một số bước quét bảo mật làm pipeline chạy lâu hơn dự kiến, có lúc công cụ scan bị treo hoặc trả về cảnh báo CVE cần xem xét lại image và dependency. Việc xử lý các kết quả này giúp nhóm hiểu rõ hơn rằng DevSecOps không chỉ là thêm nhiều công cụ vào Jenkinsfile, mà còn phải xác định ngưỡng fail, ngoại lệ chấp nhận được trong demo và finding nào cần xử lý trước khi release.

Thông qua đề tài, nhóm học được cách thiết kế một hệ thống microservices có nhiều thành phần phối hợp với nhau, cách tách trách nhiệm giữa metadata-service, document-service, workflow-service và audit-service, cũng như cách triển khai hệ thống lên môi trường Kubernetes. Nhóm cũng hiểu rõ hơn vai trò của audit log, evidence, image digest, SBOM, GitOps revision và các bằng chứng vận hành trong việc kiểm tra lại một hệ thống sau khi triển khai.

Tuy nhiên, hệ thống vẫn còn một số hạn chế. Một số phần như NetworkPolicy chi tiết, admission controller bắt buộc xác minh chữ ký image, provenance chuẩn hóa, kiểm thử trình duyệt đầy đủ, log persistence và diễn tập khôi phục dữ liệu vẫn cần tiếp tục hoàn thiện nếu muốn tiến gần hơn đến môi trường production. Ngoài ra, một số chức năng trong hệ thống hiện vẫn tập trung vào việc chứng minh luồng xử lý chính, chưa tối ưu sâu về hiệu năng, trải nghiệm người dùng và khả năng mở rộng trong môi trường thực tế.

Nhìn chung, kết quả hiện tại đã đáp ứng được mục tiêu của đề án chuyên ngành. DocVault cho thấy cách một tài liệu nhạy cảm có thể được quản lý bằng workflow, policy, audit và evidence, đồng thời cách bản triển khai của hệ thống có thể được truy vết từ commit đến workload. Đây là nền tảng để nhóm tiếp tục mở rộng hệ thống theo hướng ổn định, dễ vận hành và gần production hơn trong tương lai.

PHỤ LỤC A

HƯỚNG DẪN CHẠY HỆ THỐNG

A.1 Yêu cầu môi trường

Môi trường phát triển DocVault cần các công cụ sau:

- Node.js 20 trở lên.
- pnpm 9 trở lên.
- Docker Desktop hoặc Docker Engine kèm Docker Compose.
- Git.
- Các port local trống cho gateway, microservices, frontend, Keycloak, PostgreSQL, MongoDB và MinIO.

A.2 Cài đặt phụ thuộc

Từ thư mục gốc repository, chạy:

```
pnpm install
```

A.3 Khởi động hạ tầng local

Hạ tầng local nằm trong `infra/docker-compose.dev.yml`. Lệnh chạy:

```
docker compose -f infra/docker-compose.dev.yml --env-file infra/.env up -d
```

Các thành phần được khởi động gồm PostgreSQL, MongoDB, MongoDB Express, MinIO, MinIO init job và Keycloak.

A.4 Chạy migration

Sau khi PostgreSQL đã sẵn sàng:

```
pnpm --filter metadata-service prisma:deploy
pnpm --filter audit-service prisma:deploy
```

A.5 Khởi động backend

Khuyến nghị chạy từng service ở terminal riêng theo thứ tự:

```
pnpm --filter metadata-service start:dev
pnpm --filter audit-service start:dev
pnpm --filter document-service start:dev
pnpm --filter notification-service start:dev
pnpm --filter workflow-service start:dev
pnpm --filter gateway start:dev
```

Các service mặc định:

- Gateway: <http://localhost:3000>
- Metadata-service: <http://localhost:3001>
- Document-service: <http://localhost:3002>
- Workflow-service: <http://localhost:3003>
- Audit-service: <http://localhost:3004>
- Notification-service: <http://localhost:3005>

A.6 Khởi động frontend

Frontend chạy trên port 3006 theo script dev của apps/web:

```
pnpm --filter web dev
```

Truy cập:

```
http://localhost:3006
```

A.7 Tài khoản demo

Các tài khoản demo thường dùng:

- viewer1 (vai trò viewer)
- editor1 (vai trò editor)
- approver1 (vai trò approver)
- co1 (vai trò compliance_officer)
- admin1 (vai trò admin)

Mật khẩu demo nằm trong tài liệu hướng dẫn chạy nội bộ của repository và cần được thay đổi nếu triển khai ngoài môi trường dev.

A.8 Kiểm thử nhanh

Sau khi backend hoạt động:

```
pnpm test:e2e
```

Lệnh này kiểm tra các luồng chính: auth boundary, deny viewer, create/upload/submit của editor, approve của approver, download published document, audit của compliance và deny download của compliance.

PHỤ LỤC B

TÓM TẮT API CONTRACT

B.1 Nguyên tắc chung

Frontend gọi toàn bộ API thông qua gateway với base URL:

```
http://localhost:3000/api
```

Các request cần header:

```
Authorization: Bearer <keycloak_jwt>
```

Gateway xác thực JWT, kiểm tra role ở mức route và proxy request xuống service tương ứng. Các endpoint chi tiết tài liệu, workflow history, comments và ACL được policy-filter ở backend.

B.2 Nhóm endpoint chính

Bảng B.1: Tóm tắt endpoint chính

Chức năng	Endpoint	Vai trò chính
Login	GET /auth/login	Public
Current user	GET /auth/me	Authenticated
List documents	GET /metadata/documents	All roles
Create document	POST /metadata/documents	Editor, Admin
Document detail	GET /metadata/documents/:docId	Policy-filtered
Update metadata	PATCH /metadata/documents/:docId	Owner Editor, Admin
ACL list/upsert	GET/POST /metadata/documents/:docId/acl	Policy-filtered, Editor/Admin
Workflow history	GET /metadata/documents/:docId/workflow-history	Policy-filtered
Upload file	POST /documents/:docId/upload	Editor, Admin
Authorize download	POST /metadata/documents/:docId/download-authorize	Policy-filtered

Chức năng	Endpoint	Vai trò chính
Presign download	POST /documents/:docId/presign-download	Policy-filtered
Stream file	GET /documents/:docId/versions/:version/stream	Policy-filtered
Submit	POST /workflow/:docId/submit	Editor, Admin
Approve	POST /workflow/:docId/approve	Approver, Admin
Reject	POST /workflow/:docId/reject	Approver, Admin
Archive	POST /workflow/:docId/archive	Editor, Admin
Audit query	GET /audit/query	Compliance, Admin
Verify chain	GET /audit/verify-chain	Compliance, Admin
Security summary	GET /audit/security-summary	Compliance, Admin
Retention list	GET /metadata/retention/documents	Compliance, Admin
Retention run	POST /metadata/retention/run	Admin
Evidence packet	GET /metadata/documents/:docId/evidence-packet	Compliance, Admin

B.3 Compliance Officer rule

Compliance Officer được phép xem audit, retention, security summary và evidence packet. Tuy nhiên, vai trò này không được preview, stream, presign hoặc download file content. Quy tắc này là một trong các acceptance criteria quan trọng của đề tài.

B.4 Grant token

Download và preview grant token được cấp bởi metadata-service sau khi policy check thành công. Token có thời hạn ngắn và được document-service xác minh trước khi trả nội dung. Thiết kế này giúp tách bước quyết định quyền khỏi bước phát file, đồng thời tránh để frontend tự quyết định quyền.

TÀI LIỆU THAM KHẢO

- [1] Microsoft. “Introduction to SharePoint and OneDrive in Microsoft 365”, visited on June 28, 2026. address: <https://learn.microsoft.com/en-us/sharepoint/introduction>
- [2] Hyland. “Alfresco Content Services: Set Up Authentication and Security”, visited on June 28, 2026. address: <https://docs.alfresco.com/content-services/latest/admin/security/>
- [3] Hyland. “Alfresco Content Services: Audit”, visited on June 28, 2026. address: <https://docs.alfresco.com/content-services/latest/admin/audit/>
- [4] Nextcloud. “Nextcloud Files”, visited on June 28, 2026. address: <https://nextcloud.com/files/>
- [5] Dropbox. “Dropbox for Business”, visited on June 28, 2026. address: <https://www.dropbox.com/business>
- [6] Dropbox. “Dropbox Security”, visited on June 28, 2026. address: <https://www.dropbox.com/security>
- [7] Box. “Using Security Classification”, visited on June 28, 2026. address: <https://support.box.com/hc/en-us/articles/360043696734>
- [8] Box. “Managing Automated Classification Policies”, visited on June 28, 2026. address: <https://support.box.com/hc/en-us/articles/360050901213>
- [9] Box. “Anomalous Download Detection Rule”, visited on June 28, 2026. address: <https://support.box.com/hc/en-us/articles/46976557112851>
- [10] Box. “Malicious Content Detection Rule”, visited on June 28, 2026. address: <https://support.box.com/hc/en-us/articles/46976541627411>
- [11] Keycloak. “Keycloak Documentation”, visited on June 14, 2026. address: <https://www.keycloak.org/documentation>
- [12] OWASP Foundation. “OWASP Application Security Verification Standard (ASVS)”, visited on June 30, 2026. address: <https://owasp.org/www-project-application-security-verification-standard/>
- [13] National Institute of Standards and Technology. “Guide to Computer Security Log Management”, visited on June 30, 2026. address: <https://csrc.nist.gov/pubs/sp/800/92/final>

- [14] National Institute of Standards and Technology. “Security and Privacy Controls for Information Systems and Organizations”, visited on June 30, 2026. address: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>
- [15] OWASP Foundation. “OWASP DevSecOps Guideline: Overview”, visited on June 28, 2026. address: <https://owasp.org/www-project-devsecops-guideline/latest/00a-Overview>
- [16] National Institute of Standards and Technology. “Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities”, visited on June 14, 2026. address: <https://csrc.nist.gov/pubs/sp/800/218/final>
- [17] SLSA Project. “SLSA Specification Version 1.2”, visited on June 30, 2026. address: <https://slsa.dev/spec/v1.2/>
- [18] Argo CD Project. “Argo CD: Declarative GitOps CD for Kubernetes”, visited on June 14, 2026. address: <https://argo-cd.readthedocs.io/en/stable/>
- [19] Cloud Native Computing Foundation. “Solving Configuration Drift Using GitOps with Argo CD”, visited on June 28, 2026. address: <https://www.cncf.io/blog/2020/12/17/solving-configuration-drift-using-gitops-with-argo-cd/>
- [20] Harbor Project. “Working with Images and Tags”, visited on June 25, 2026. address: <https://goharbor.io/docs/main/working-with-projects/working-with-images/>
- [21] SonarSource. “SonarQube Server Documentation”, visited on June 14, 2026. address: <https://docs.sonarsource.com/sonarqube-server>
- [22] Aqua Security. “Trivy Documentation”, visited on June 14, 2026. address: <https://trivy.dev/docs/latest/guide/>
- [23] ZAP by Checkmarx. “ZAP Baseline Scan”, visited on June 14, 2026. address: <https://www.zaproxy.org/docs/docker/baseline-scan/>
- [24] Amazon Web Services. “IAM Roles for Service Accounts”, visited on June 25, 2026. address: <https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html>
- [25] Amazon Web Services. “Using Server-Side Encryption with AWS KMS Keys”, visited on June 25, 2026. address: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html>
- [26] HashiCorp. “Backend Block Configuration Overview”, visited on June 25, 2026. address: <https://developer.hashicorp.com/terraform/language/backend>
- [27] Kubernetes Authors. “Auditing”, visited on June 30, 2026. address: <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>

- [28] Kubernetes Authors. “Security Checklist”, visited on June 30, 2026. address: <https://kubernetes.io/docs/concepts/security/security-checklist/>
- [29] CloudNativePG Project. “Backup and Recovery”, visited on June 25, 2026. address: <https://cloudnative-pg.io/documentation/current/backup/>
- [30] Percona. “Percona Backup for MongoDB Documentation”, visited on June 25, 2026. address: <https://docs.percona.com/percona-backup-mongodb/index.html>
- [31] Kubernetes Authors. “Pod Security Standards”, visited on June 30, 2026. address: <https://kubernetes.io/docs/concepts/security/pod-security-standards/>
- [32] Kubernetes Authors. “Network Policies”, visited on June 25, 2026. address: <https://kubernetes.io/docs/concepts/services-networking/network-policies/>
- [33] External Secrets Operator Project. “AWS Secrets Manager Provider”, visited on June 25, 2026. address: <https://external-secrets.io/latest/provider/aws-secrets-manager/>
- [34] Sigstore Project. “Signing Containers with Cosign”, visited on June 25, 2026. address: https://docs.sigstore.dev/cosign/signing/signing_with_containers/
- [35] Kyverno Project. “Kyverno CLI Reference”, visited on June 25, 2026. address: <https://kyverno.io/docs/kyverno-cli/>
- [36] Checkov Project. “Checkov Documentation”, visited on June 14, 2026. address: <https://www.checkov.io/1.Welcome/What%20is%20Checkov.html>
- [37] Prometheus Authors. “Prometheus Overview”, visited on June 25, 2026. address: <https://prometheus.io/docs/introduction/overview/>
- [38] Grafana Labs. “Grafana Loki Overview”, visited on June 25, 2026. address: <https://grafana.com/docs/loki/latest/get-started/overview/>